CSE 270 Sports Analytics

Network analysis

Habet Madoyan

Network analysis

Libraries used

```
library(igraph)
library(intergraph)
library(network)
library(sna)
library(statnet)
library(statnetWeb)
library(circlize)
library(SportsAnalytics270)
library(ggplot2)
```

Introduction

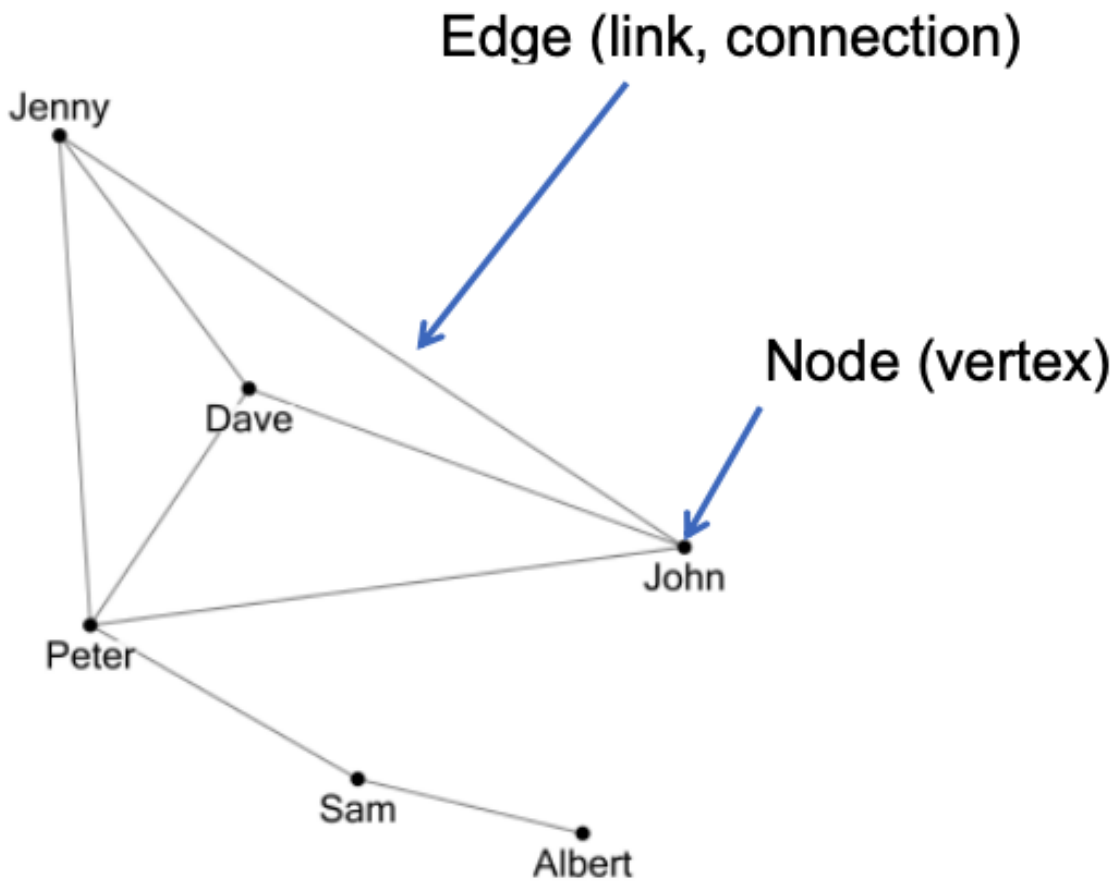
Networks are basically entities (peoples, players, clubs, magazines, countries, etc) and the connections among them.

The basic elements of the networks are:

- Nodes (also called vertices or vertexes)
- Edges (connections or links between them)

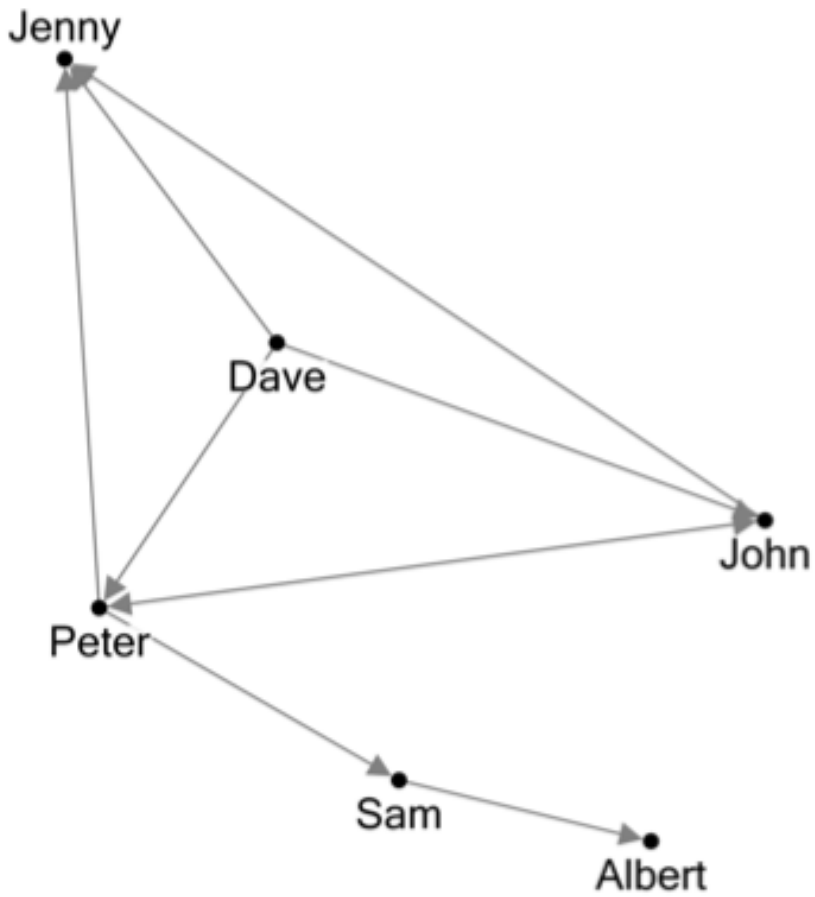
Example: social network

Facebook: undirected network



Example: social network

Twitter: Directed network



Installing necessary packages

We will use 3 main packages for the network analsysis for creating and manipulating graph objects

- igraph
- network

For changing the class of objects from network to igraph and vice versa

- intergraph

```
install.packages("igraph")
install.packages("network")
install.packages("intergraph")
```

Tools for social network analysis

- sna - A range of tools for social network analysis
- statnet: metapackage statnet: installs everything you need
- statnetweb: r shiny application for network analysis

```
install.packages("sna")
install.packages("statnet")
install.packages("statnetWeb")
```

Types of data stored in the network data object

Type	Description	Required?
Nodes	List of nodes in network, along with node labels	Required
Ties	List of ties in the network	Required
Node attributes	Attributes of the nodes	Optional
Tie attributes	Attributes of the ties	Optional
Metadata	Other information about the entire network	Depends

Input data

There are two main types of data structures that are used for network analysis.

- Adjacency matrix (sociomatrix)
- Edge list

Adjacency matrix

An adjacency matrix is a square matrix where a 1 indicates a tie between two nodes, and a 0 indicates no tie.

The convention is that rows indicate the starting node, and columns indicate the receiving node

```
mat <- rbind(c(0,1,1,0,0),c(0,0,1,1,0),c(0,1,0,0,0),
             c(0,0,0,0,0),c(0,0,1,0,0))
rownames(mat) <- c("A","B","C","D","E")
colnames(mat) <- c("A","B","C","D","E")
mat
```

```
##   A B C D E
## A 0 1 1 0 0
## B 0 0 1 1 0
## C 0 1 0 0 0
## D 0 0 0 0 0
## E 0 0 1 0 0
```

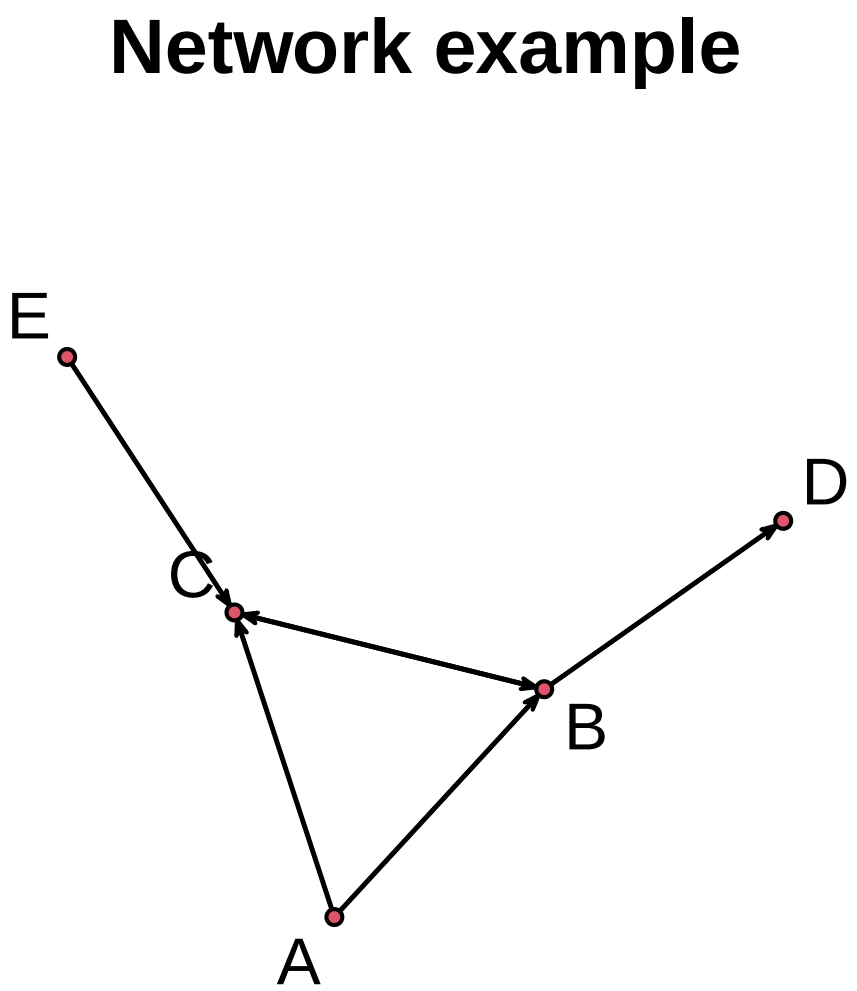
Adjacency matrix

```
net<-network(mat, matrix.type="adjacency")
summary(net)
```

```
## Network attributes:
##   vertices = 5
##   directed = TRUE
##   hyper = FALSE
##   loops = FALSE
##   multiple = FALSE
##   bipartite = FALSE
##  total edges = 6
##    missing edges = 0
##    non-missing edges = 6
##  density = 0.3
##
## Vertex attributes:
##   vertex.names:
##    character valued attribute
##    5 valid vertex names
##
## No edge attributes
##
## Network adjacency matrix:
##   A B C D E
## A 0 1 1 0 0
## B 0 0 1 1 0
## C 0 1 0 0 0
## D 0 0 0 0 0
## E 0 0 1 0 0
```

Plotting

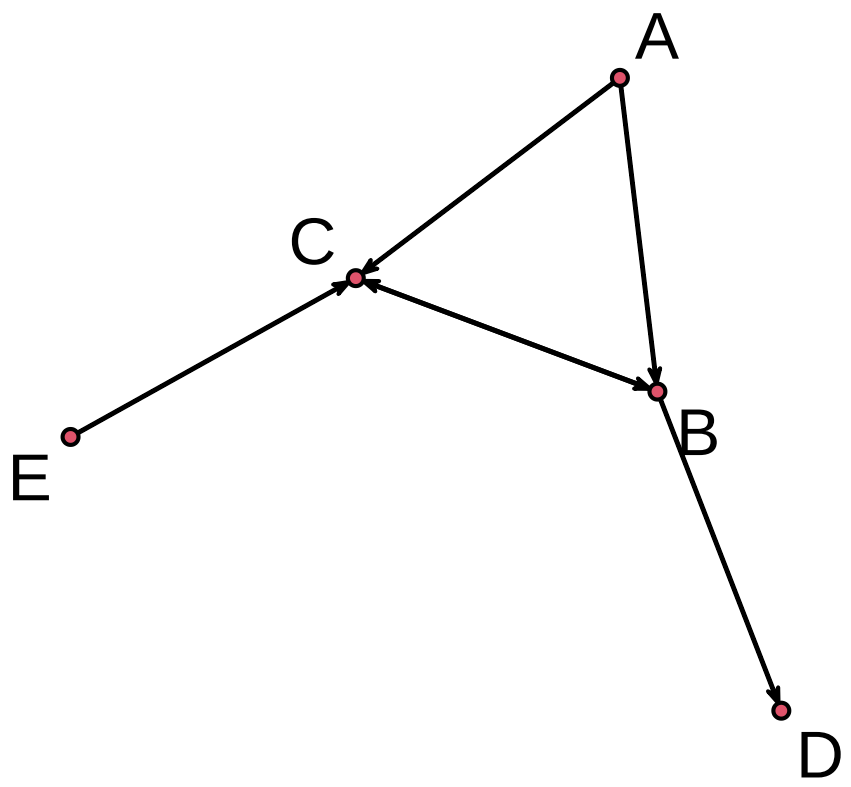
```
plot(net, main="Network example", displaylabels=T)
```



Plotting

Add the vertex labels with ***label=network.vertex.names()***

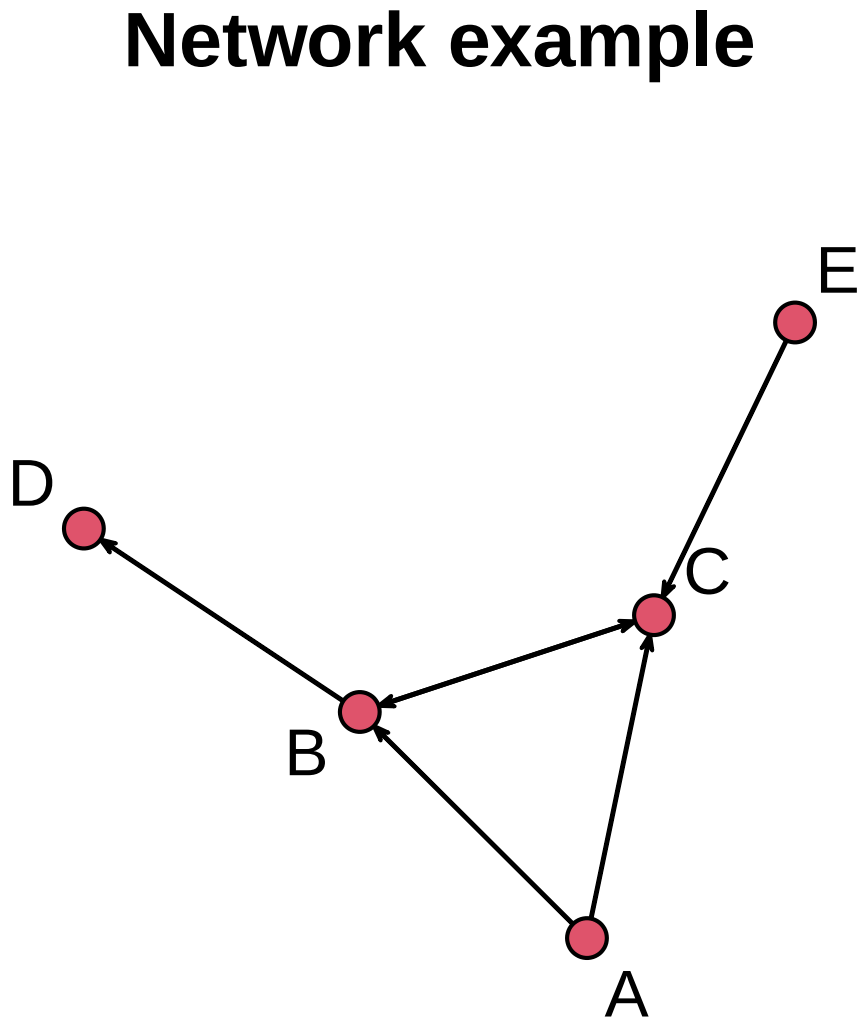
```
plot(net, main="Network example", label=network.vertex.names(net))
```



Plotting

You can increase the vertex size by the parameter ***vertex.cex***

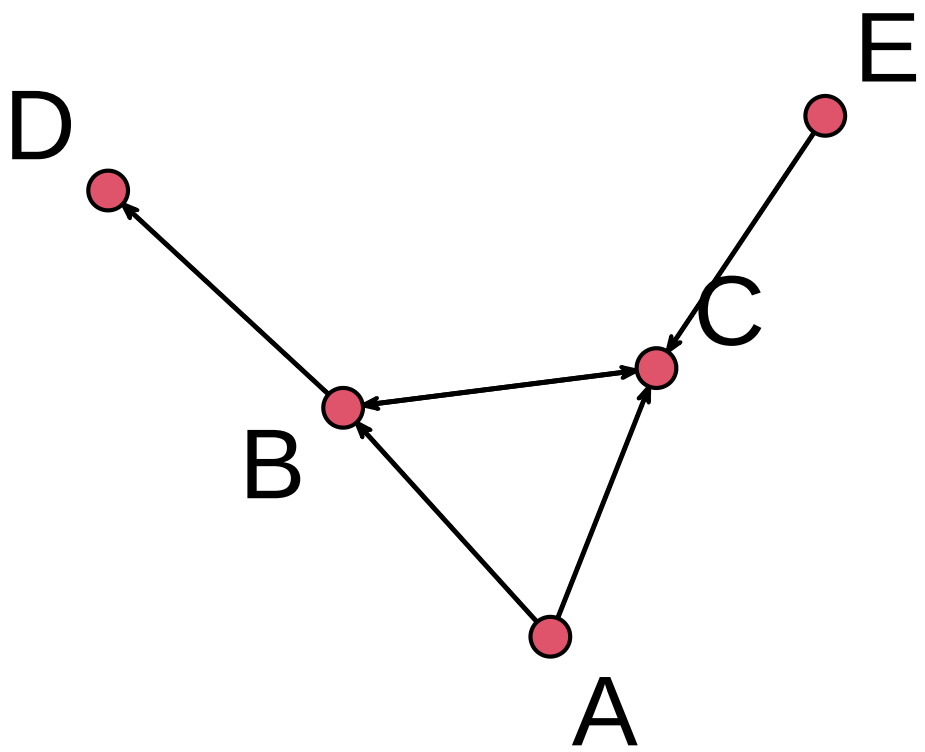
```
plot(net, main="Network example", label=network.vertex.names(net), vertex.cex=2.5)
```



Plotting

Increase the labels size with ***label.cex***

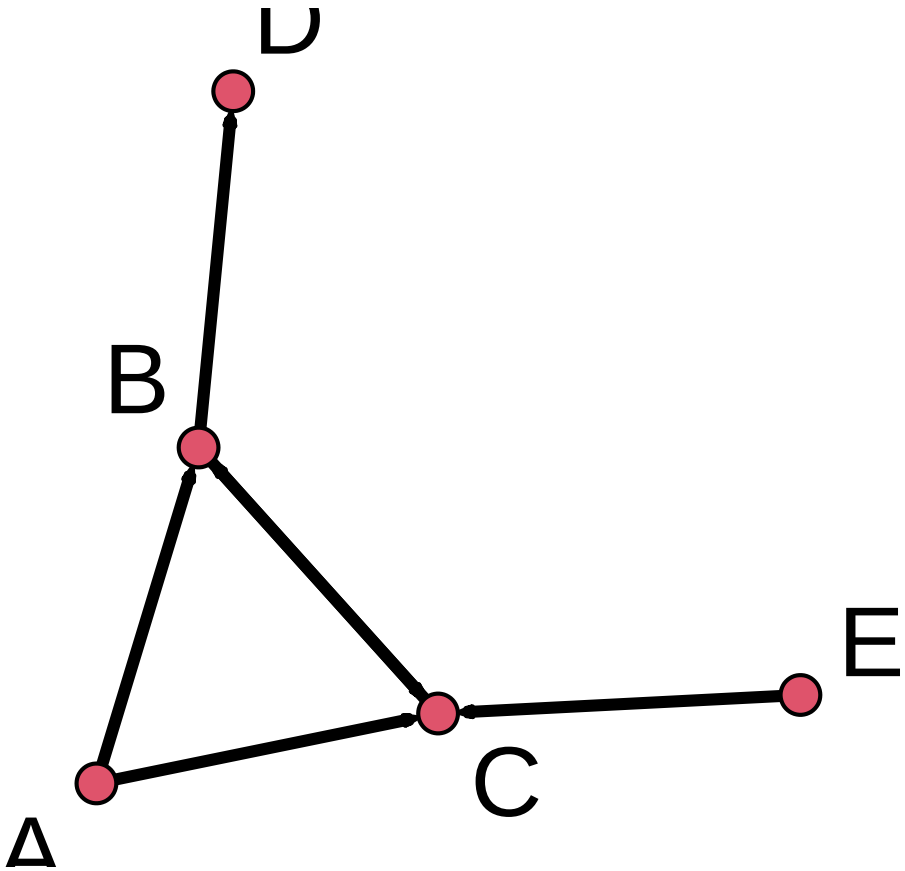
```
plot(net, main="Network example",
     label=network.vertex.names(net), vertex.cex=2.5, label.cex=1.5)
```



Plotting

Increase the edge size with ***edge.lwd***

```
plot(net, main="Network example",
     label=network.vertex.names(net), vertex.cex=2.5, label.cex=1.5, edge.lwd=10)
```



Edge list

Edge list is a form of representing the network data, where each node is a tie/edge, with two columns specifying the FROM and TO nodes

Edge list for the previous example

```
mat1<-cbind(FROM=c("A", "A", "B", "B", "C", "E"),
            TO=c("B", "C", "C", "D", "B", "C"))
mat1
```

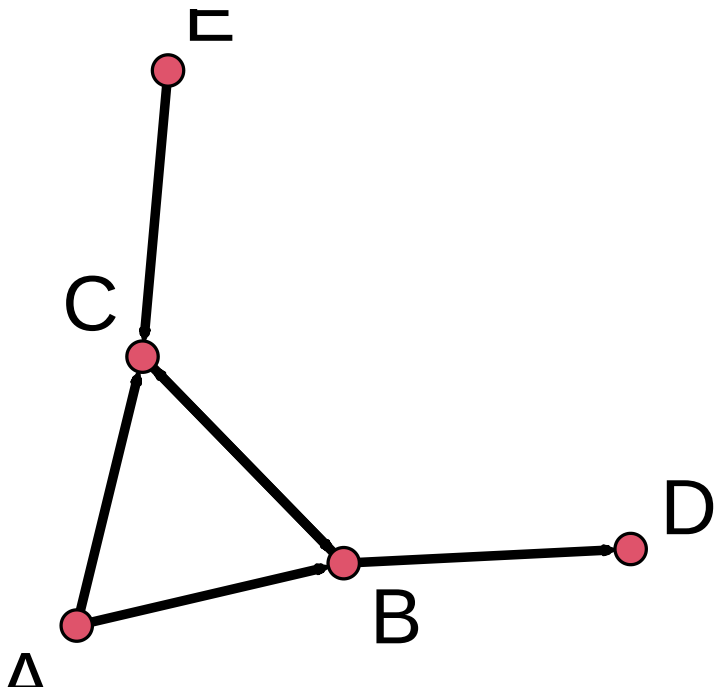
```
##      FROM TO
## [1,] "A"  "B"
## [2,] "A"  "C"
## [3,] "B"  "C"
## [4,] "B"  "D"
## [5,] "C"  "B"
## [6,] "E"  "C"
```

Edge list

Specify ***matrix.type="edgelist"***

```
net1<-network(mat1, matrix.type="edgelist")
```

```
plot(net1, main="Network example",
     label=network.vertex.names(net), vertex.cex=2.5, label.cex=1.5, edge.lwd=10)
```



With igraph

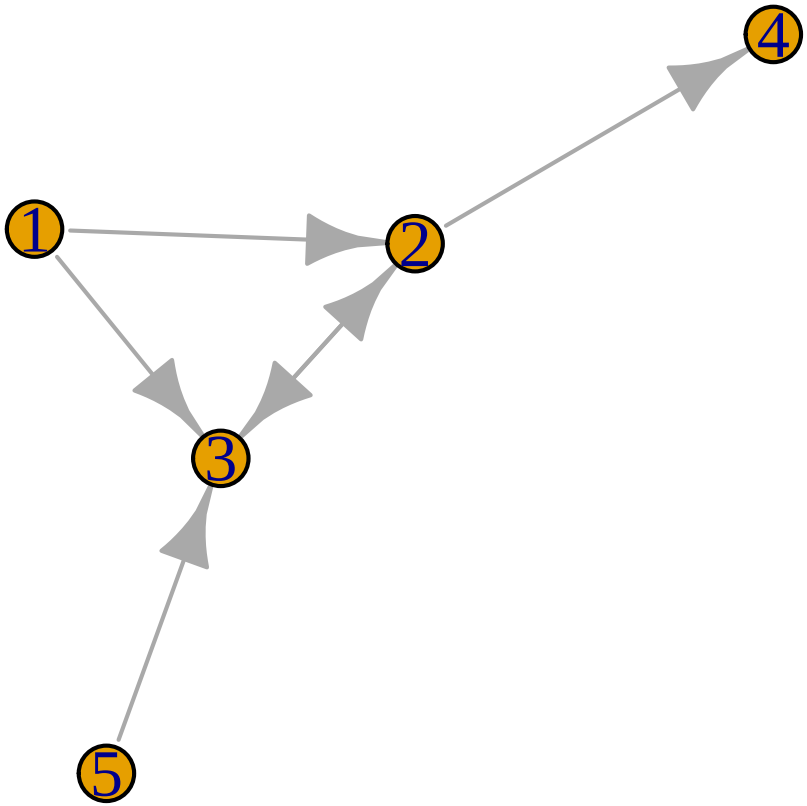
We can also get the edge list from the network object

```
(net_ed<-as.matrix(net,matrix.type = "edgelist"))
```

```
##      [,1] [,2]
## [1,]    1    2
## [2,]    3    2
## [3,]    1    3
## [4,]    2    3
## [5,]    5    3
## [6,]    2    4
## attr(,"n")
## [1] 5
## attr(,"vnames")
## [1] "A" "B" "C" "D" "E"
```

Using igraph

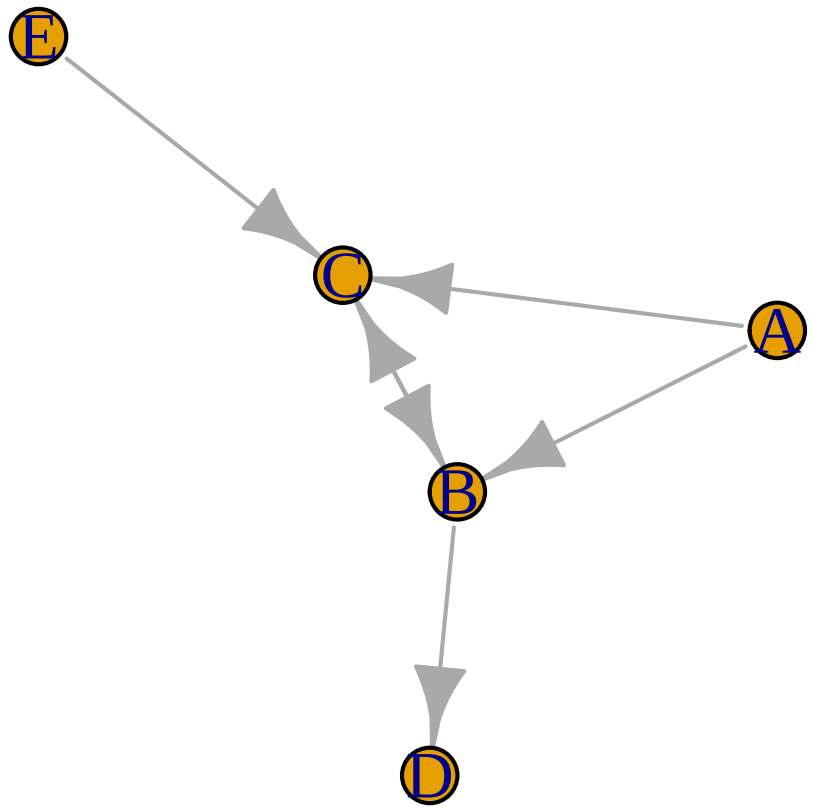
```
net3<-igraph::graph_from_edgelist(net_ed)
plot(net3)
```



With igraph

Create graph using adjacency matrix with igraph

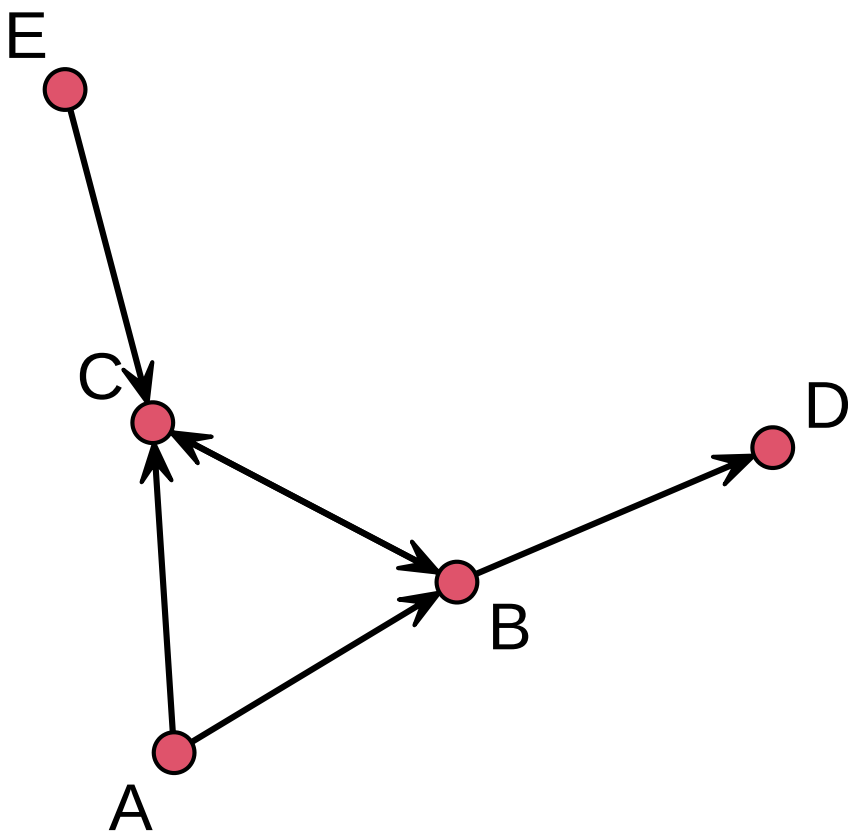
```
net4<-graph_from_adjacency_matrix(mat)
plot(net4)
```



Vizualizing with gplot

gplot comes from the package sna

```
sna::gplot(net1, displaylabels = T)
```



Converting from igraph to network and vice versa

Use package Intergraph to convert types of objects from igraph to networka nd vice versa

```
net_i<-intergraph::asIgraph(net1)
class(net_i)
```

```
## [1] "igraph"
```

vice versa

```
net_n<-intergraph::asNetwork(net_i)
class(net_n)
```

```
## [1] "network"
```

Adding vertex attributes with network package

```
## object from network package
network::set.vertex.attribute(net1, "gender", c("F", "F", "M", "F", "M"))
summary(net1)
```

```
## Network attributes:
##   vertices = 5
##   directed = TRUE
##   hyper = FALSE
##   loops = FALSE
##   multiple = FALSE
##   bipartite = FALSE
##  total edges = 6
##    missing edges = 0
##    non-missing edges = 6
##  density = 0.3
##
## Vertex attributes:
##
##  gender:
##    character valued attribute
##    attribute summary:
## F M
## 3 2
##   vertex.names:
##    character valued attribute
##    5 valid vertex names
##
## No edge attributes
##
## Network adjacency matrix:
##   A B C D E
## A 0 1 1 0 0
## B 0 0 1 1 0
## C 0 1 0 0 0
## D 0 0 0 0 0
## E 0 0 1 0 0
```

Adding vertex attributes with network package

```
network::get.vertex.attribute(net1, "gender")
```

```
## [1] "F" "F" "M" "F" "M"
```

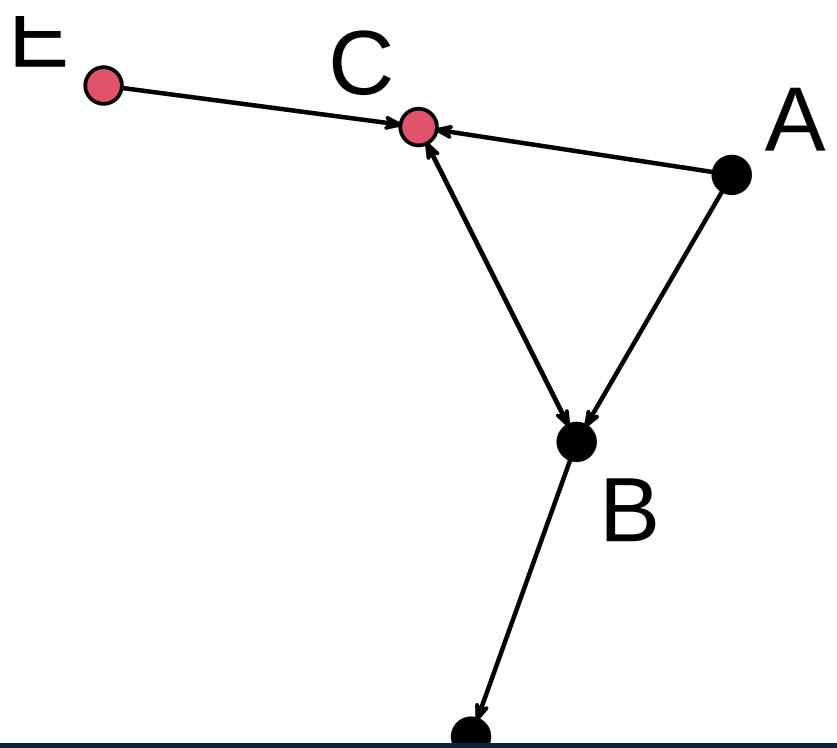
```
## The same
net1 %v% "gender"
```

```
## [1] "F" "F" "M" "F" "M"
```

Vertex attributes

Plot by coloring with network attribute

```
plot(net1,label=network.vertex.names(net1),
     vertex.cex=2.5, label.cex=1.5,
     main="Sample graph with node attributes",
     vertex.col="gender")
```



Adding edge attribute

```
network::set.edge.attribute(net1, "edge.size", c(3,0,5,2,6))
## Check if added
network::get.edge.attribute(net1, "edge.size")
```

```
## [1] 3 0 5 2 6 3
```

```
## the same
net1 %e% "edge.size"
```

```
## [1] 3 0 5 2 6 3
```

Looking what is inside: List node and edge attributes

```
network::list.vertex.attributes(net1)
```

```
## [1] "gender"       "na"           "vertex.names"
```

```
network::list.edge.attributes(net1)
```

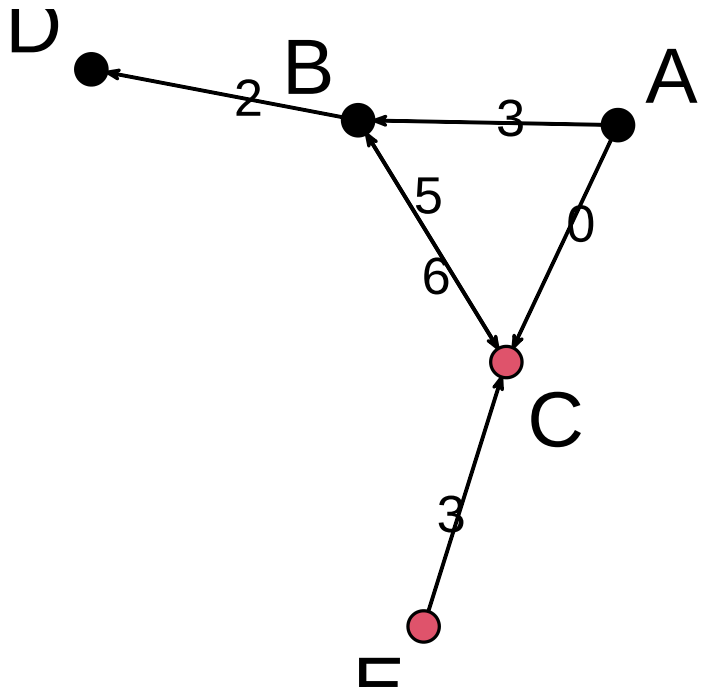
```
## [1] "edge.size" "na"
```

Plotting Edge Attributes

Edge attribute as an edge label, ***edge.label="edge.size"***

```
plot(net1,label=network.vertex.names(net1),
     vertex.cex=2.5, label.cex=1.5,
     main="Sample graph with node attributes",
     vertex.col="gender", edge.label="edge.size")
```

Sample graph with node attributes

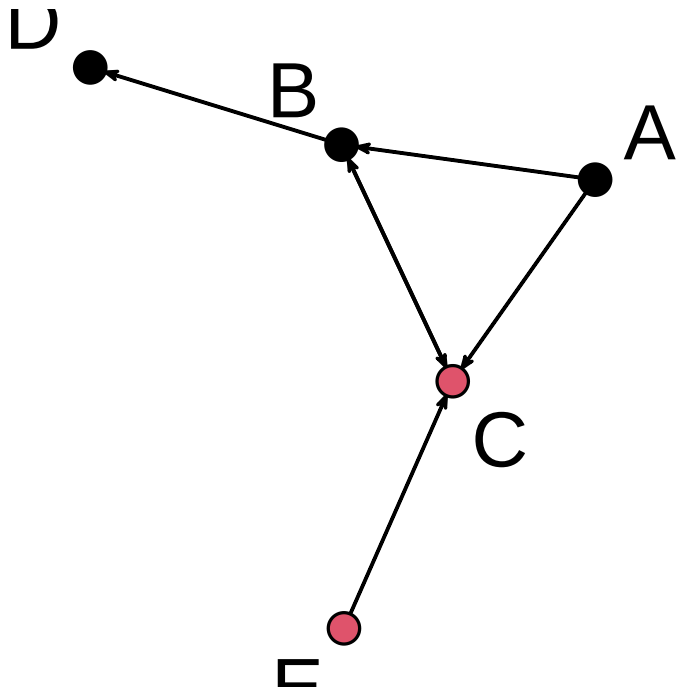


Plotting Edge attribute

Edge attribute is used to set edge size, with ***edge.lwd="edge.size"***

```
plot(net1,label=network.vertex.names(net1),
     vertex.cex=2.5, label.cex=1.5,
     main="Sample graph with node attributes",
     vertex.col="gender")
```

Sample graph with node attributes



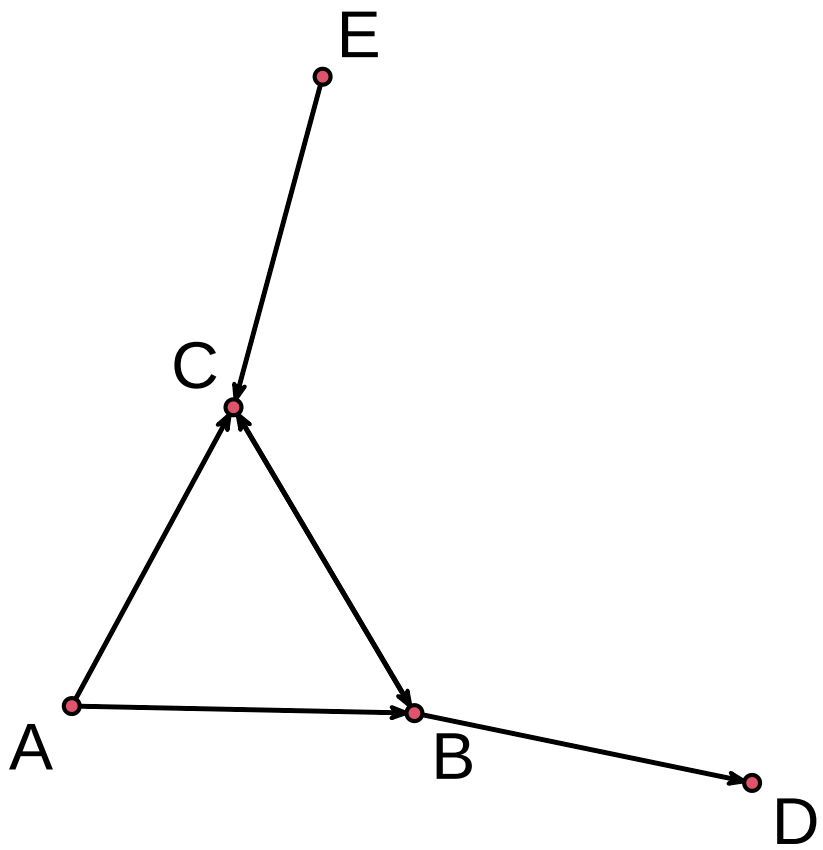
Graph layout

- The graph layout (the nodes coordinates) are drawn randomly
- So every time you make the same graph, you are going to get different picture Solutions (if it is a problem at all)
- use set.seed() to get the same graph
- Save the coordinates
- Use predefined layouts

Graph layout

use set.seed() when running the plot command

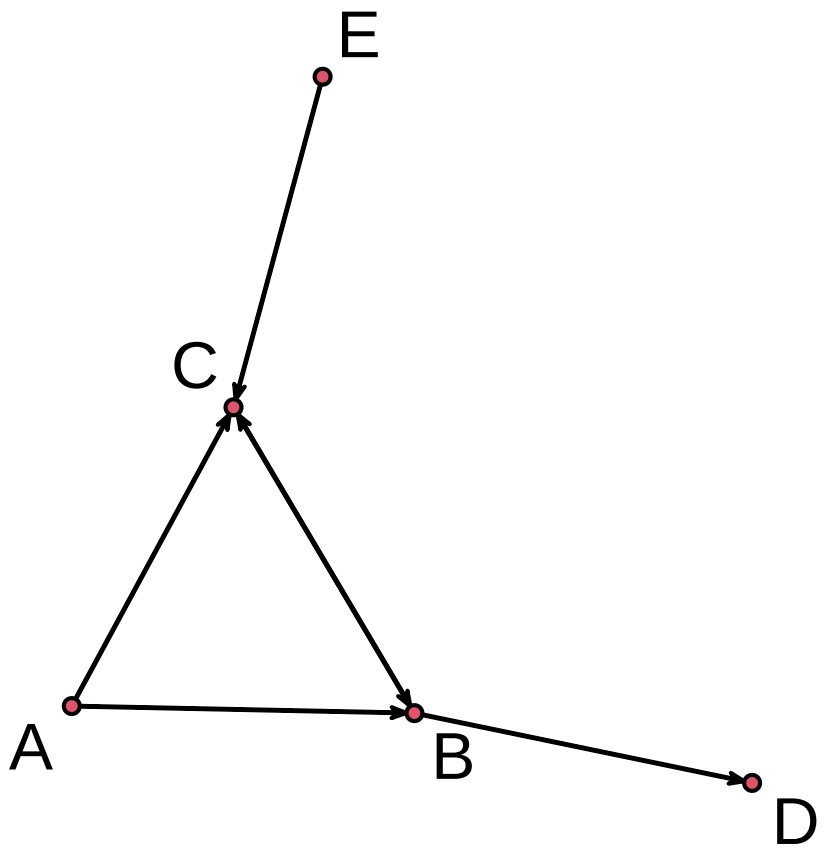
```
set.seed(1)
plot(net1,label=network.vertex.names(net1))
```



Graph layout

Make the coordinates matrix

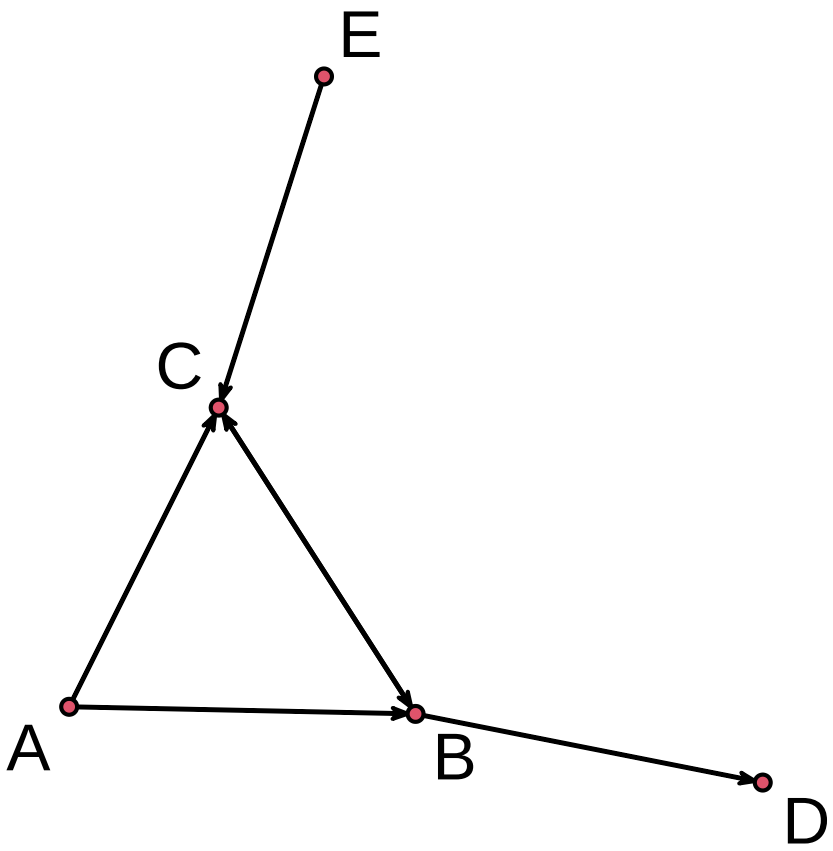
```
set.seed(1)
my_coord<-plot(net1,displaylabels=T)
```



Graph layout

Use argument coord

```
plot(net1,label=network.vertex.names(net1), coord=my_coord)
```

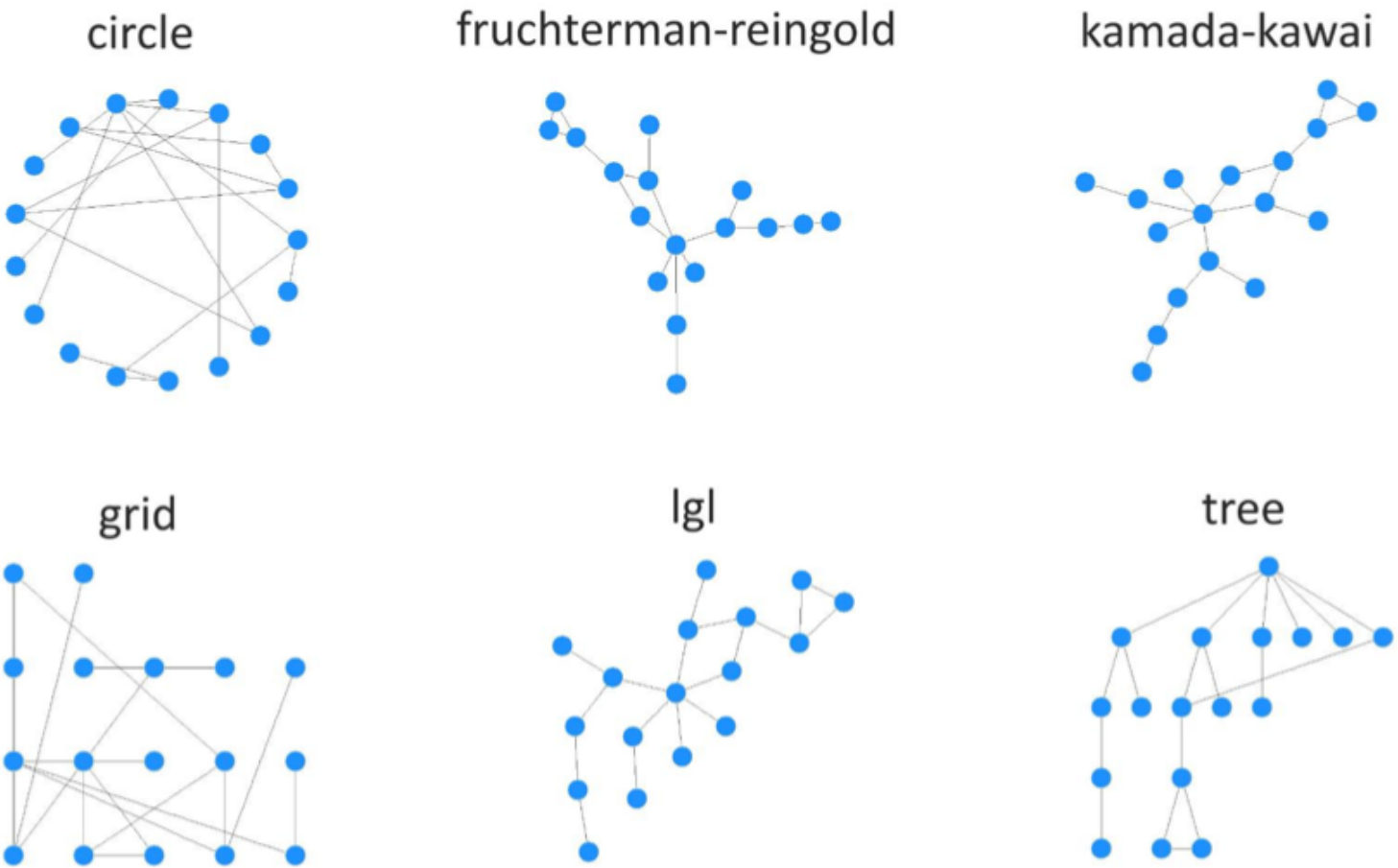


Graph layout

You can use also predefined layouts, such as

- Circle
- Fruchterman Reingold
- Kamada-Kawai
- Random

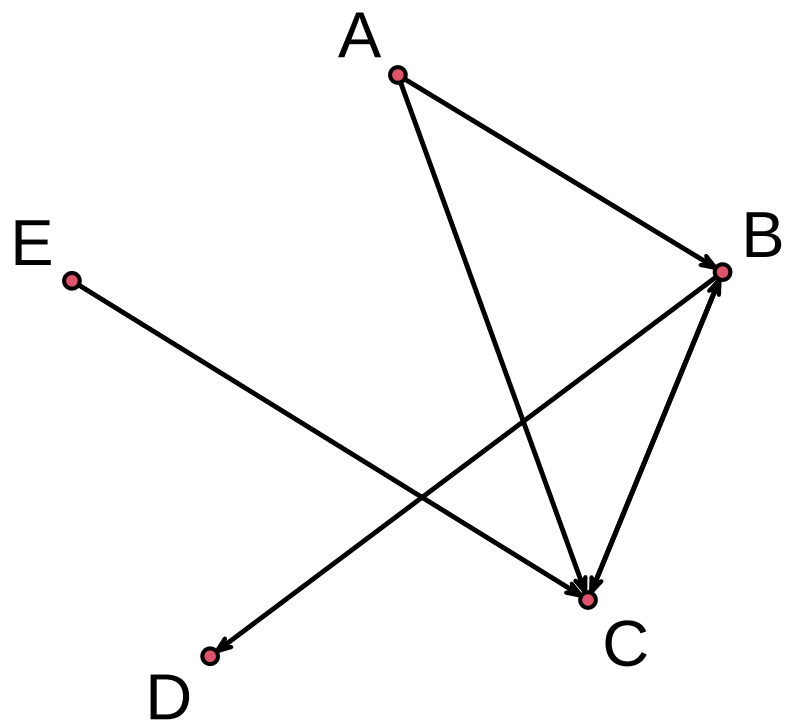
Graph layout



Graph layout: network package

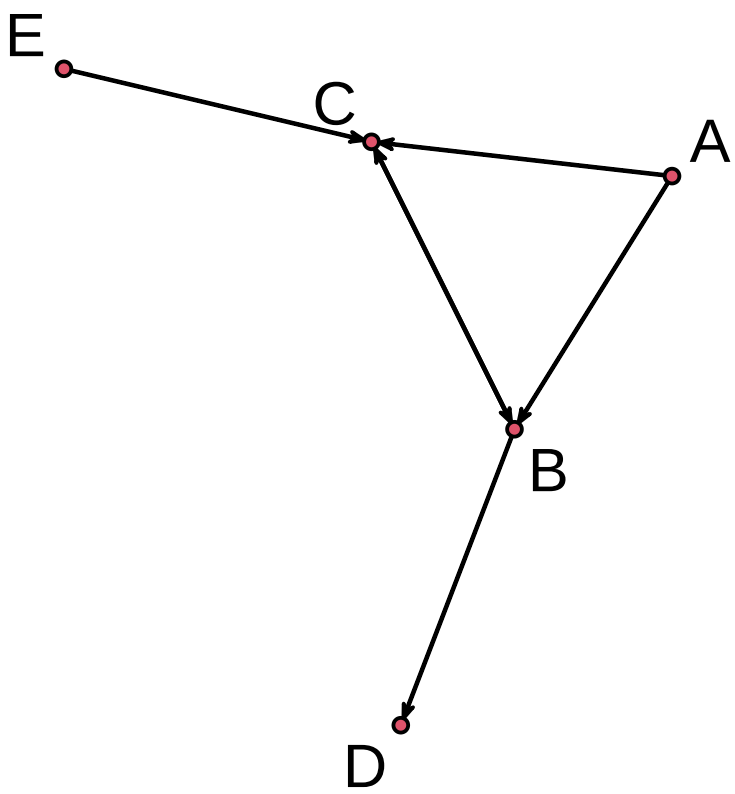
Set argument ***mode="circle"***

```
plot(net1,label=network.vertex.names(net1),
     mode="circle", main="Circle Layout")
```



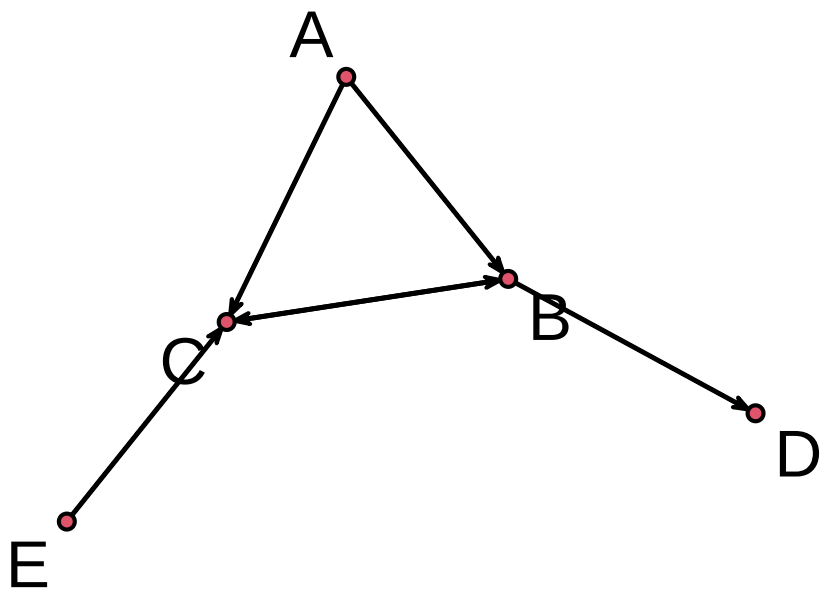
Graph layout

```
plot(net1,label=network.vertex.names(net1),
     mode="fruchtermanreingold",
     main="Fruchterman - Reingold: The default layout")
```



Graph layout

```
plot(net1,label=network.vertex.names(net1),
     mode="kamadakawai", main="Kamada-kawai")
```



Graph layout: igraph

With igraph package

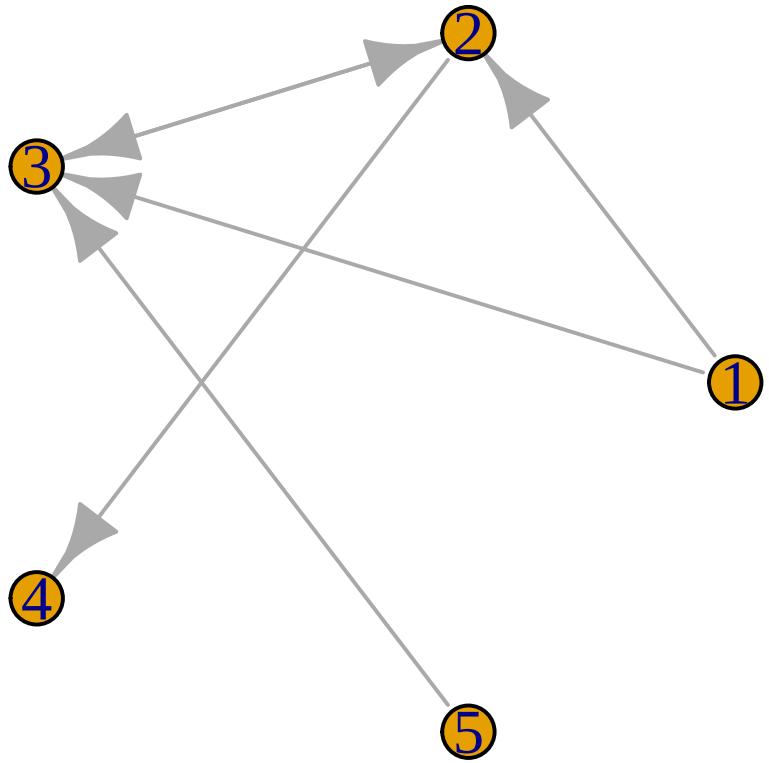
```
net5<-asIgraph(net1)
coords<-layout_in_circle(net5)
coords
```

```
##            [,1]       [,2]
## [1,]  1.000000  0.0000000
## [2,]  0.309017  0.9510565
## [3,] -0.809017  0.5877853
## [4,] -0.809017 -0.5877853
## [5,]  0.309017 -0.9510565
```

Note: Coordinates are generated randomly, so if you don't specify set.seed() you are going to have different results every time

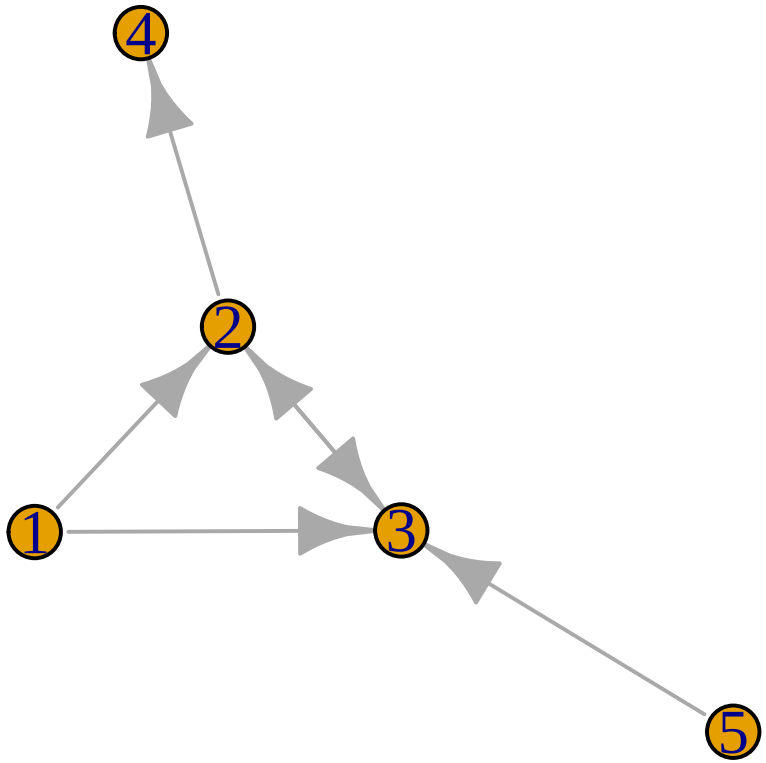
Graph layout

```
plot(net5, layout=coords, main="Circle Layout")
```



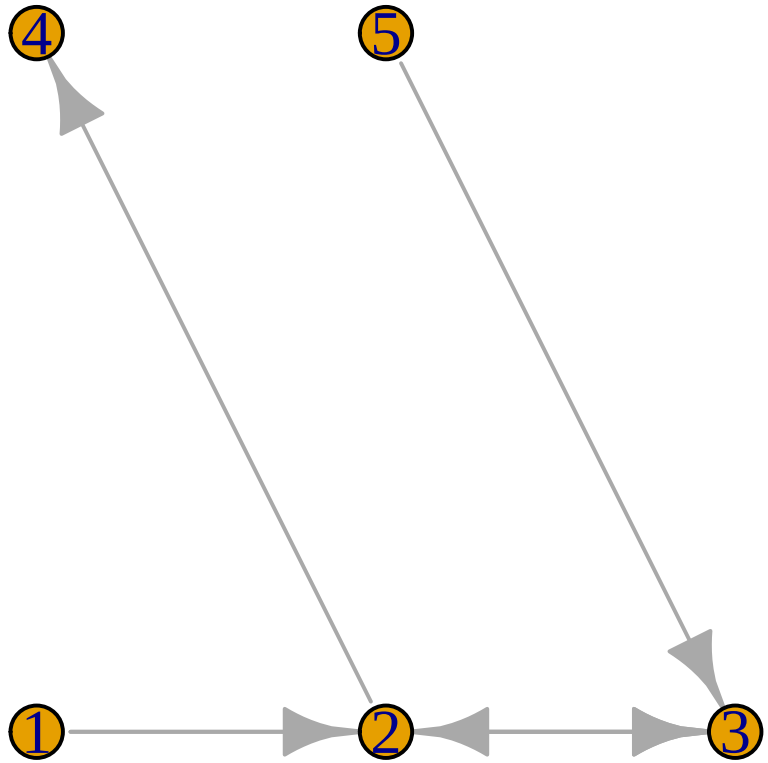
Graph layout

```
plot(net5, layout=layout_with_fr(net5), main="Fruchterman - Reingold")
```



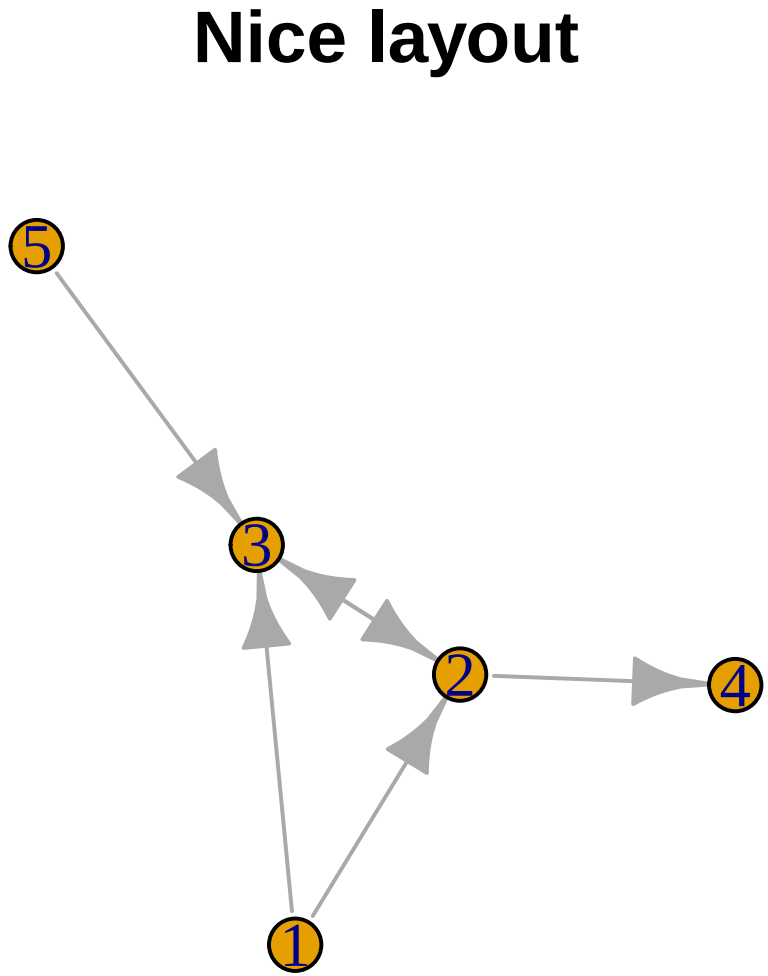
Graph layout

```
plot(net5, layout=layout_on_grid(net5), main="Grid layout")
```



Graph layout

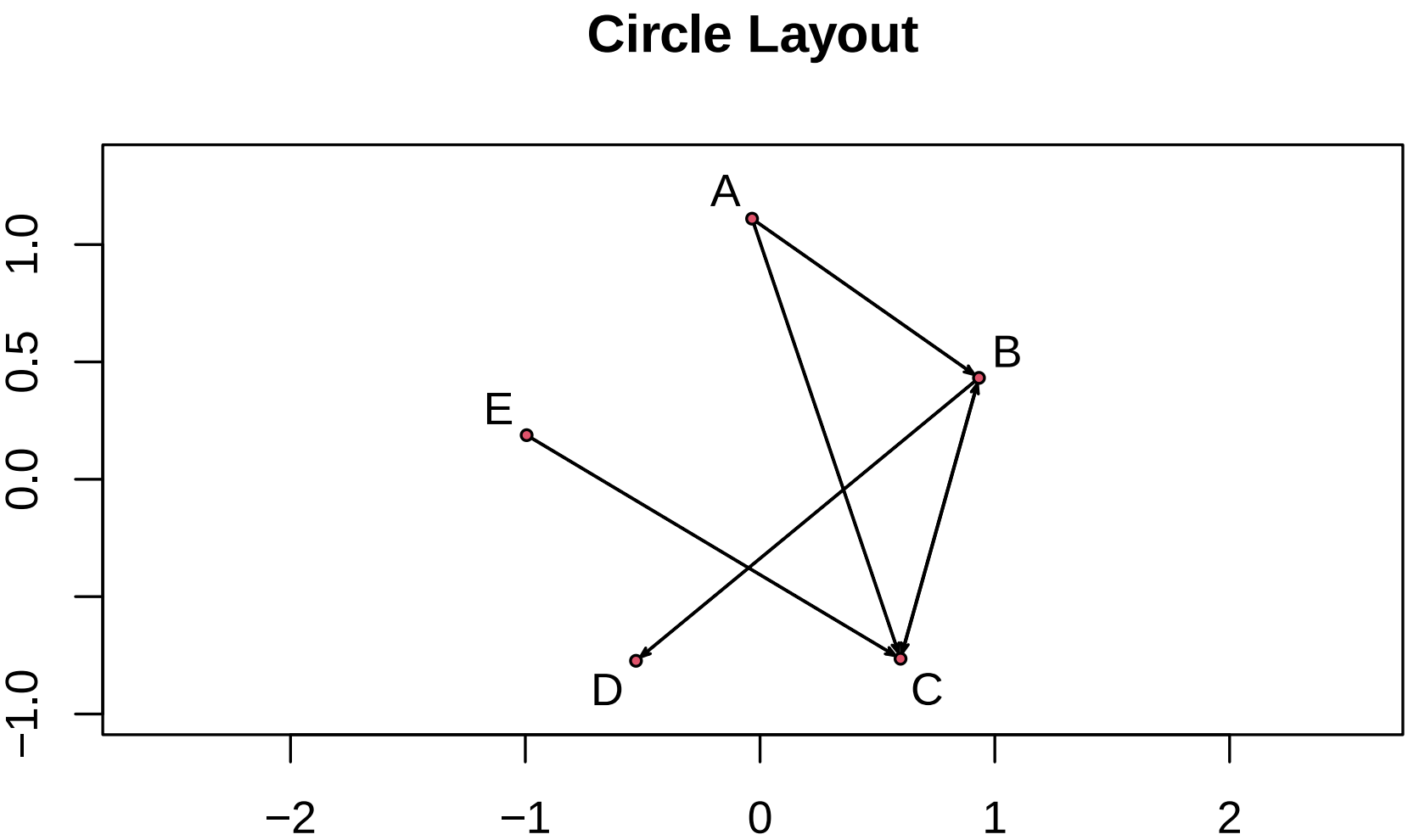
```
plot(net5, layout=layout_nicely(net5), main="Nice layout")
```



Graphing with coordinates

Altering the coordinates

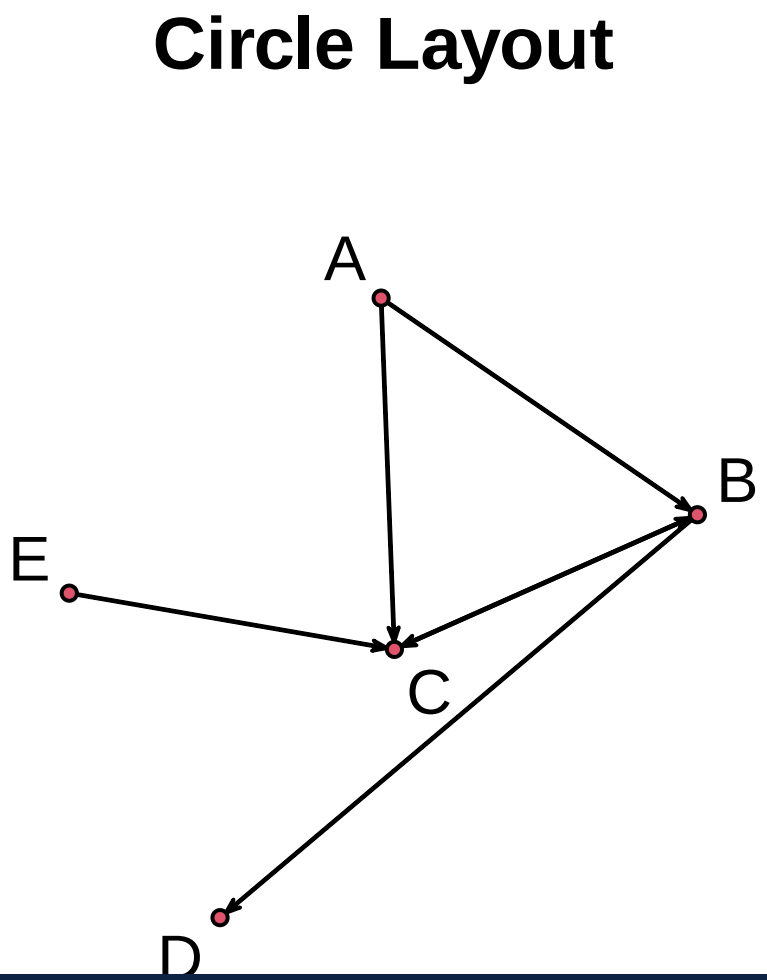
```
set.seed(1)
coord1<-plot(net1,displaylabels=T, mode="circle", main="Circle Layout", suppress.axe
```



Altering the coordinates

Lets say you want to bring Node C to the center

```
#C is the third node, set coordinates to zero
coord1[3,]<-0
plot(net1,displaylabels=T, mode="circle", main="Circle Layout", coord=coord1)
```

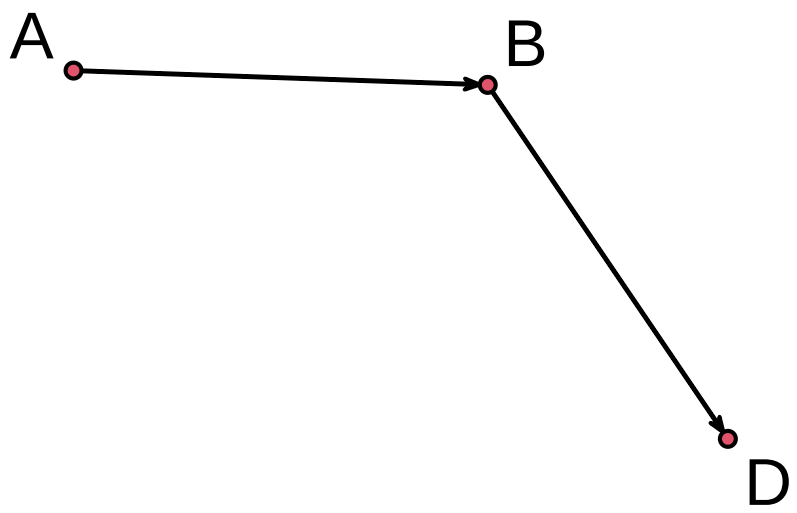


Sub setting the graph: by vertex attribute

```
net1 %v% "gender"
```

```
## [1] "F" "F" "M" "F" "M"
```

```
net_f<-get.inducedSubgraph(net1, v=which(net1 %v% "gender" == "F"))
plot(net_f, label=network.vertex.names(net_f))
```

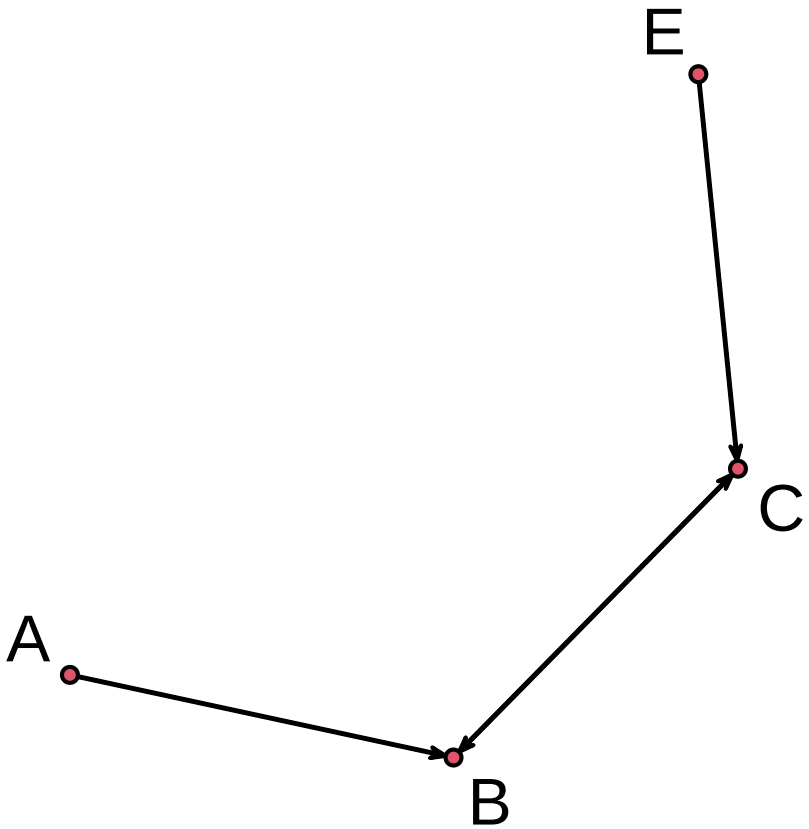


Sub setting the graph: by edge attribute

```
net1 %e% "edge.size"
```

```
## [1] 3 0 5 2 6 3
```

```
## get the edges for which the edge.size is >= 3
net_e<-get.inducedSubgraph(net1, eid=which(net1 %e% "edge.size" >=3))
plot(net_e, label=network.vertex.names(net_e))
```



Accessing network information with igraph

For vertex attributes

```
class(net4)
```

```
## [1] "igraph"
```

```
net4<-set_vertex_attr(net4,"gender", value=c("F", "F", "M",
"F", "M"))
## Accesing the vertex attribute
V(net4)$gender
```

```
## [1] "F" "F" "M" "F" "M"
```

```
V(net4)$name
```

```
## [1] "A" "B" "C" "D" "E"
```

Altering vertex attributes with igraph

Setting edge attributes

```
## altering the values of attributes
## change the first element of the gender attribute to M
V(net4)$gender[1]<-"M"
V(net4)$gender
```

```
## [1] "M" "F" "M" "F" "M"
```

```
net4<-set_edge_attr(net4,"edge.val", value=c(5, 0, 3,
1, 0, 3))
E(net4)$edge.val
```

```
## [1] 5 0 3 1 0 3
```

Marvel Network

Marvel Network



Marvel network

Number of common characters in the movies

row 1: Ant Man and Avengers Age of Ultron have 2 common characters

```
marvel <- read.csv("./data/marvel.csv")
head(marvel)
```

```
##        V1                      V2 V3
## 1 Ant_Man     Ave_Age_Of_Ultron  2
## 2 Ant_Man          CA_Civil_War  4
## 3 Ant_Man  CA_The_First_Avenger  2
## 4 Ant_Man CA_The_Winter_Soldier  2
## 5 Ant_Man              Iron_Man  1
## 6 Ant_Man            Iron_Man_2  1
```

Marvel network

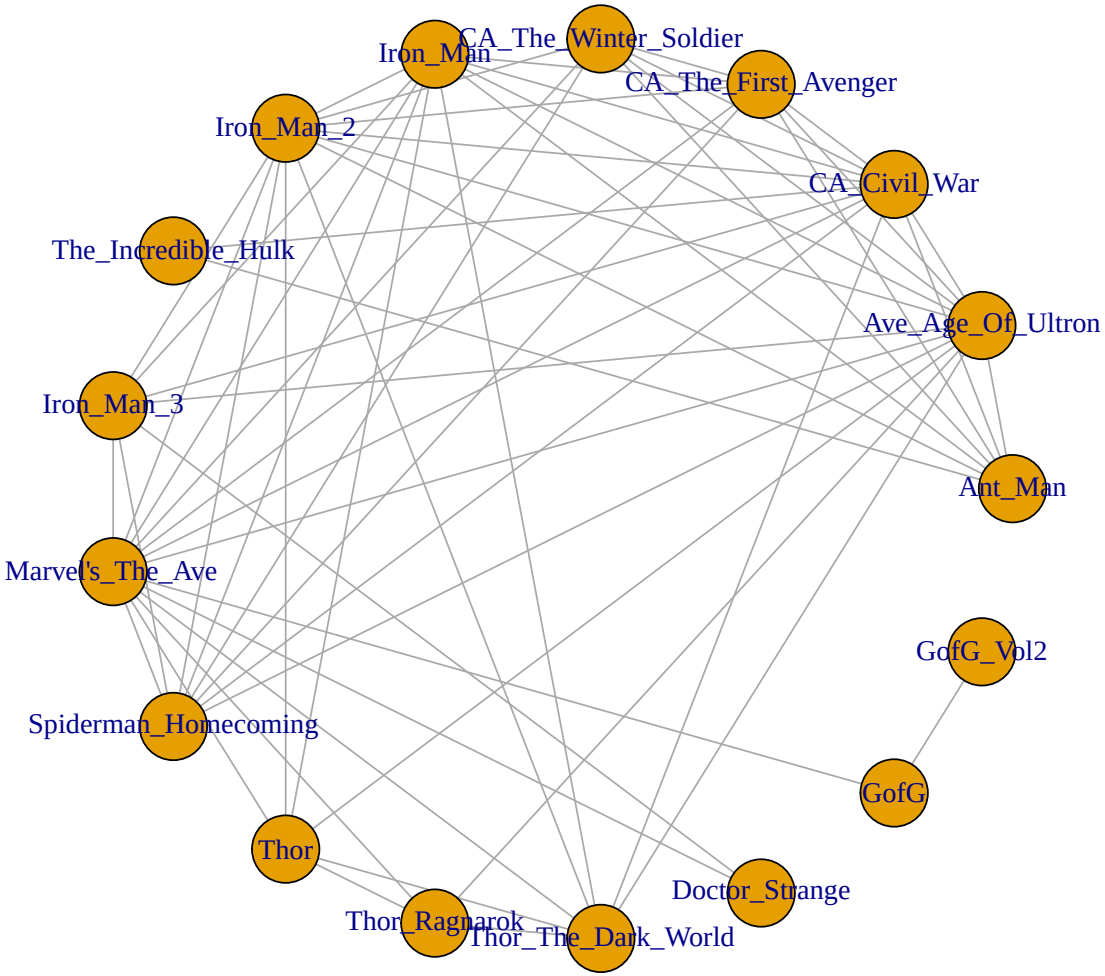
- We will create a network where movies are going to be the vertices and the edges are going to show if two movies have common characters.
- First create a network/igraph object

```
m_igraph <- igraph::graph_from_edgelist(as.matrix(marvel[,1:2]), directed = F)
m_igraph <- set_edge_attr(m_igraph, "characters", value = marvel[,3])
class(m_igraph)
```

```
## [1] "igraph"
```

Marvel network

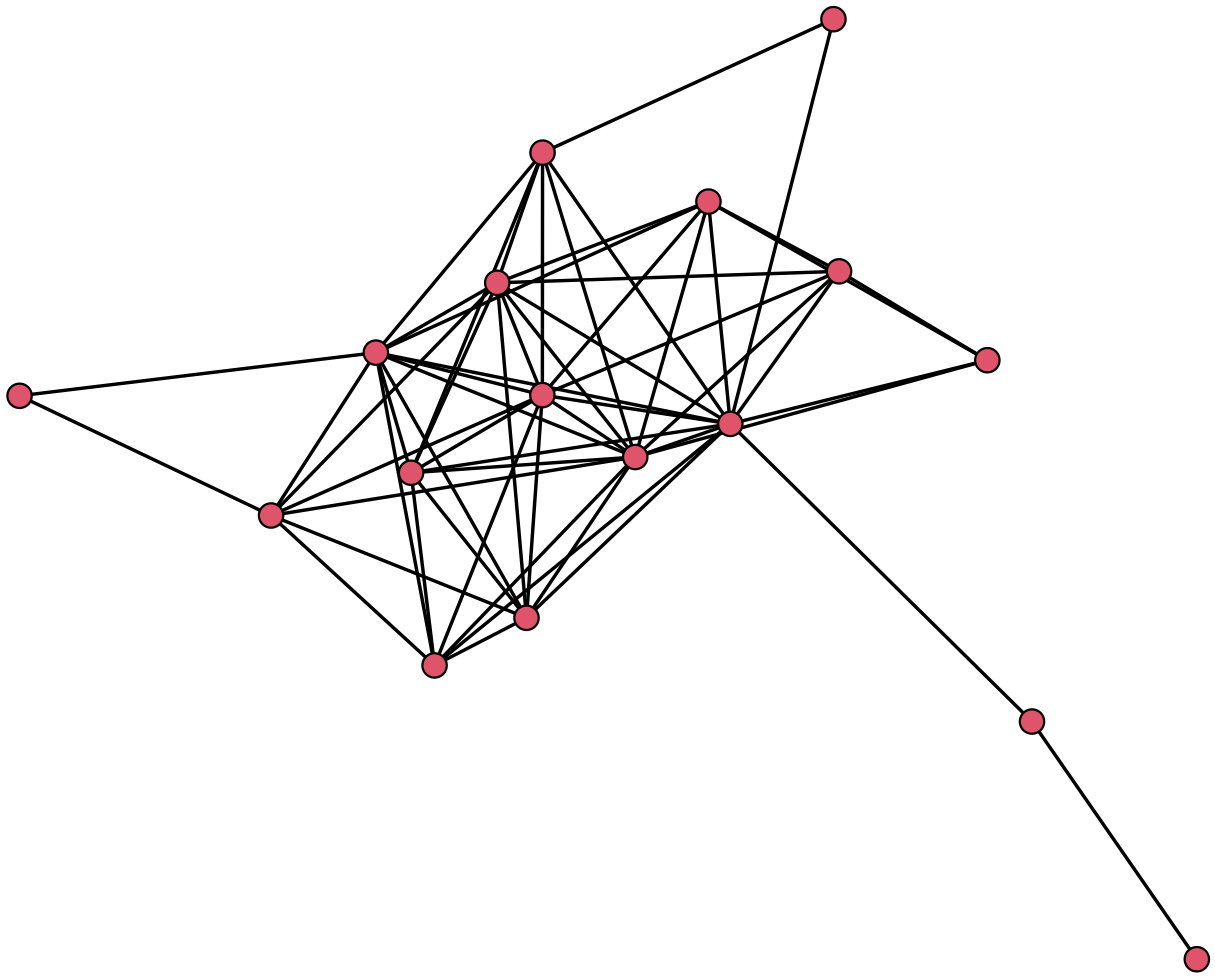
```
plot(m_igraph, layout = layout_in_circle(m_igraph))
```



Marvel network

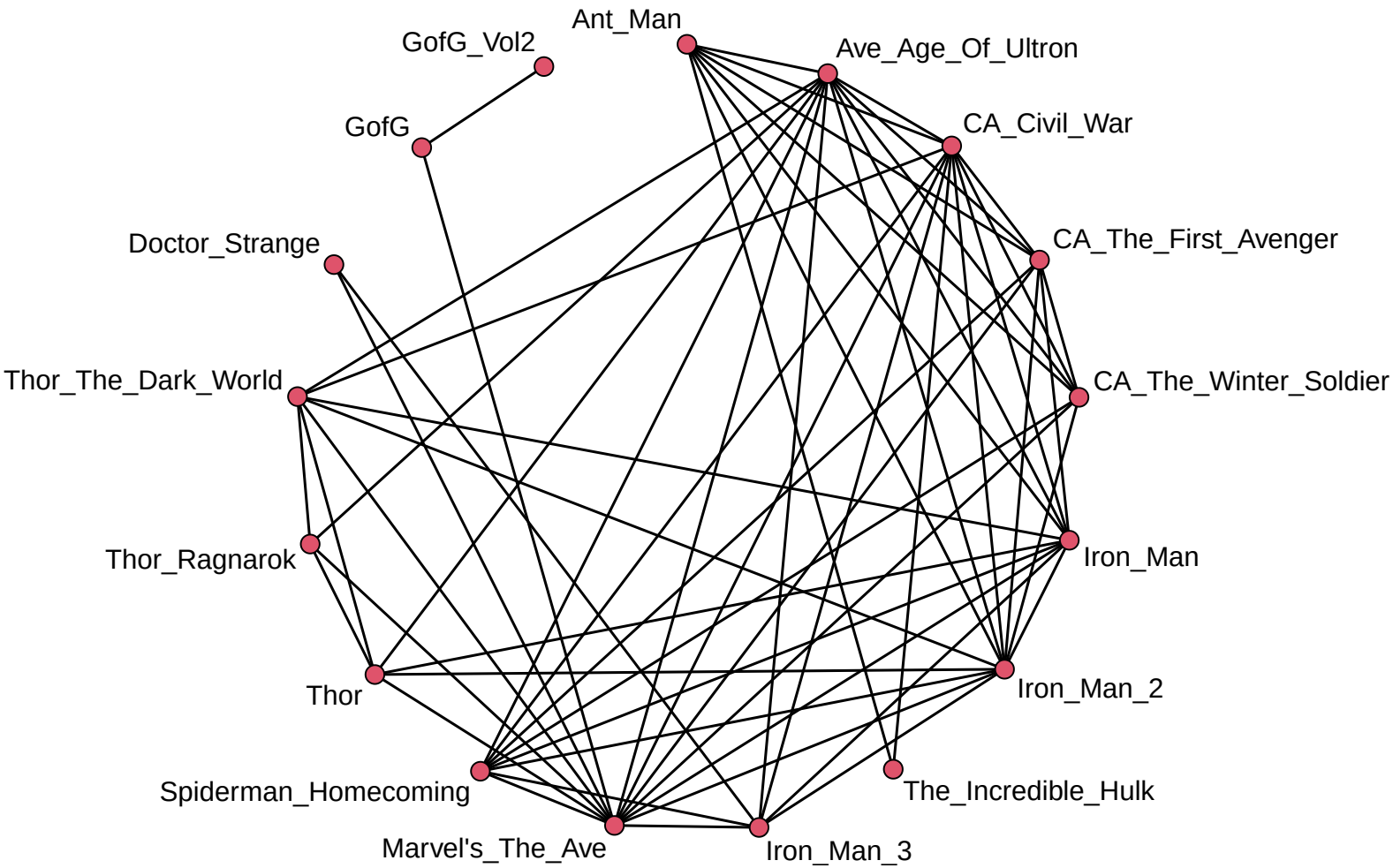
Convert to network object

```
m_net <- intergraph::asNetwork(m_igraph)
plot(m_net)
```



Marvel network

```
plot(m_net, mode="circle", displaylabels=T, label.pos=6)
```



Marvel network

Lets add some attributes to vertices

```
movies <- read.csv("./data/marvel_movies.csv", stringsAsFactors = F)
head(movies)
```

```
##              Movie.title Year        In.theatres Budget US_gross Foreign_gross
## 1               Ant_Man 2015 Jul 17, 2015  Wide    130      180           339
## 2     Ave_Age_Of_Ultron 2015  May 1, 2015  Wide    250      459           946
## 3          CA_Civil_War 2016  May 6, 2016  Wide    250      408           745
## 4  CA_The_First_Avenger 2011 Jul 22, 2011  Wide    140      177           194
## 5 CA_The_Winter_Soldier 2014  Apr 4, 2014  Wide    170      260           454
## 6              Iron_Man 2008 Apr 14, 2008  Wide    140      318           267
##   World_gross Metascore
## 1         519        64
## 2        1405        66
## 3        1153        75
## 4         371        66
## 5         714        70
## 6         585        79
```

Marvel network

If both variables are in the same order, than you can just simple add the variable to the network object

```
m_net %v% "vertex.names"
```

```
##  [1] "Ant_Man"                "Ave_Age_Of_Ultron"      "CA_Civil_War"
##  [4] "CA_The_First_Avenger"   "CA_The_Winter_Soldier"  "Iron_Man"
##  [7] "Iron_Man_2"             "The_Incredible_Hulk"    "Iron_Man_3"
## [10] "Marvel's_The_Ave"       "Spiderman_Homecoming"   "Thor"
## [13] "Thor_Ragnarok"          "Thor_The_Dark_World"    "Doctor_Strange"
## [16] "GofG"                   "GofG_Vol2"
```

```
identical(movies$Movie.title, m_net %v% "vertex.names")
```

```
## [1] TRUE
```

Marvel network

Lets add to both ***network*** and ***igraph*** objects the variable BoxOffice

```
set.vertex.attribute(m_net, attrname = "BoxOffice", movies$World_gross)
get.vertex.attribute(m_net, "BoxOffice")
```

```
##  [1]  519 1405 1153  371  714  585  624  263 1215 1519  880  449  650  645  678
## [16]  773  864
```

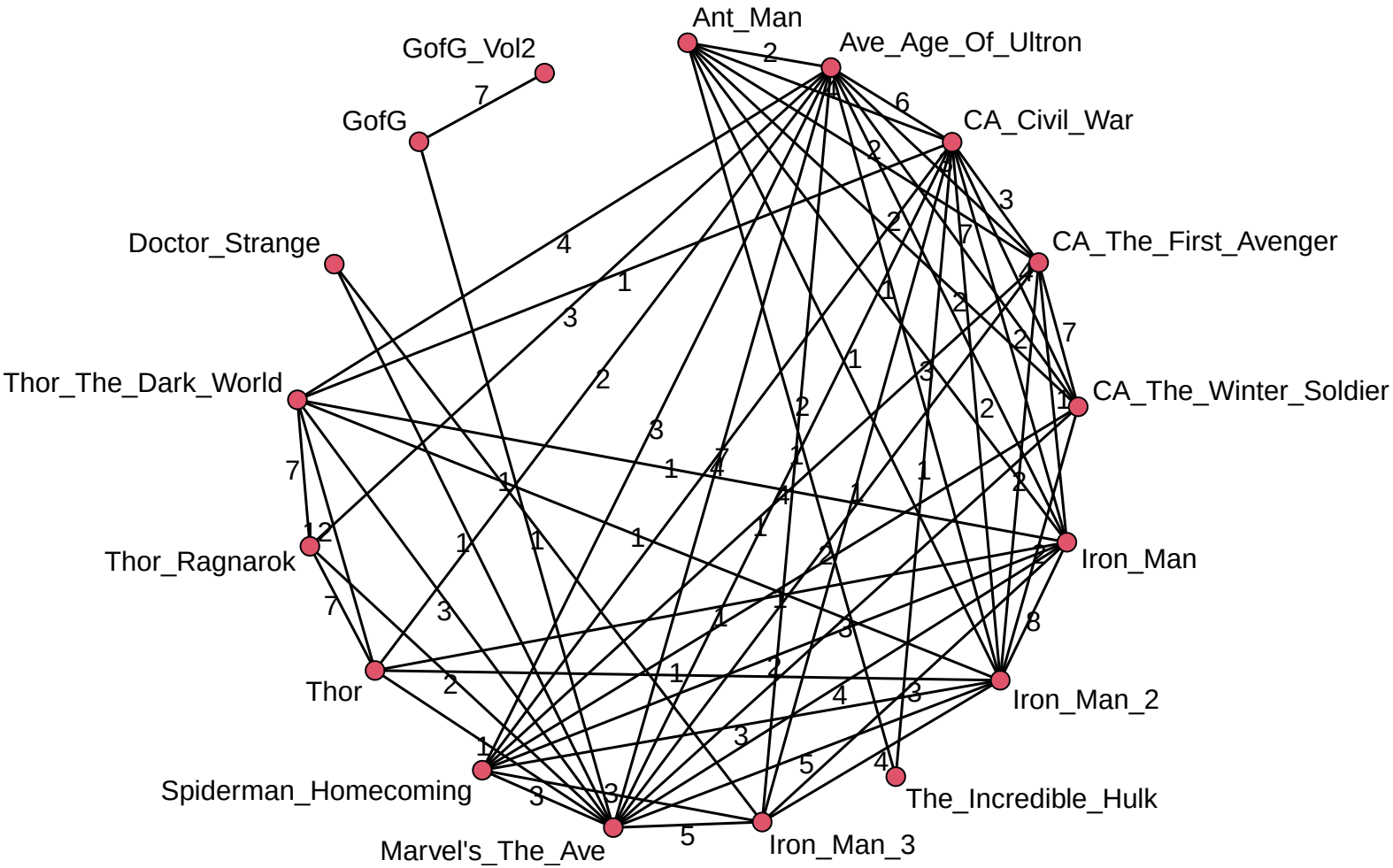
```
m_igraph <- set_vertex_attr(m_igraph, name = "BoxOffice", value = movies$World_gross
V(m_igraph)$BoxOffice
```

```
##  [1]  519 1405 1153  371  714  585  624  263 1215 1519  880  449  650  645  678
## [16]  773  864
```

Marvel network

Plot the network with the common characters as edge label

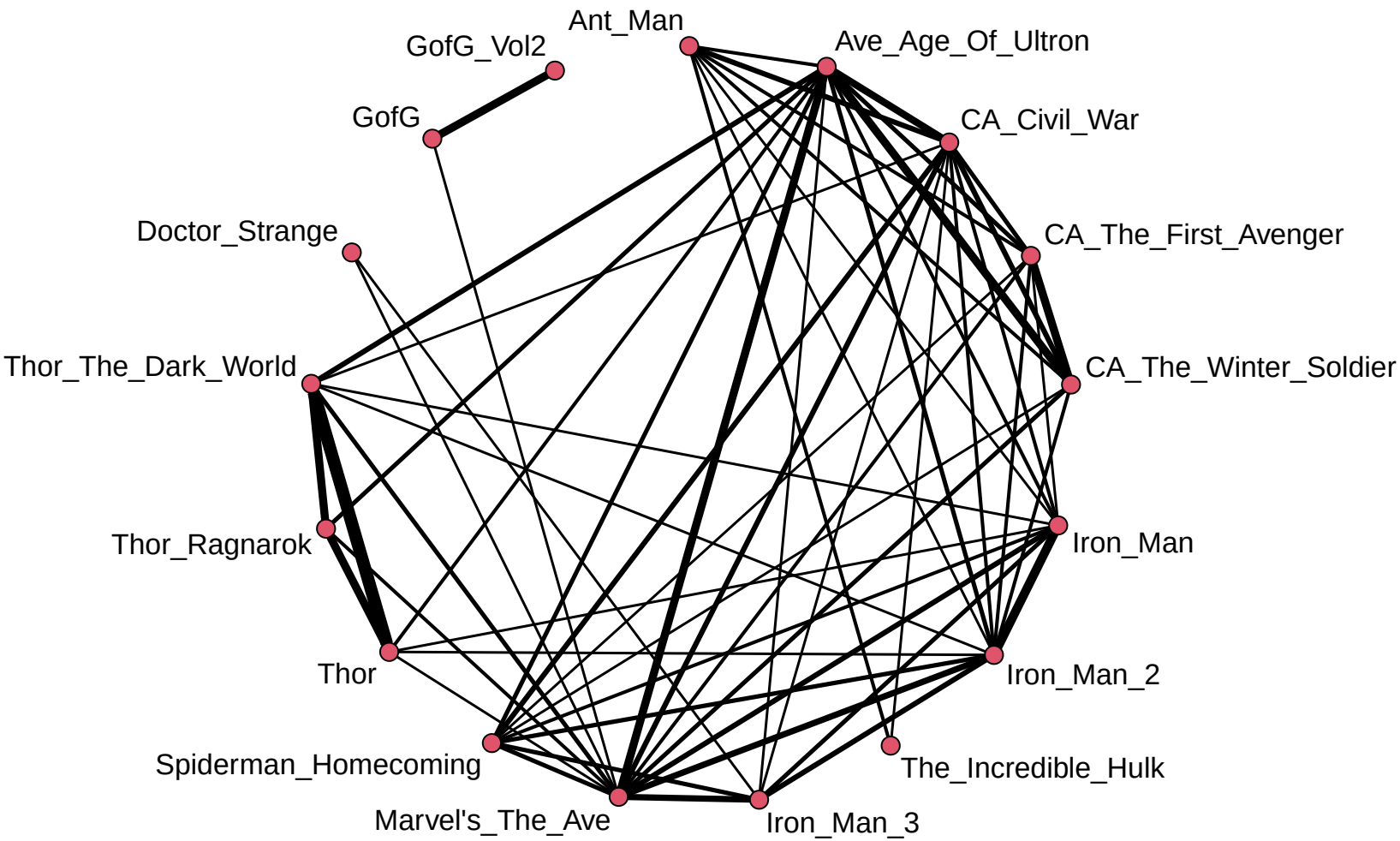
```
plot(m_net, mode="circle", displaylabels=T,
     label.pos=6, edge.label="characters")
```



Marvel network

Set the size of the edge as a

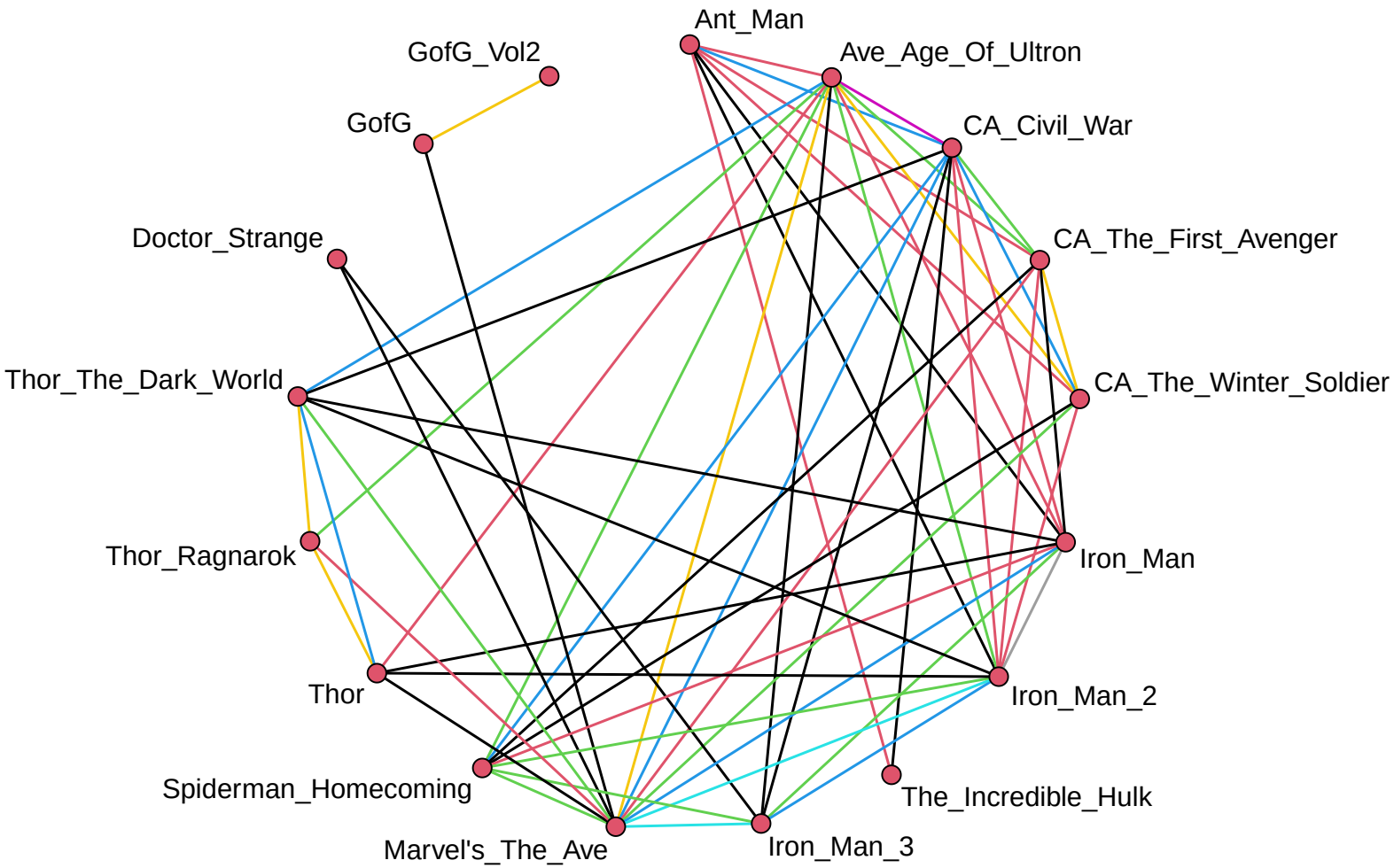
```
plot(m_net, mode="circle", displaylabels=T,
     label.pos=6, edge.lwd="characters",  label.cex=1.1)
```



Marvel network

Number of characters as an edge color

```
plot(m_net, mode="circle", displaylabels=T,
     label.pos=6, edge.col="characters")
```



Marvel network

- We want to assign BoxOffice variable to the size of the vertex.
- The conventional method will not work, need to transform the variable BoxOffice

```
plot(m_net, mode="circle", displaylabels=T,
     label.pos=6, edge.lwd="characters",
     vertex.cex = m_net %v% "BoxOffice")
```



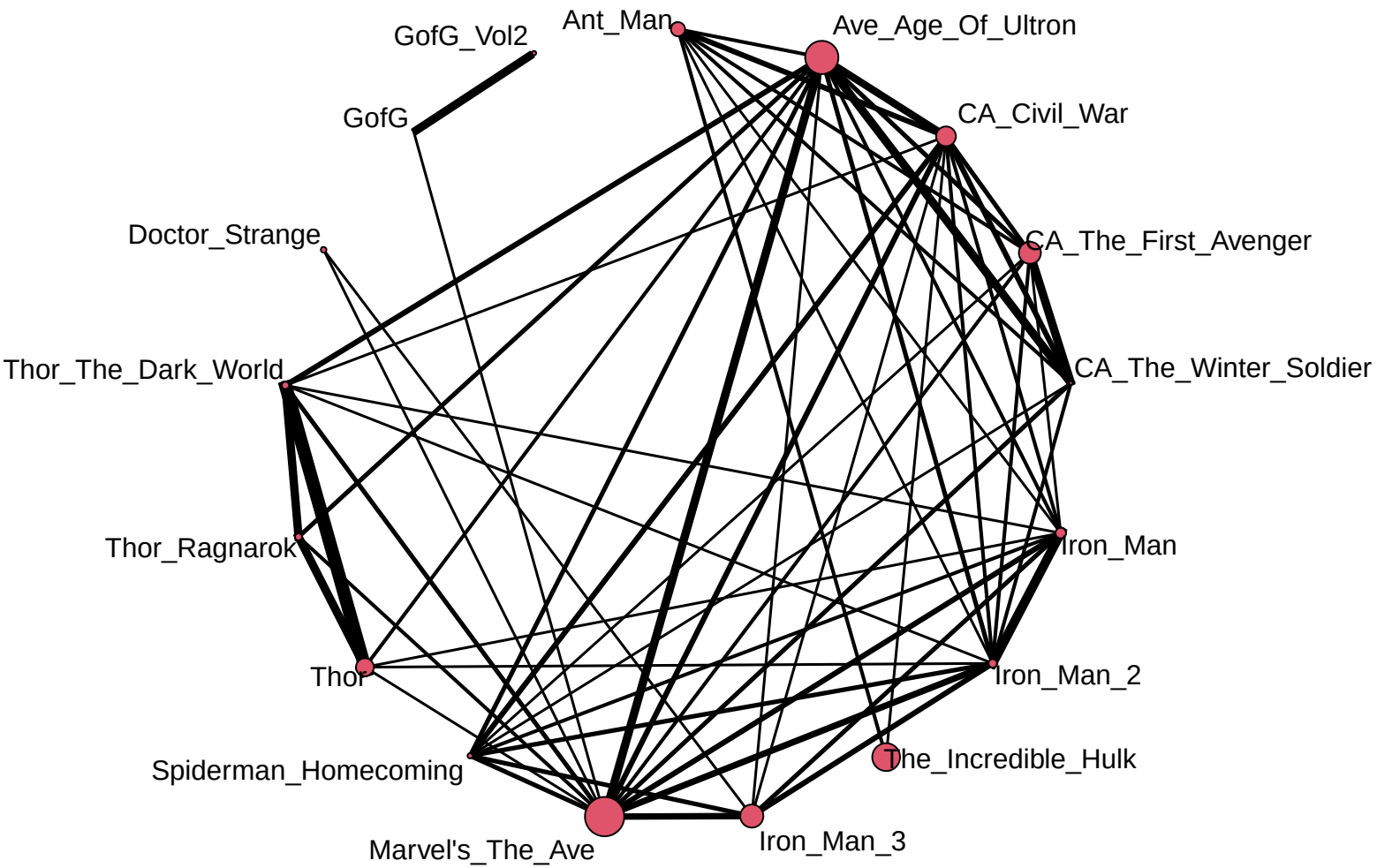
Marvel network

Instead of the variable BoxOffice, use its z-score

```
x <- m_net %v% "BoxOffice"
z <- (x-mean(x))/sd(x)
```

Marvel network

```
plot(m_net, mode="circle", displaylabels=T,
     label.pos=6, edge.lwd="characters",
     vertex.cex = z)
```



Marvel network

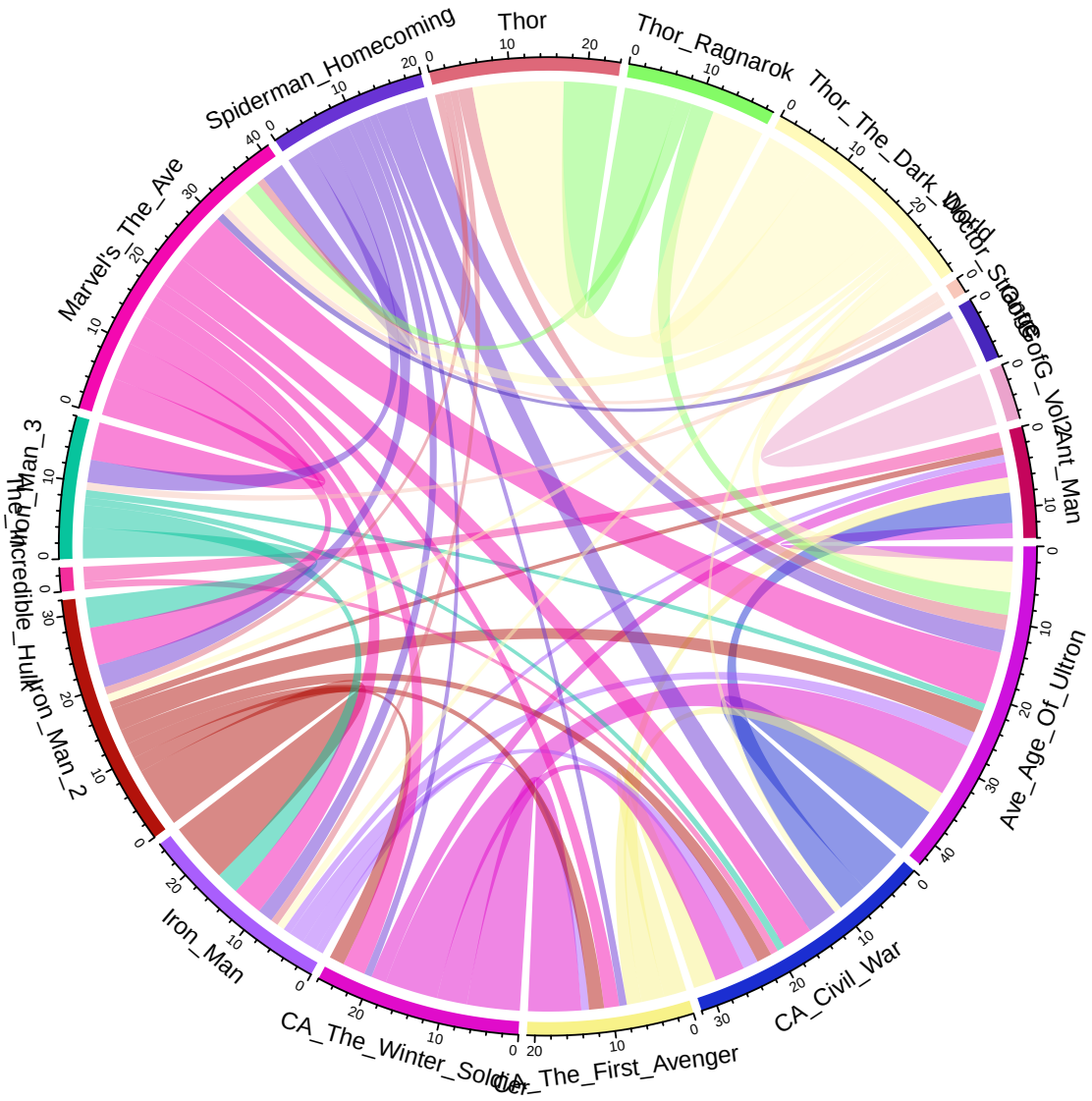
```
as_m <- as.matrix(m_net, matrix.type = "adjacency", attrname = "characters")
dim(as_m)
```

```
## [1] 17 17
```

```
as_m[upper.tri(as_m)] <- 0
```

Marvel network

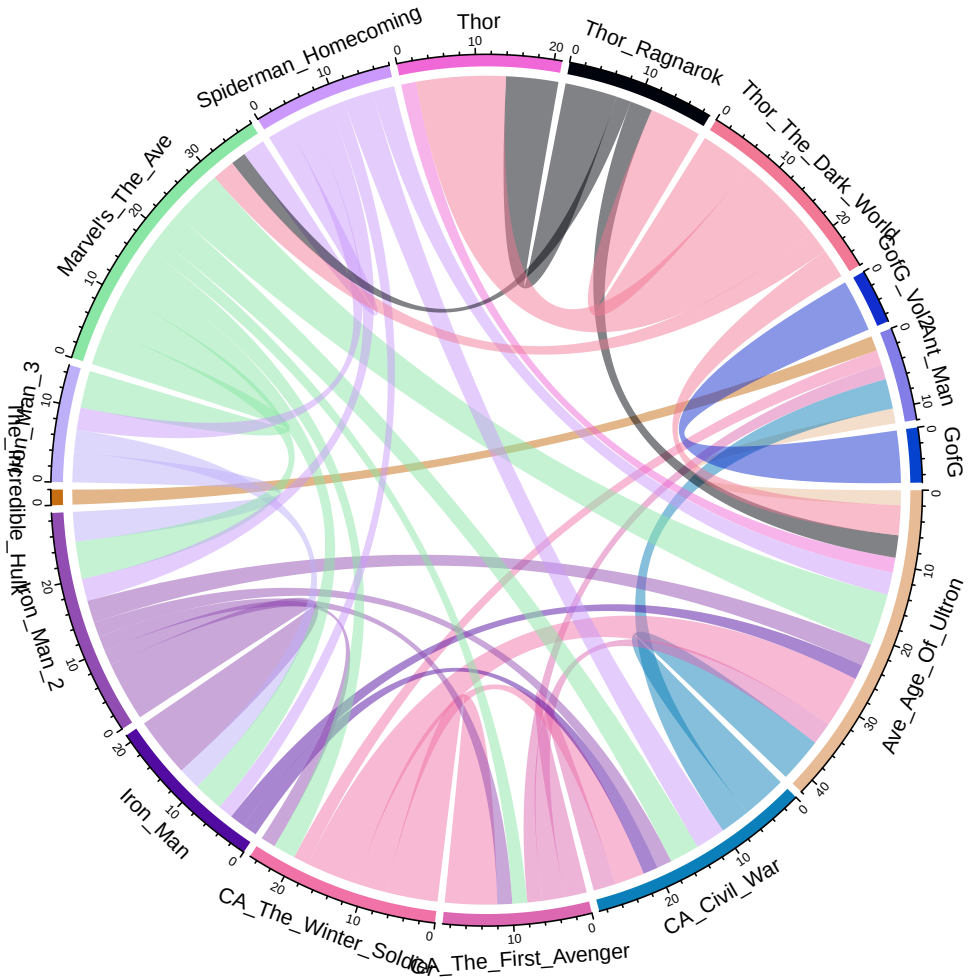
```
chordDiagram(as_m)
```



Marvel network

Lets get rid of the connections, edges, where the edge value = 1

```
as_m[as_m==1] <- 0
chordDiagram(as_m)
```



Marvel network

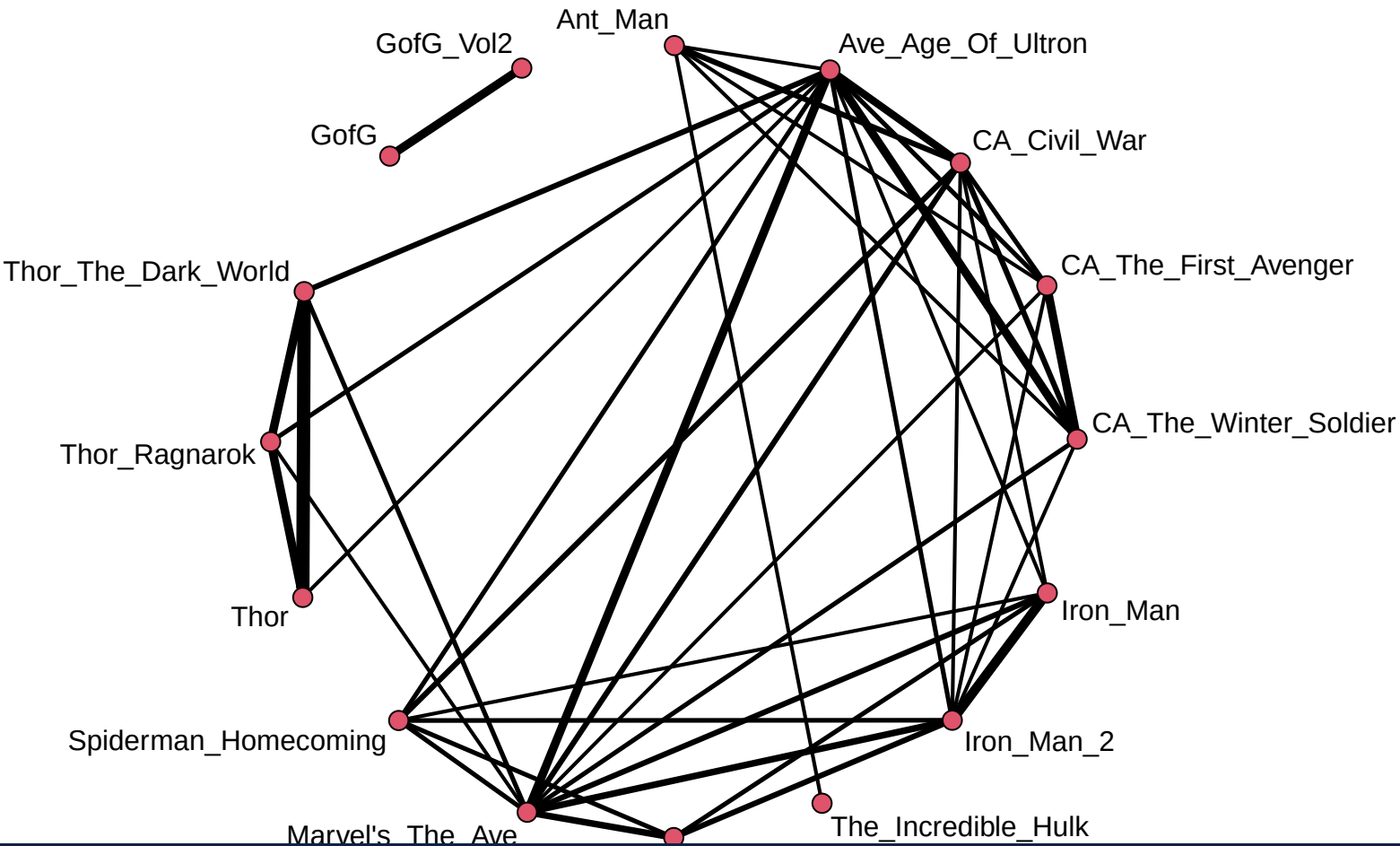
Lets induce the graph

- Keep only edges where the number of characters $>= 2$
- Keep only vertices where box office is more than 600 mln

Marvel network

```
m_net1 <- get.inducedSubgraph(m_net,
          eid=which(m_net %e% "characters" >=2))

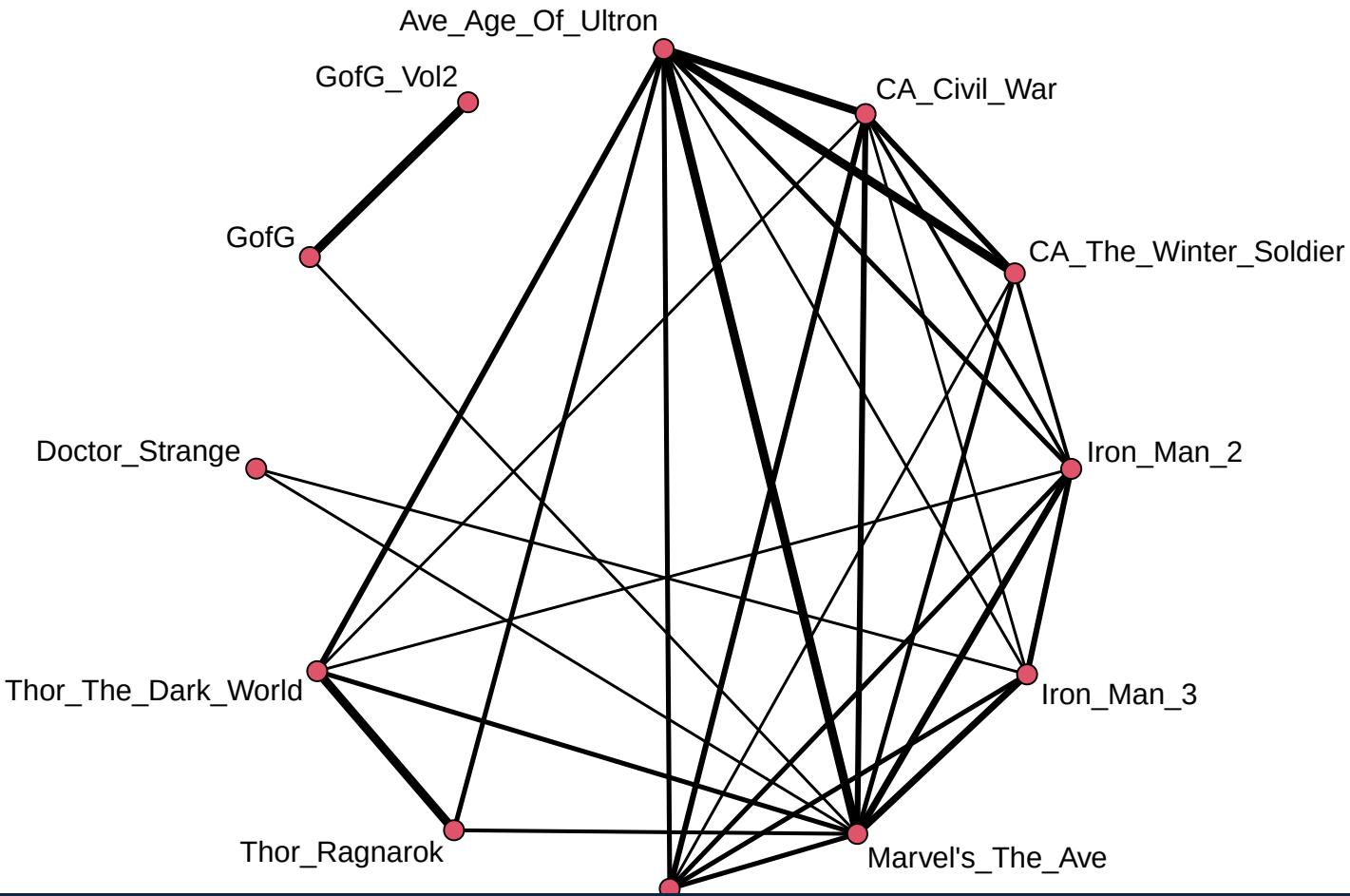
plot(m_net1, mode="circle", displaylabels=T,
     label.pos=6, edge.lwd="characters")
```



Marvel network

```
m_net2 <- get.inducedSubgraph(m_net,
          v=which(m_net %v% "BoxOffice" >=600))

plot(m_net2, mode="circle", displaylabels=T,
     label.pos=6, edge.lwd="characters")
```



Marvel network

Now we will look at the case where

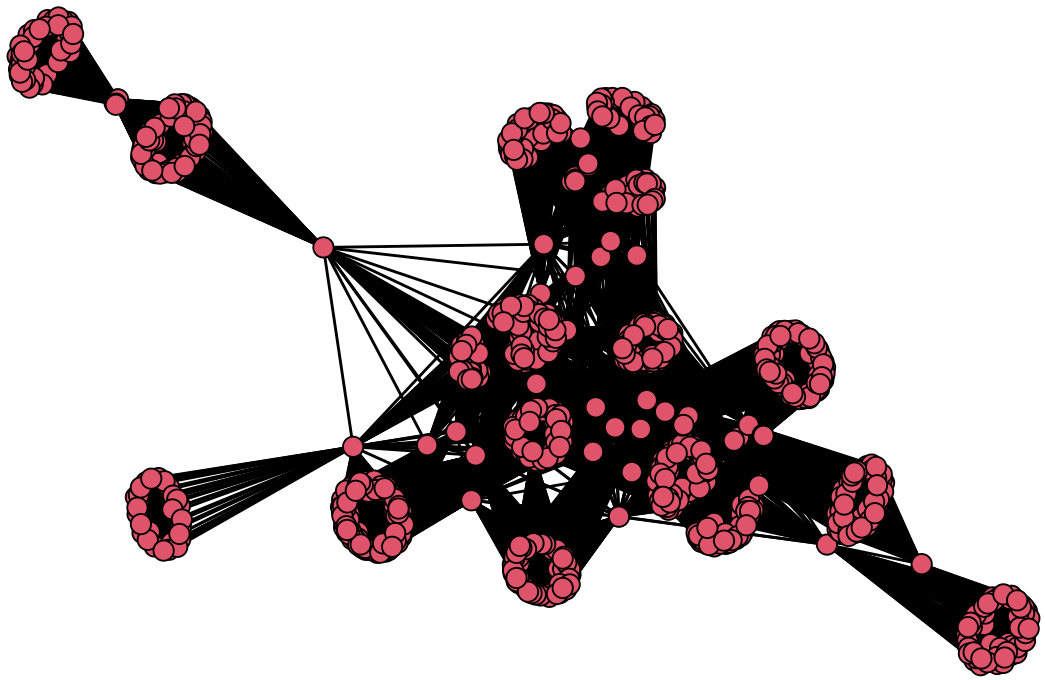
- each vertex is a character from the movie
- each edge shows if the two vertexes were in the same movie
- edge attribute weight shows in how many movies do they appear together

```
load("./data/marvel_roles.rda")
list.edge.attributes(marvel_roles)
```

```
## [1] "na"     "weight"
```

Marvel network

```
set.seed(1)
plot(marvel_roles, main="Marvel characters network")
```



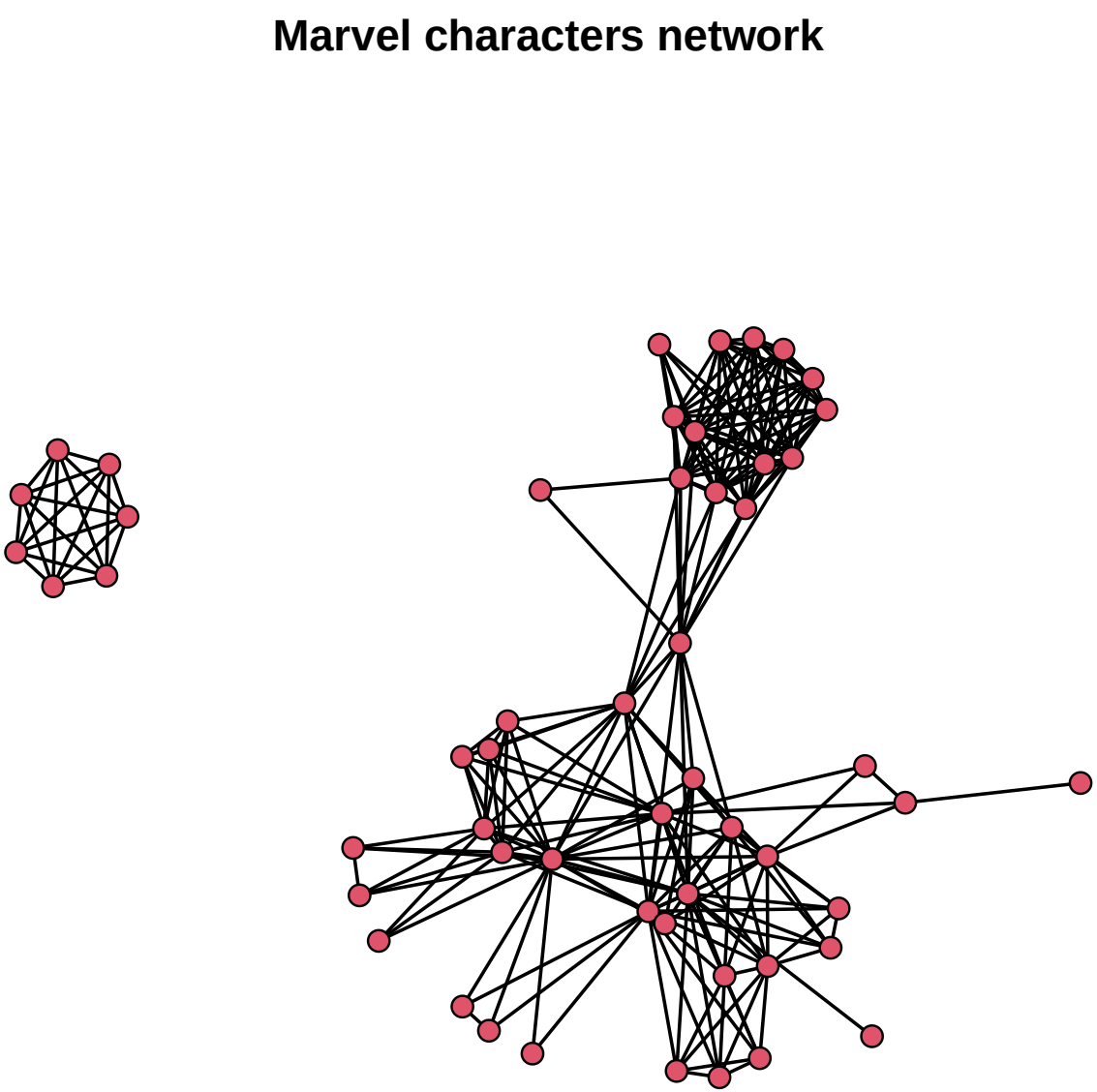
Marvel network

Lets do subsetting and keep only those edges where weight >1, so two characters appeared together in more than 1 movie

```
marvel_roles1 <- get.inducedSubgraph(marvel_roles,
                                     eid=which(marvel_roles %e% "weight">1))
```

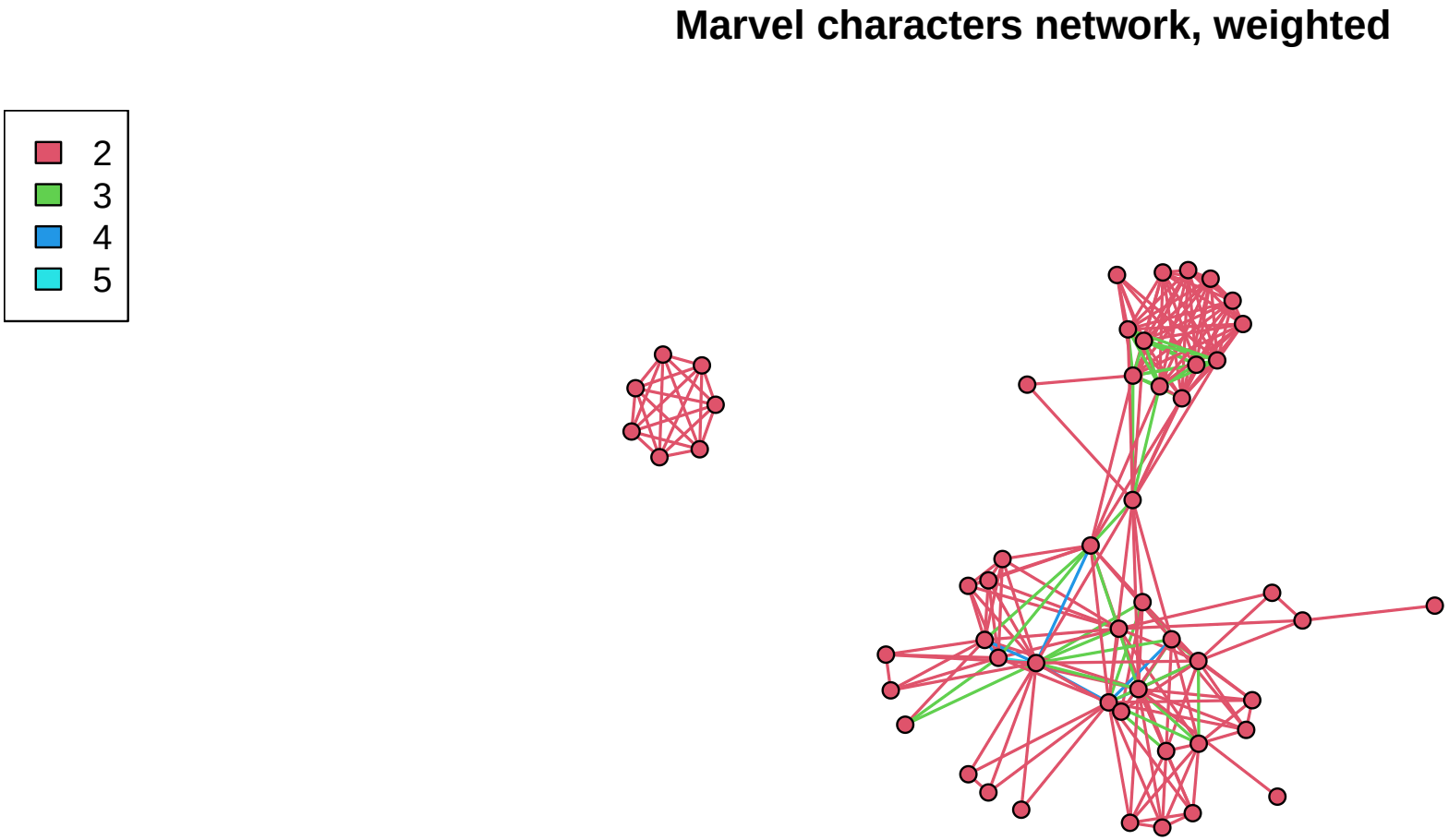
Marvel network

```
set.seed(1)
plot(marvel_roles1, main="Marvel characters network")
```



Marvel network

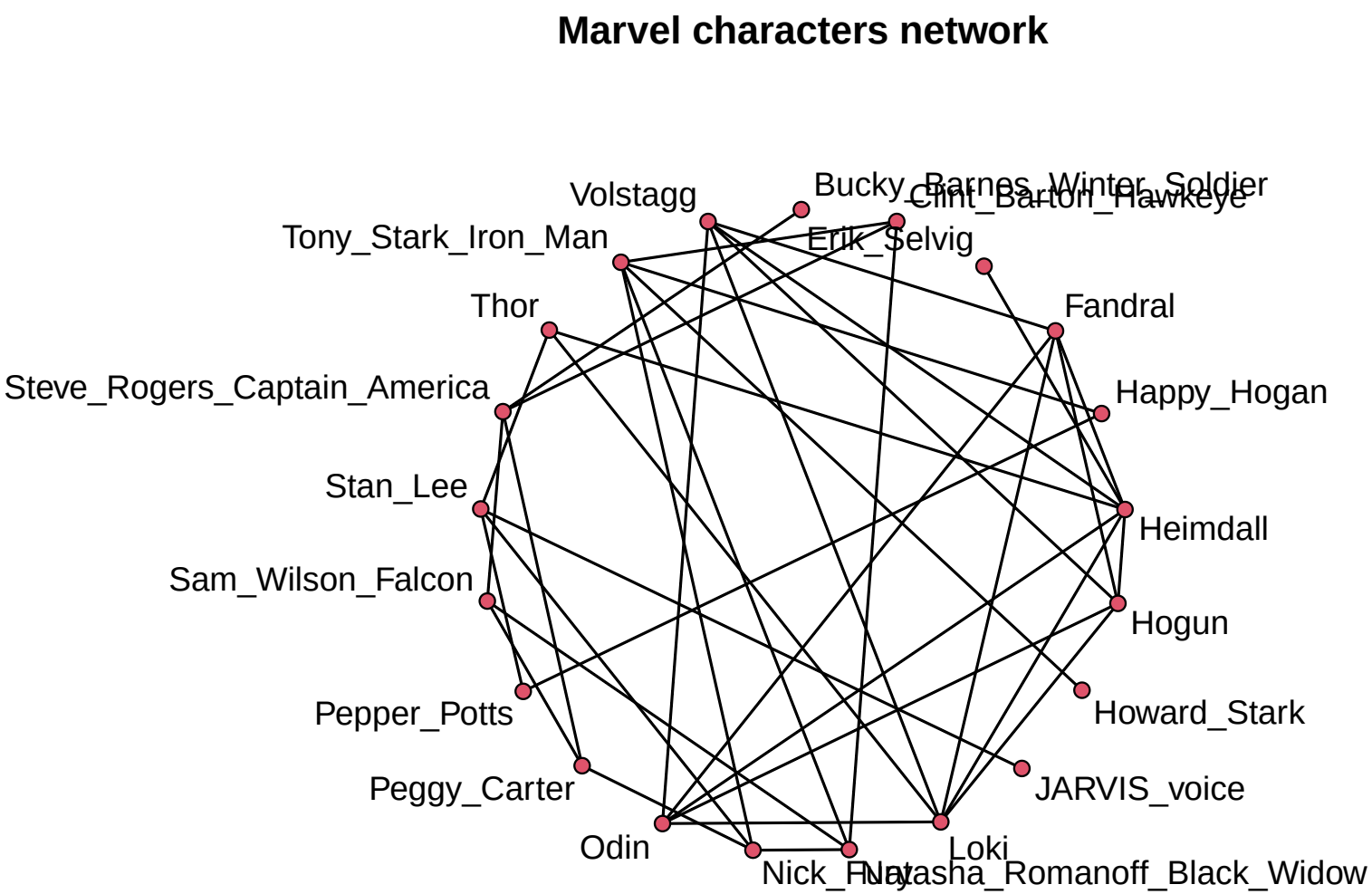
```
set.seed(1)
plot(marvel_roles1, main="Marvel characters network, weighted",
     edge.col = "weight")
legend("topleft",fill = unique(marvel_roles1 %e% "weight"),
     legend=unique(marvel_roles1 %e% "weight"))
```



Marvel network

```
set.seed(1)
marvel_roles2 <- get.inducedSubgraph(marvel_roles, eid=which(marvel_roles %e% "weight"==3))

plot(marvel_roles2, main="Marvel characters network",
     displaylabels=T, mode="circle")
```



NBA passing network

NBA passing network

- First we can get the roster
- Get it from roster.rda

```
load("./data/roster.rda")
head(roster)
```

```
##   PERSON_ID DISPLAY_LAST_COMMA_FIRST    TEAM_ID     TEAM_CITY TEAM_NAME
## 1    203518            Abrines, Alex 1610612760 Oklahoma City   Thunder
## 2    203112              Acy, Quincy 1610612751      Brooklyn      Nets
## 3    203500            Adams, Steven 1610612760 Oklahoma City   Thunder
## 4   1628389             Adebayo, Bam 1610612748         Miami      Heat
## 5    201167           Afflalo, Arron 1610612753       Orlando     Magic
## 6    201582           Ajinca, Alexis 1610612740   New Orleans  Pelicans
##   TEAM_ABBREVIATION
## 1               OKC
## 2               BKN
## 3               OKC
## 4               MIA
## 5               ORL
## 6               NOP
```

NBA: passing network

Load the roster for Golden State Warriors

```
load("./data/gsw.rda")
```

NBA: passing network

Make a vector with playerids

```
x <- gsw$PLAYER_ID
x[1:10]
```

```
##  [1] "1627775" "1628395" "1627745" "1626188" "1626172" "203110"  "202691"
##  [8] "201939"  "201580"  "201156"
```

A loop to get the passing network, using function getPassing

```
passing <- c()
for (i in x){
  try({
    p<-getPassing(2018, i, "Regular")
  passing<-rbind(passing,p)})
}
save(passing, file = "passing.rda")
```

NBA: passing network

```
load("./data/passing.rda")
head(passing)
```

```
##   PLAYER_ID   PLAYER_NAME            TEAM_NAME    TEAM_ID TEAM_ABBREVIATION
## 1   1627775 Patrick McCaw Golden State Warriors 1610612744                GSW
## 2   1627775 Patrick McCaw Golden State Warriors 1610612744                GSW
## 3   1627775 Patrick McCaw Golden State Warriors 1610612744                GSW
## 4   1627775 Patrick McCaw Golden State Warriors 1610612744                GSW
## 5   1627775 Patrick McCaw Golden State Warriors 1610612744                GSW
## 6   1627775 Patrick McCaw Golden State Warriors 1610612744                GSW
##   PASS_TYPE  G PASS_TEAMMATE_PLAYER_NAME PASS_TEAMMATE_PLAYER_ID FREQUENCY PASS
## 1      made 55                Jordan Bell                 1628395     0.089  104
## 2      made 55              Damian Jones                 1627745     0.001    1
## 3      made 55                Quinn Cook                 1626188     0.050   59
## 4      made 55              Kevon Looney                 1626172     0.061   71
## 5      made 55             Draymond Green                  203110     0.099  116
## 6      made 55              Klay Thompson                  202691     0.094  110
##   AST FGM FGA FG_PCT FG2M FG2A FG2_PCT FG3M FG3A FG3_PCT
## 1   3   4   8  0.500    4    7   0.571    0    1   0.000
## 2   0   0   0  0.000    0    0   0.000    0    0   0.000
## 3   3   6  12  0.500    5    9   0.556    1    3   0.333
## 4   2   4  11  0.364    4   10   0.400    0    1   0.000
## 5   6   6  19  0.316    3   10   0.300    3    9   0.333
## 6  14  18  37  0.486    8   16   0.500   10   21   0.476
```

NBA: passing network

- Look at the variable PASS_TYPE
- It has two values made and received
- We will need a dataframe with made values only
- And with the players that were on the roster

```
passing <- passing[passing$PASS_TYPE=="made",]
passing <- passing[passing$PASS_TEAMMATE_PLAYER_ID %in% x,]
```

NBA: passing network

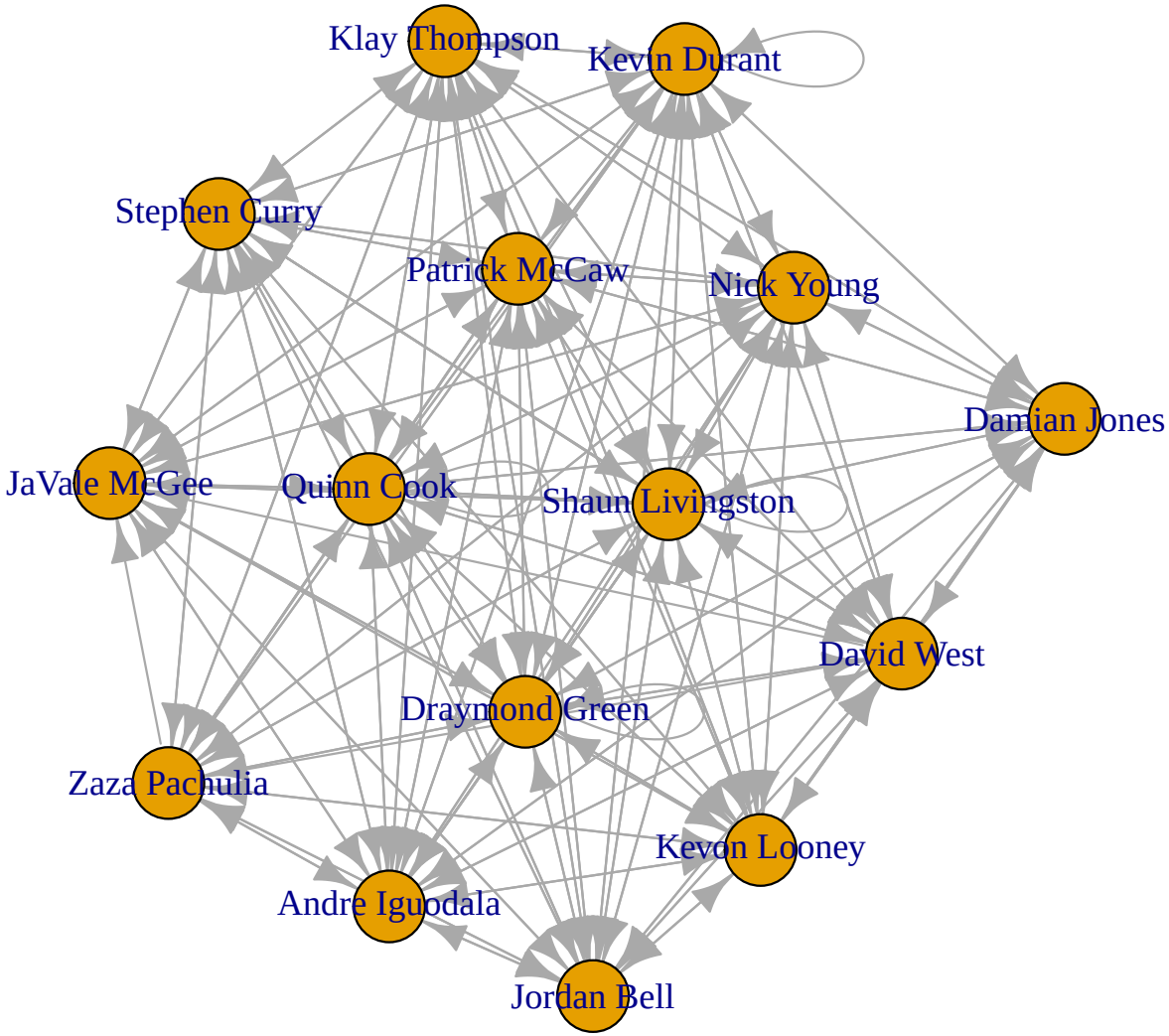
Create an igraph object from edgelist

```
i_pass <- graph_from_edgelist(
  as.matrix(passing[,c("PLAYER_NAME",
                       "PASS_TEAMMATE_PLAYER_NAME")]), directed = T)
```

NBA: passing network

Loops

```
plot(i_pass)
```



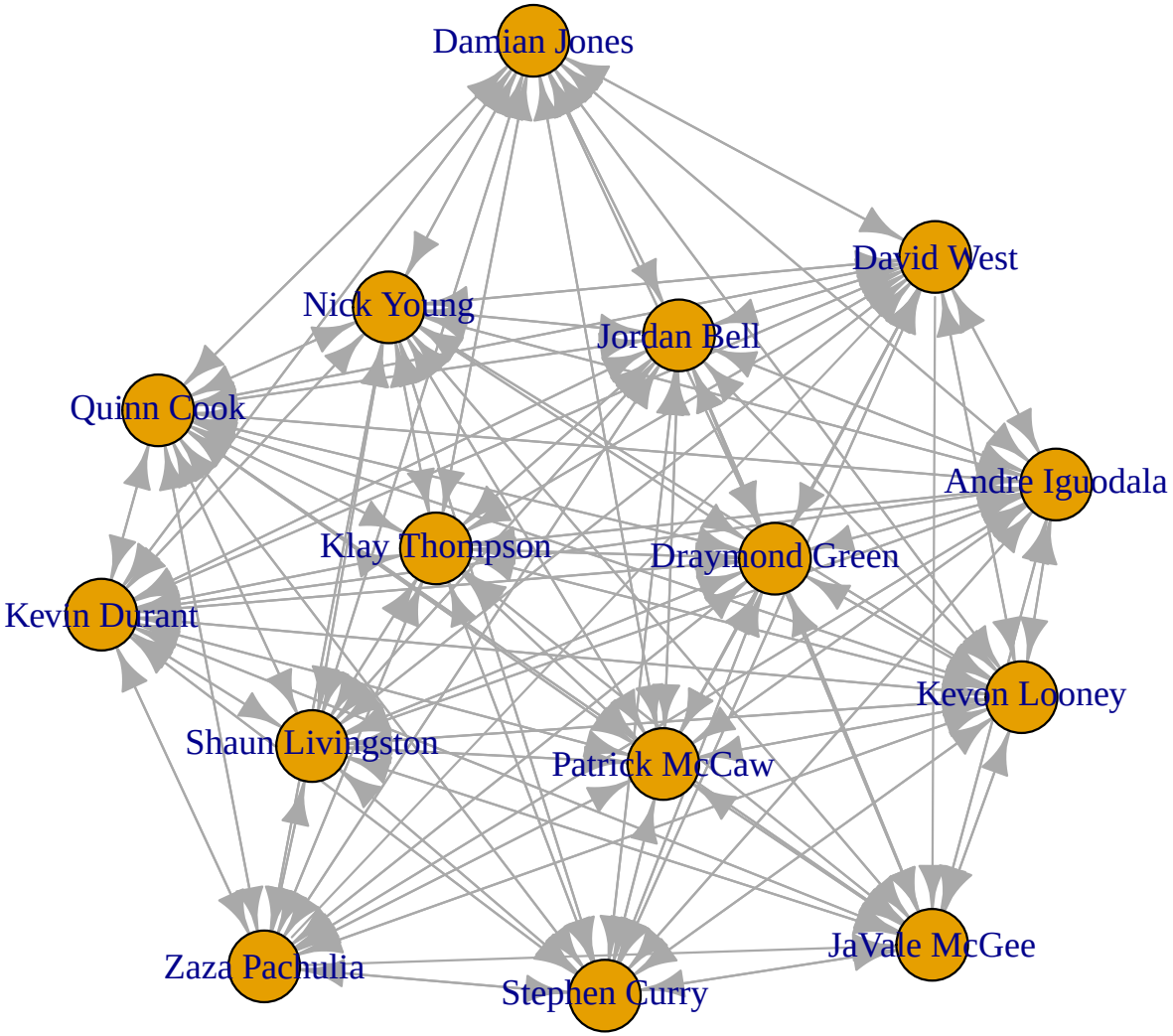
Nba passing

Remove loops and plot

```
passing <- passing[passing$PLAYER_NAME !=passing$PASS_TEAMMATE_PLAYER_NAME,]
```

NBA: passing network

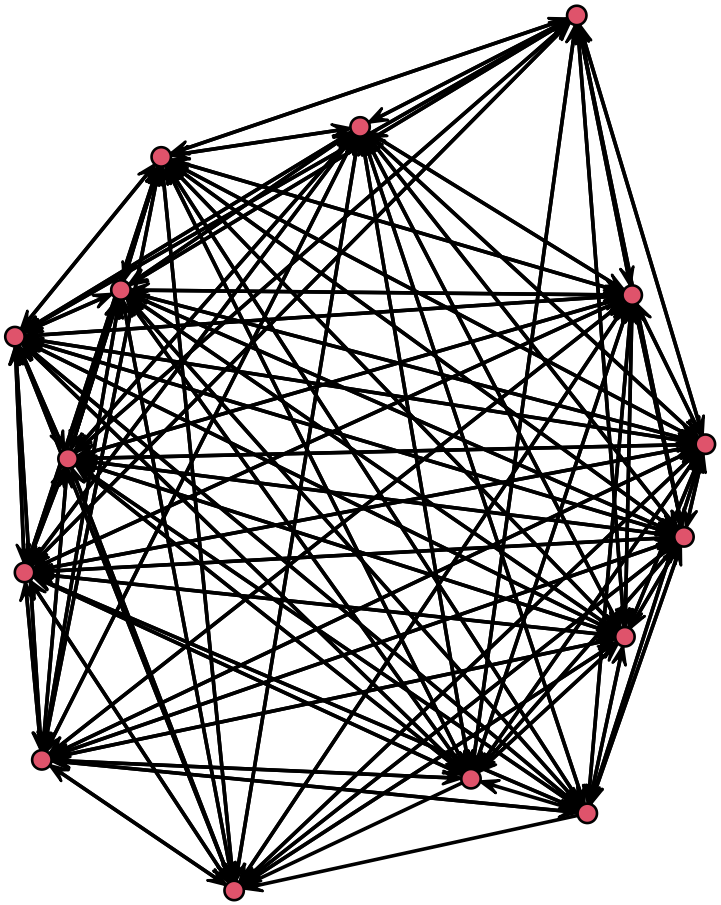
```
i_pass <- graph_from_edgelist(
  as.matrix(passing[,c("PLAYER_NAME",
                       "PASS_TEAMMATE_PLAYER_NAME")]), directed = T)
plot(i_pass)
```



NBA: passing network

Transform to network object

```
n_pass <- asNetwork(i_pass)
plot(n_pass)
```



NBA: passing network

Lets add some attributes

start with vertex attributes
currently we have vertex name only

```
p <- n_pass %v% "vertex.names"
p
```

```
##  [1] "Patrick McCaw"    "Jordan Bell"      "Damian Jones"     "Quinn Cook"
##  [5] "Kevon Looney"     "Draymond Green"   "Klay Thompson"    "Stephen Curry"
##  [9] "JaVale McGee"     "Nick Young"       "Kevin Durant"     "Andre Iguodala"
## [13] "Shaun Livingston" "Zaza Pachulia"    "David West"
```

NBA: passing network

- We want to get the player position as a network attribute
- First need to be sure that they have the same order

```
gsw <- gsw[order(match(gsw$PLAYER,p)),]
gsw$PLAYER
```

```
##  [1] "Patrick McCaw"    "Jordan Bell"      "Damian Jones"     "Quinn Cook"
##  [5] "Kevon Looney"     "Draymond Green"   "Klay Thompson"    "Stephen Curry"
##  [9] "JaVale McGee"     "Nick Young"       "Kevin Durant"     "Andre Iguodala"
## [13] "Shaun Livingston" "Zaza Pachulia"    "David West"
```

NBA: passing network

Set vertex attribute

```
network::set.vertex.attribute(n_pass, "position", gsw$POSITION)
n_pass %v% "position"
```

```
##  [1] "G"   "C-F" "C"   "G"   "F"   "F"   "G"   "G"   "C"   "G-F" "F"   "F-G"
## [13] "G"   "C"   "F"
```

NBA: passing network

Get the number of games played (variable G)

```
(ngames <- unique(passing[,c("PLAYER_NAME", "G")]))
```

```
##          PLAYER_NAME  G
## 1      Patrick McCaw 55
## 31      JaVale McGee 64
## 57       Jordan Bell 55
## 87        David West 72
## 116       Quinn Cook 33
## 148     Kevon Looney 66
## 179       Nick Young 79
## 209   Andre Iguodala 63
## 239    Klay Thompson 72
## 271     Damian Jones 13
## 295   Draymond Green 69
## 328    Zaza Pachulia 67
## 355    Stephen Curry 50
## 384 Shaun Livingston 70
## 417     Kevin Durant 67
```

NBA: passing network

Set the right order

```
(ngames <- ngames[order(match(ngames$PLAYER_NAME,p)),])
```

```
##          PLAYER_NAME  G
## 1      Patrick McCaw 55
## 57       Jordan Bell 55
## 271     Damian Jones 13
## 116       Quinn Cook 33
## 148     Kevon Looney 66
## 295   Draymond Green 69
## 239    Klay Thompson 72
## 355    Stephen Curry 50
## 31      JaVale McGee 64
## 179       Nick Young 79
## 417     Kevin Durant 67
## 209   Andre Iguodala 63
## 384 Shaun Livingston 70
## 328    Zaza Pachulia 67
## 87        David West 72
```

NBA: passing network

Set the vertex attribute

```
network::set.vertex.attribute(n_pass, "ngames", ngames$G)
n_pass %v% "ngames"
```

```
##  [1] 55 55 13 33 66 69 72 50 64 79 67 63 70 67 72
```

NBA: passing network

- Lets look at the edge attributes
- Start with number of passes as n edge attribute

```
network::set.edge.attribute(n_pass, "passes", passing$PASS)
n_pass %e% "passes"
```

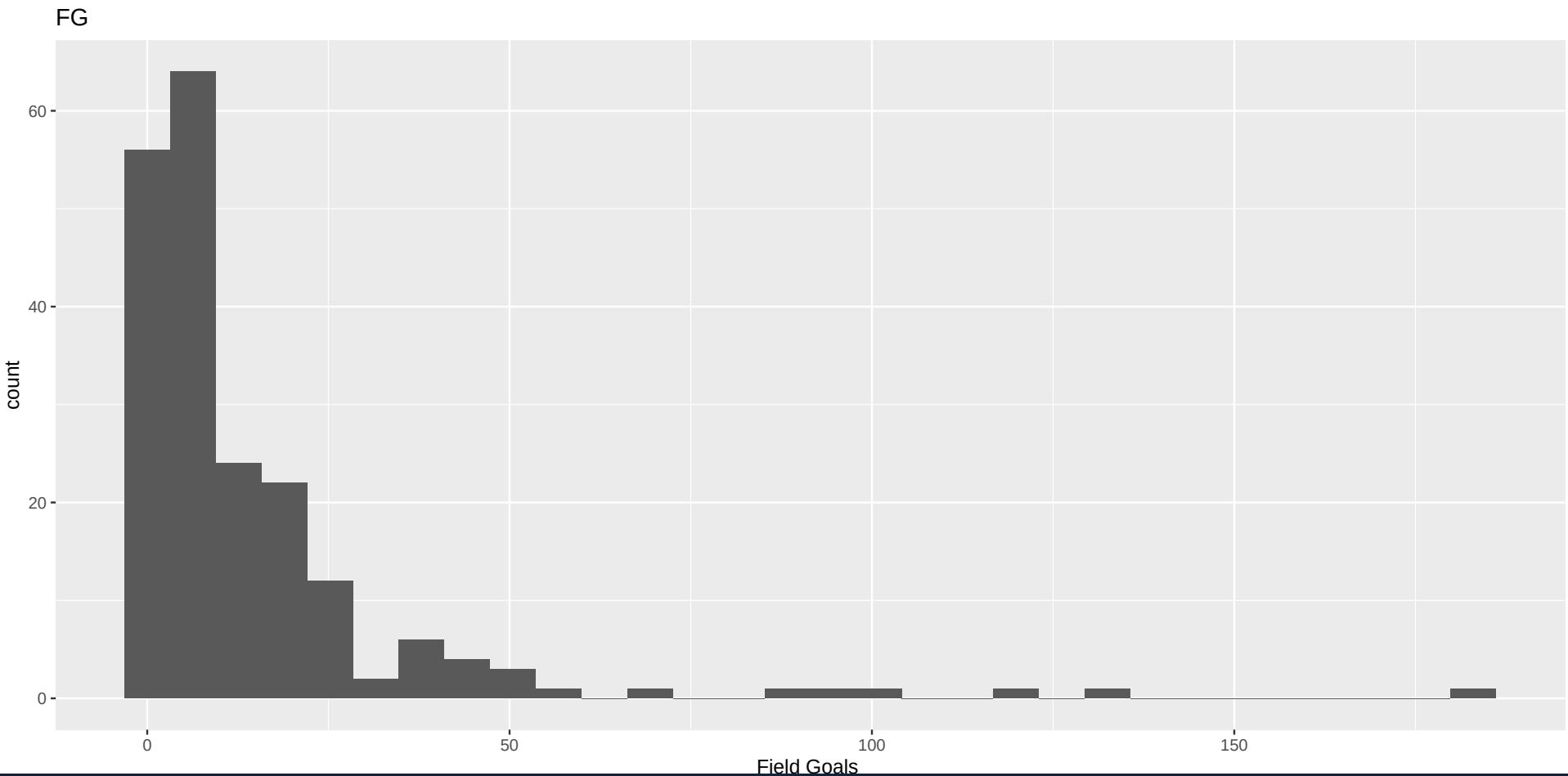
```
##   [1] 104   1  59  71 116 110  66  46 104 150  81  53  28  86  16  49  70  23
##  [19]  88  42  96  35  88  63  23 176   4 177  23 108  92 127  15  86 207  78
##  [37]  81  18  15  20  90   1  53  18 217 247  42   1  94  93 179 278 119  51
##  [55]  22 109 193  99   4  69 124 172  90  52  86  61  17  94   2 156  79  63
##  [73] 122  10  85 168 134 119  10  13  62  70   5  80  59 118  49  65  33  84
##  [91]  78  66  38  97  68  89   3 100  89 369 256 322  42 124 270 225  49 180
## [109]  70  66   6  53  61 434 150  46  46 277 149 161 149 267   3   2  29   4
## [127]   5   2   8   8   3   7   1  62 147   4 246  45 697 973  42 147 837 370
## [145] 423 147 212  23  26  91   7 261 198 375   1  47 212  60  49   1  79  91
## [163]   3 100 613 343  77  94 523 238  53 288  37  51  58   9  38  67 347 255
## [181]  68  14 124 213 205  34 281 149  92   9 179 101 498 394 583  74  79 188
## [199] 150 170  72
```

NBA: passing network

Lets look at two more variables for edge attributes
Field Goals made

```
ggplot(passing, aes(x=FGM))+geom_histogram()+
  labs(x="Field Goals ", title = "FG")
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

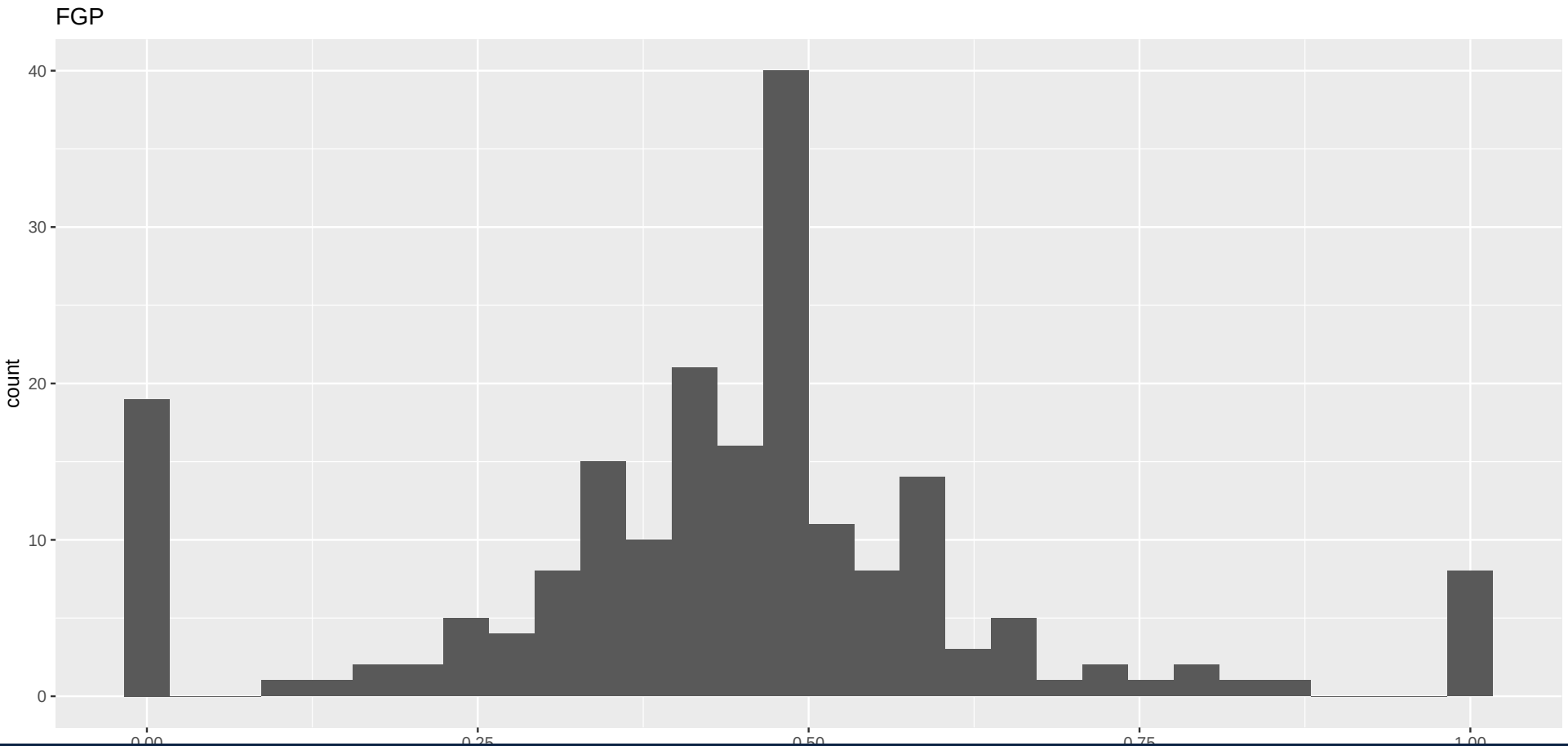


NBA: passing network

Field goal percentage

```
ggplot(passing, aes(x=FG_PCT))+geom_histogram()+
  labs(x="Field Goal Percentage", title = "FGP")
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



NBA: passing network

```
network::set.edge.attribute(n_pass, "FGP", passing$FG_PCT)
n_pass %e% "FGP"
```

```
##   [1] 0.500 0.000 0.500 0.364 0.316 0.486 0.524 0.417 0.447 0.341 0.273 0.750
##  [13] 0.500 0.522 1.000 0.222 0.429 0.875 0.250 0.533 0.478 0.500 0.316 0.600
##  [25] 0.000 0.400 0.333 0.529 0.714 0.500 0.429 0.450 0.833 0.400 0.478 0.444
##  [37] 0.455 0.364 0.500 0.250 0.333 0.000 0.500 0.200 0.478 0.500 0.267 1.000
##  [49] 0.370 0.455 0.355 0.481 0.650 0.231 0.429 0.353 0.375 0.548 0.000 0.579
##  [61] 0.333 0.569 0.467 0.600 0.571 0.357 0.500 0.500 0.000 0.393 0.500 0.188
##  [73] 0.320 0.800 0.308 0.436 0.409 0.300 1.000 0.000 0.286 0.357 0.000 0.583
##  [85] 0.500 0.111 0.467 0.300 0.500 0.321 0.429 0.286 0.167 0.429 0.467 0.357
##  [97] 0.000 0.313 0.476 0.400 0.462 0.480 0.412 0.458 0.490 0.536 0.500 0.512
## [109] 0.533 0.444 1.000 0.143 0.357 0.338 0.478 0.647 0.333 0.464 0.515 0.400
## [121] 0.615 0.567 0.000 1.000 0.500 1.000 0.000 0.000 0.000 1.000 0.000 0.000
## [133] 0.000 0.778 0.364 0.667 0.571 0.429 0.432 0.439 0.727 0.359 0.567 0.414
## [145] 0.526 0.490 0.481 0.600 0.400 0.429 0.250 0.488 0.565 0.575 0.000 0.467
## [157] 0.381 0.625 0.417 0.000 0.563 0.348 0.000 0.640 0.468 0.493 0.576 0.452
## [169] 0.450 0.359 0.500 0.500 0.500 0.600 0.417 0.000 0.429 0.700 0.377 0.461
## [181] 0.409 0.571 0.487 0.394 0.474 1.000 0.577 0.575 0.385 0.500 0.511 0.517
## [193] 0.463 0.487 0.503 0.611 0.405 0.258 0.564 0.560 0.652
```

NBA: passing network

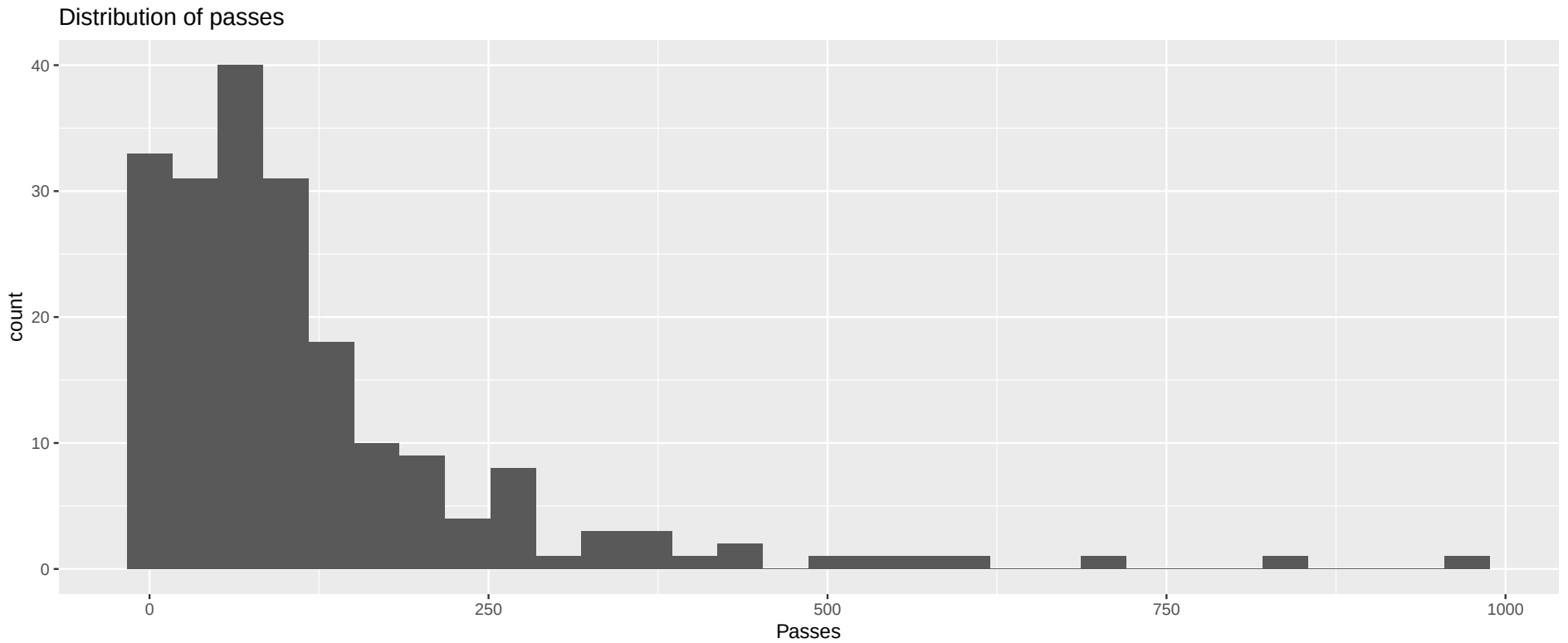
Lets look at the distribution of the passes

```
summary(n_pass %e% "passes")
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##     1.0    38.0    79.0   121.3   150.0   973.0
```

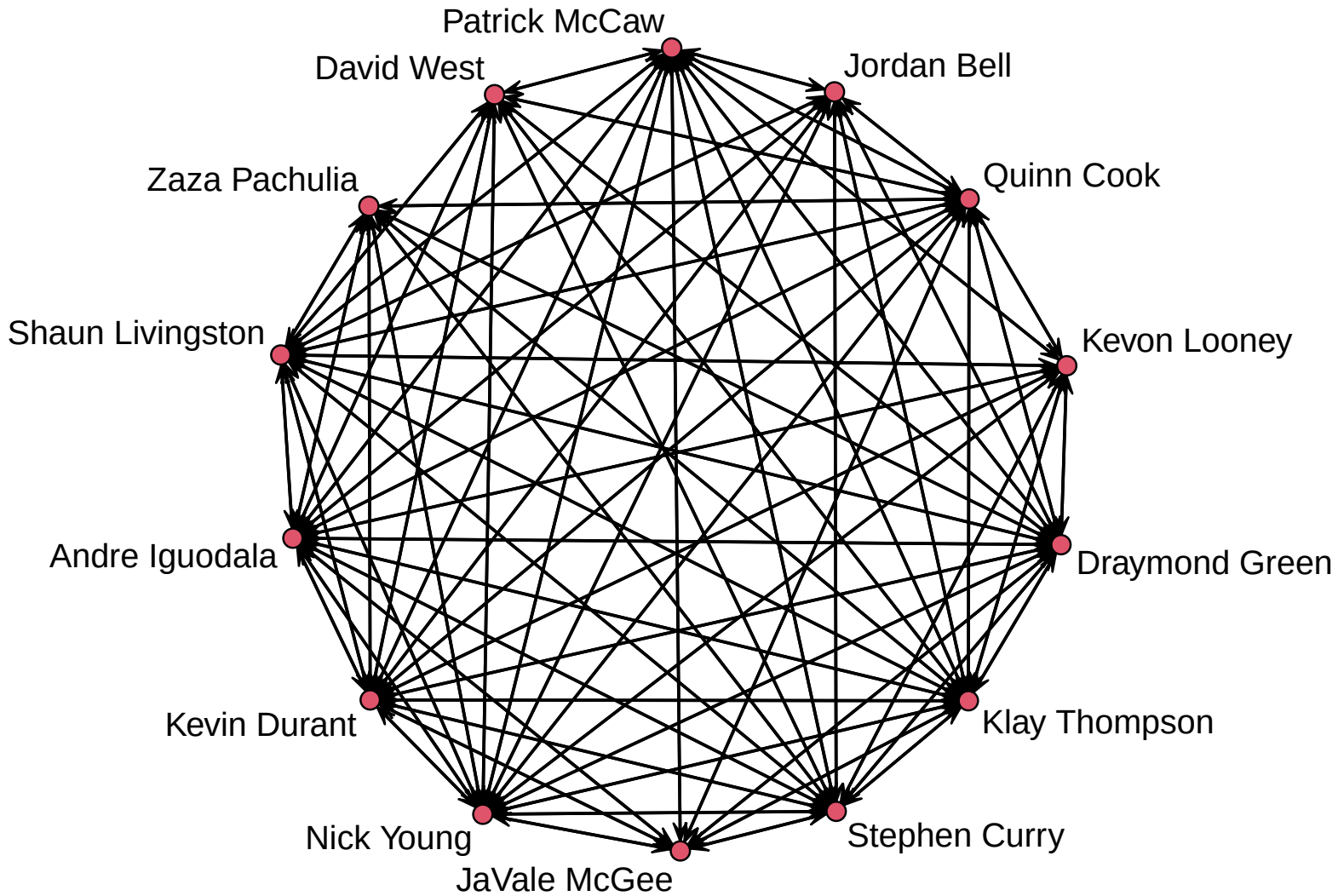
```
ggplot(passing, aes(x=PASS))+geom_histogram()+
  labs(x="Passes", title = "Distribution of passes")
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



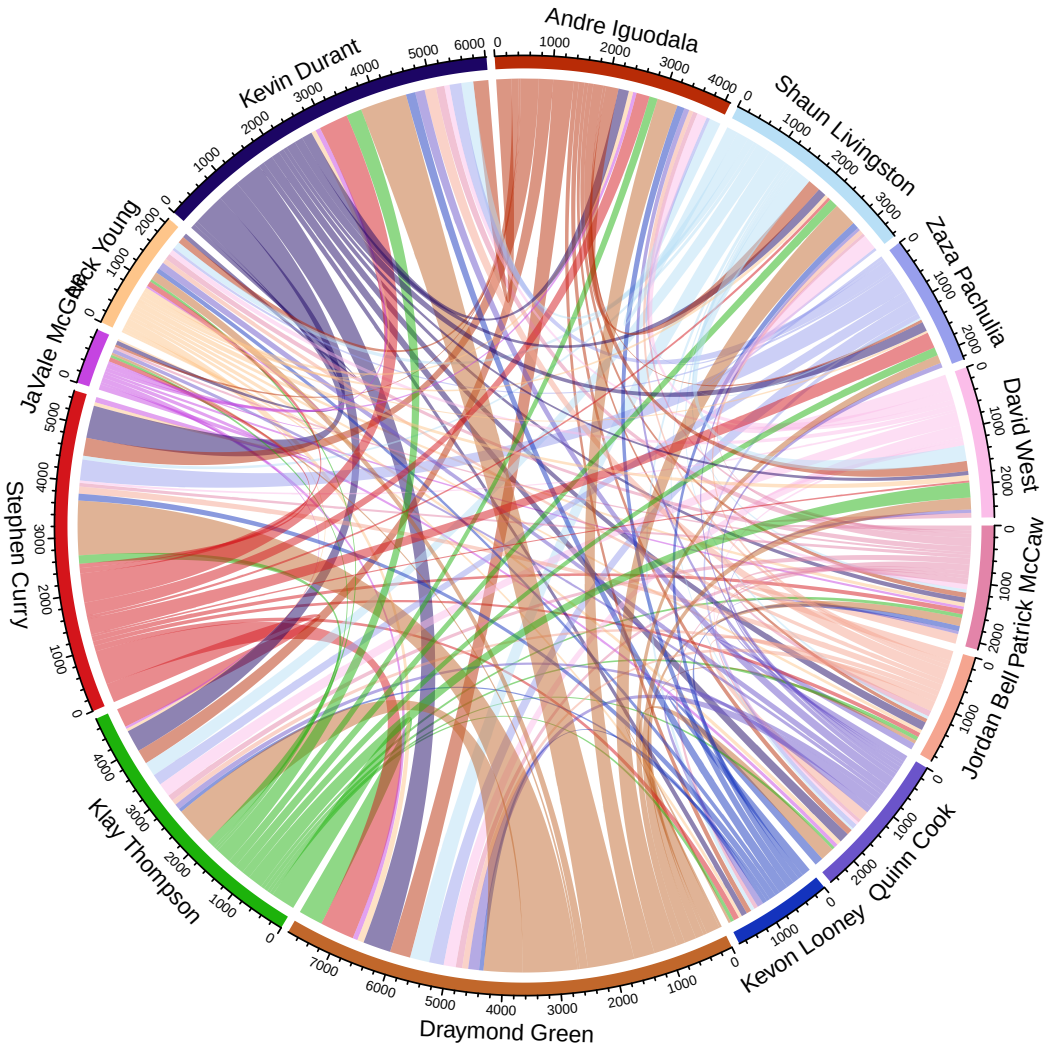
NBA: passing network

```
n_pass1<-get.inducedSubgraph(n_pass,
                             eid=which(n_pass %e% "passes" > 30))
plot(n_pass1, displaylabels=T, mode="circle")
```



NBA: passing network

```
mat <- as.matrix(n_pass1, matrix.type = "adjacency",
                 attrname="passes")
chordDiagram(mat)
```



NBA: passing network

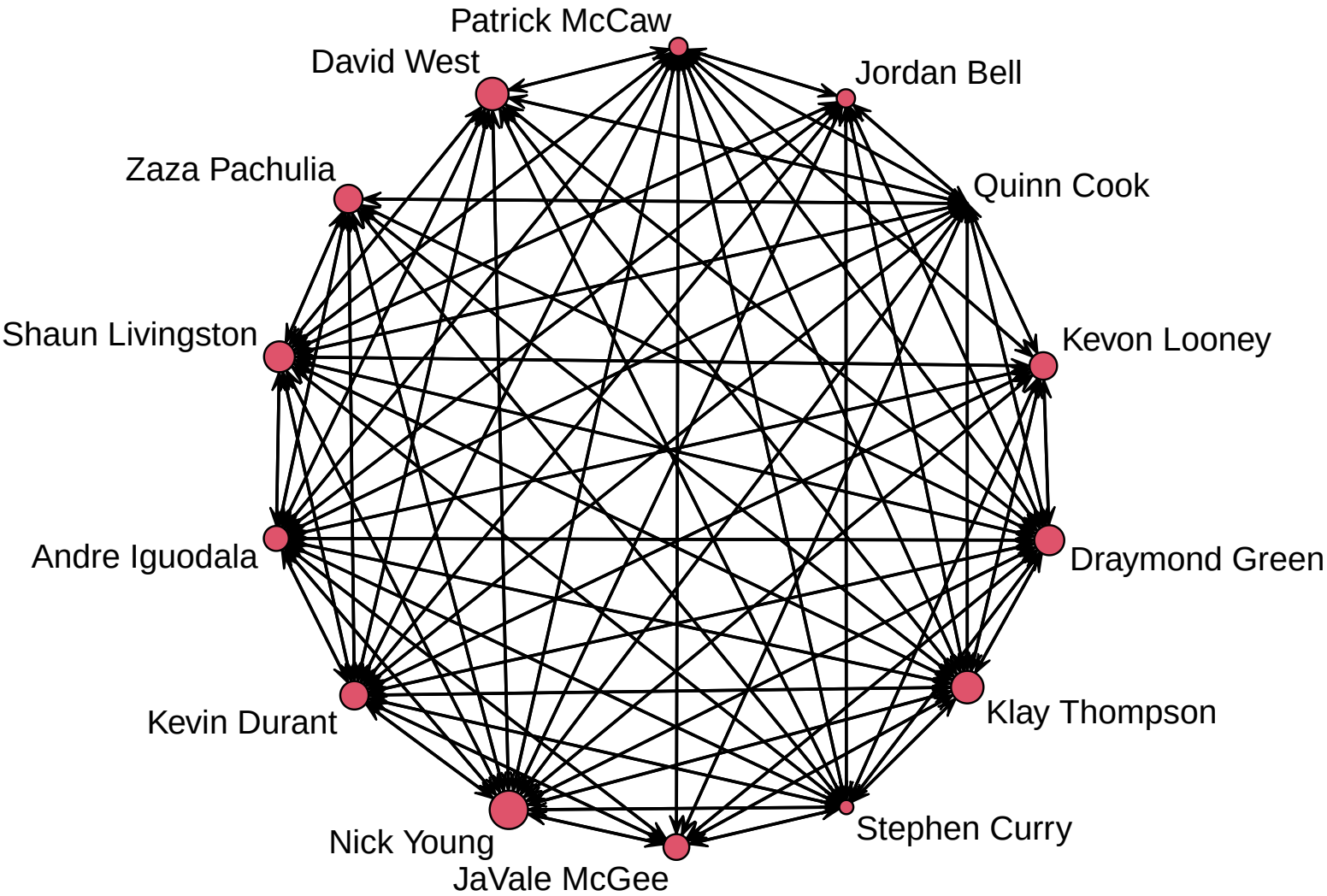
Lets plot the network with number of games played as vertex size

```
x <- n_pass1 %v% "ngames"
(z <- 2*(x - min(x))/(max(x)-min(x)))
```

```
##  [1] 0.9565217 0.9565217 0.0000000 1.4347826 1.5652174 1.6956522 0.7391304
##  [8] 1.3478261 2.0000000 1.4782609 1.3043478 1.6086957 1.4782609 1.6956522
```

NBA: passing network

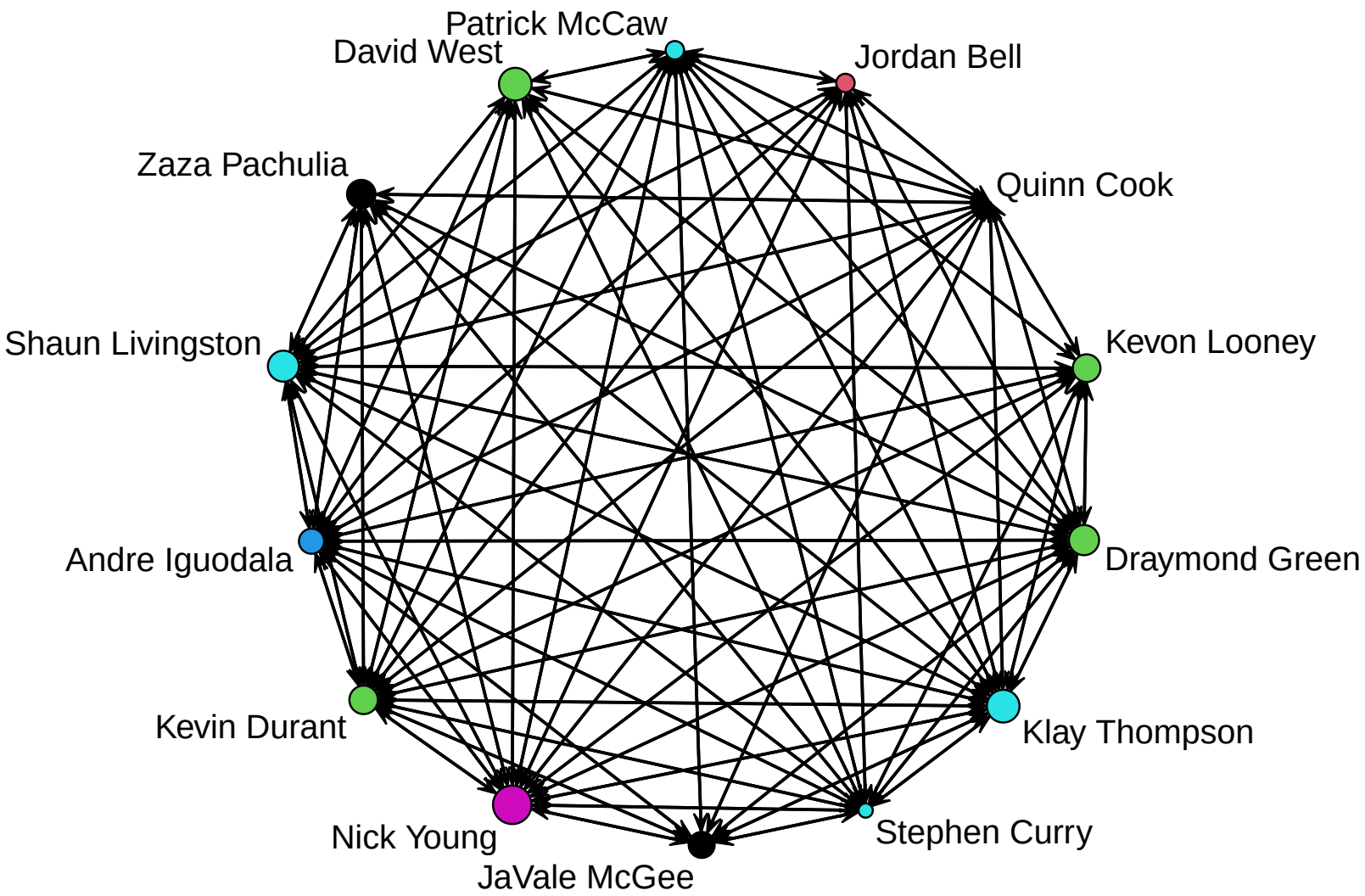
```
plot(n_pass1, displaylabels=T, mode="circle",
     vertex.cex = z)
```



NBA: passing network

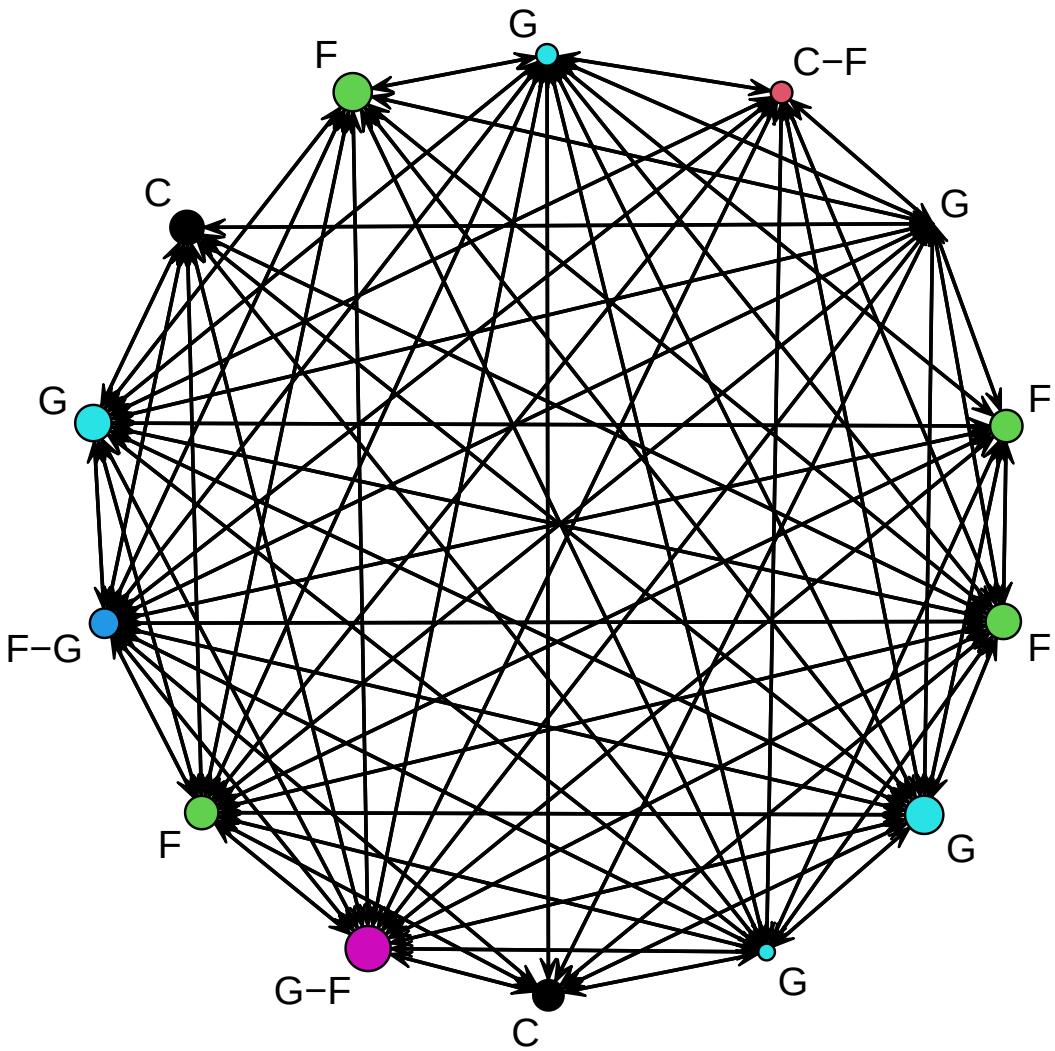
Colorize by player position

```
plot(n_pass1, displaylabels=T, mode="circle",
     vertex.cex = z, vertex.col = "position")
```



NBA: passing network

```
plot(n_pass1, displaylabels=T, mode="circle",
     vertex.cex = z, vertex.col = "position",
     label = n_pass1 %v% "position")
```



NBA: passing network



Kevin
Durant



Zaza
Pachulia



Draymond
Green



Stephen
Curry



Klay
Thompson

NBA: passing network

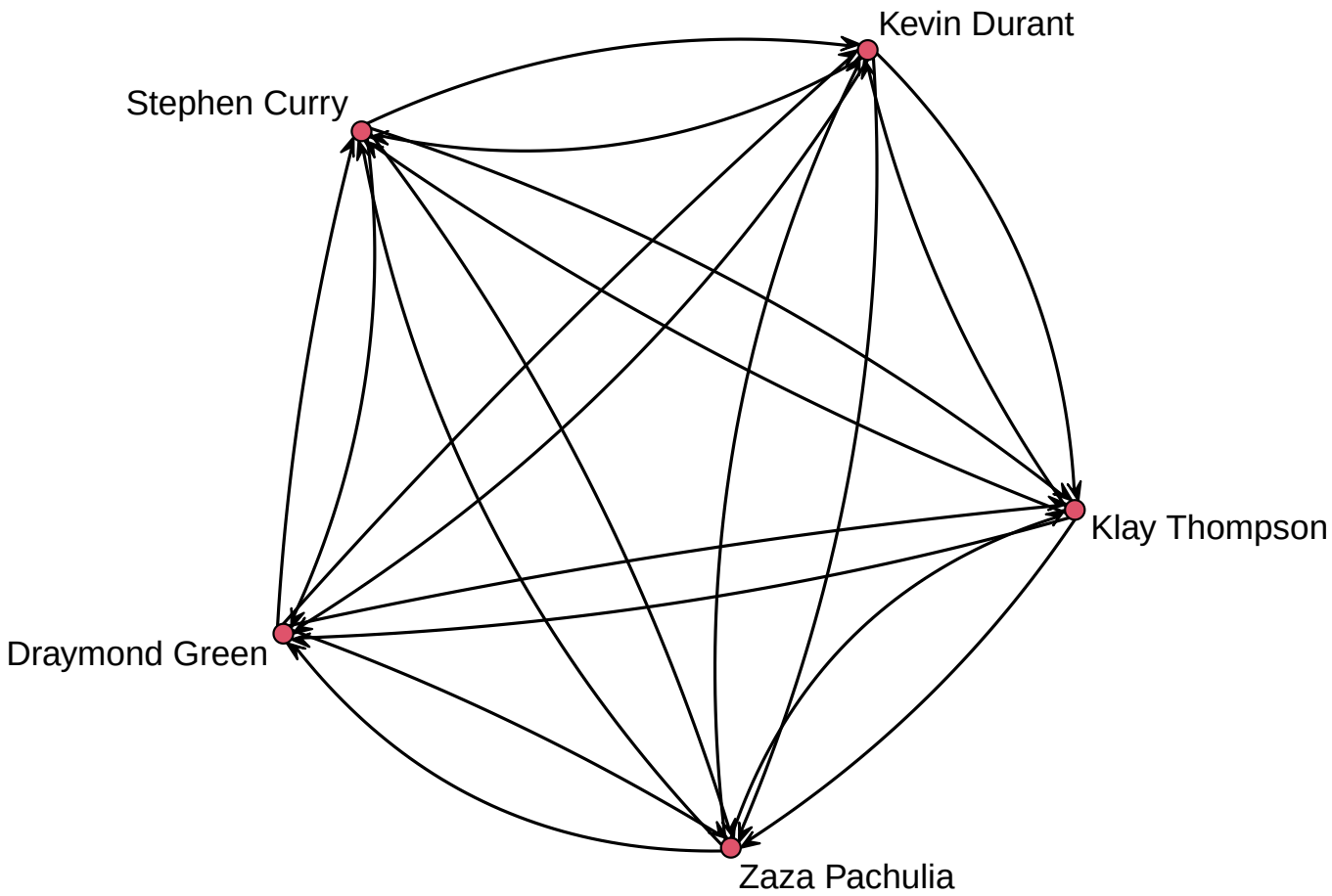
Create a new network with the given lineup

```
lineup <- c("Kevin Durant", "Zaza Pachulia",
            "Draymond Green", "Stephen Curry",
            "Klay Thompson")
n_pass2 <- get.inducedSubgraph(n_pass1,
            v=which(n_pass1 %v% "vertex.names"  %in% lineup))
```

NBA: passing network

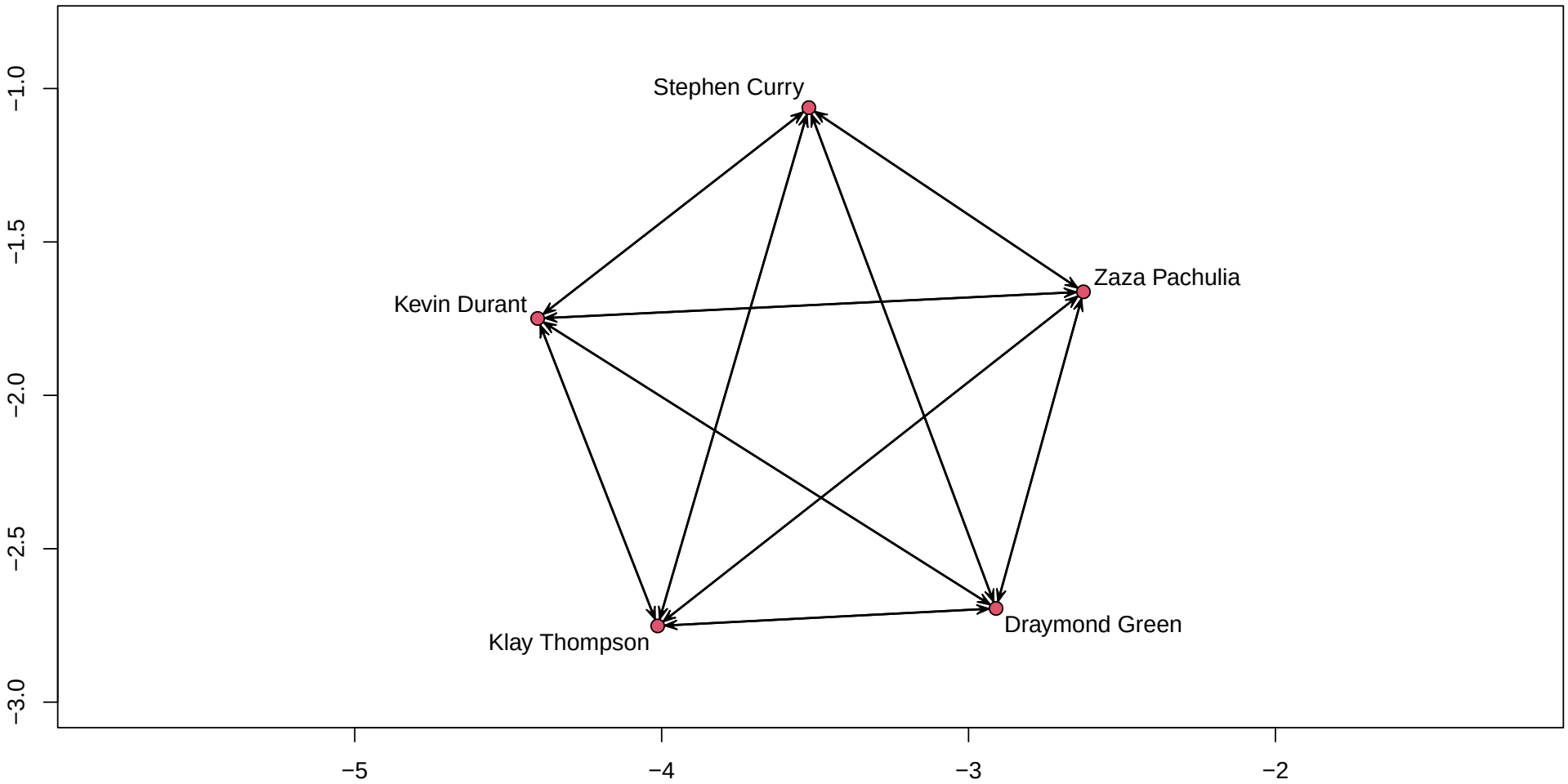
Plot with two additional arguments

```
plot(n_pass2, displaylabels=T,
      usecurve=T, edge.curve=0.025)
```



NBA: passing network

```
coords <- plot(n_pass2, displaylabels=T,
      suppress.axes = F)
```



NBA: passing network

We can try to change the coordinates

```
coords
```

```
##              cx        cy
## [1,] -2.910352 -2.694570
## [2,] -4.012951 -2.751592
## [3,] -3.520333 -1.062279
## [4,] -4.404068 -1.749408
## [5,] -2.625655 -1.662598
```

```
n_pass2 %v% "vertex.names"
```

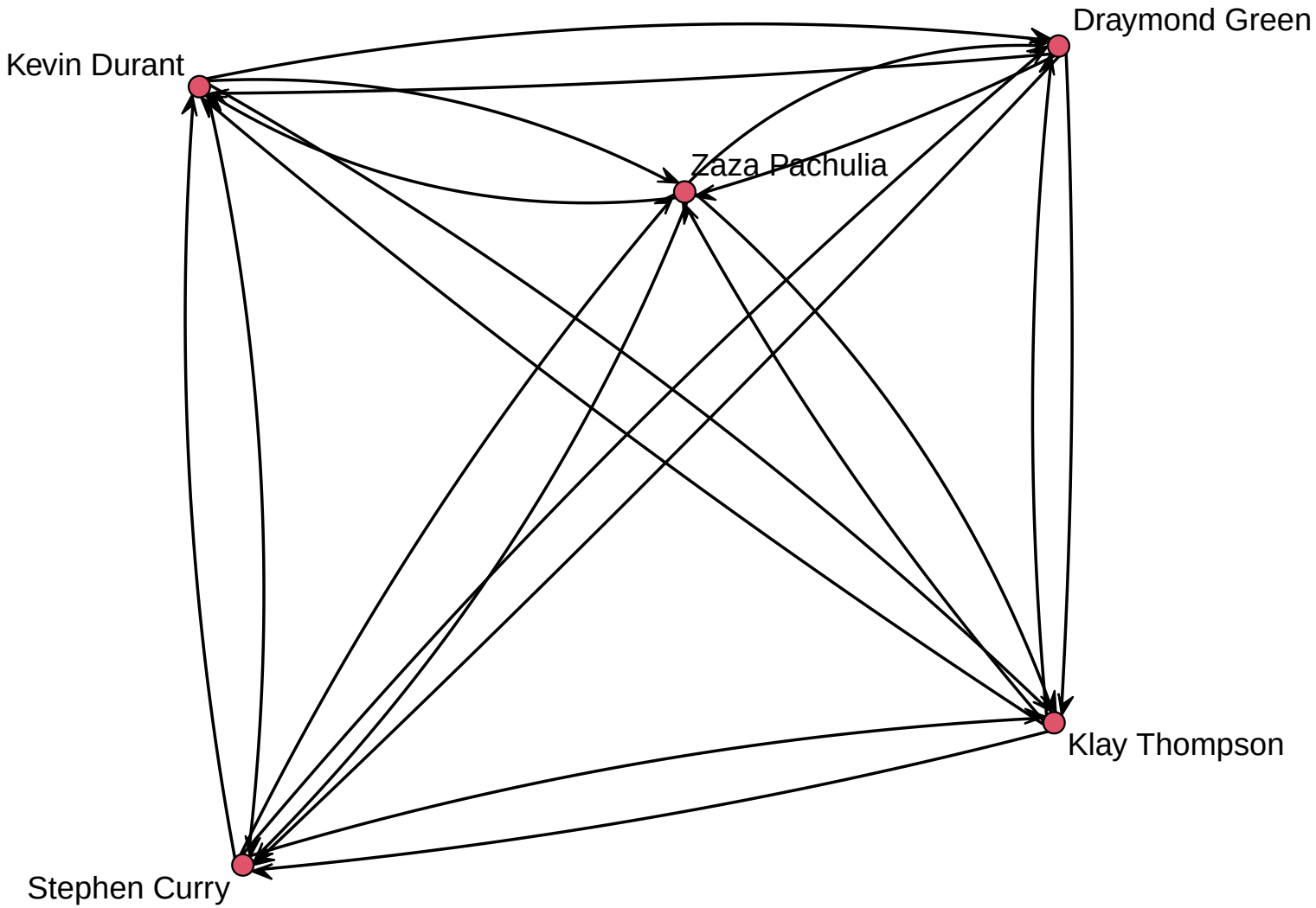
```
## [1] "Draymond Green" "Klay Thompson"  "Stephen Curry"  "Kevin Durant"
## [5] "Zaza Pachulia"
```

NBA: passing network

```
## "Draymond Green"
coords[1,] <- c(-2, -3)
## "Klay Thompson"
coords[2,] <- c(-2, -4.2)
## "Stephen Curry"
coords[3,] <- c(-3.5, -4.5)
## "Kevin Durant"
coords[4,] <- c(-3.6, -3)
## "Zaza Pachulia"
coords[5,] <- c(-2.7, -3.2)
```

NBA: passing network

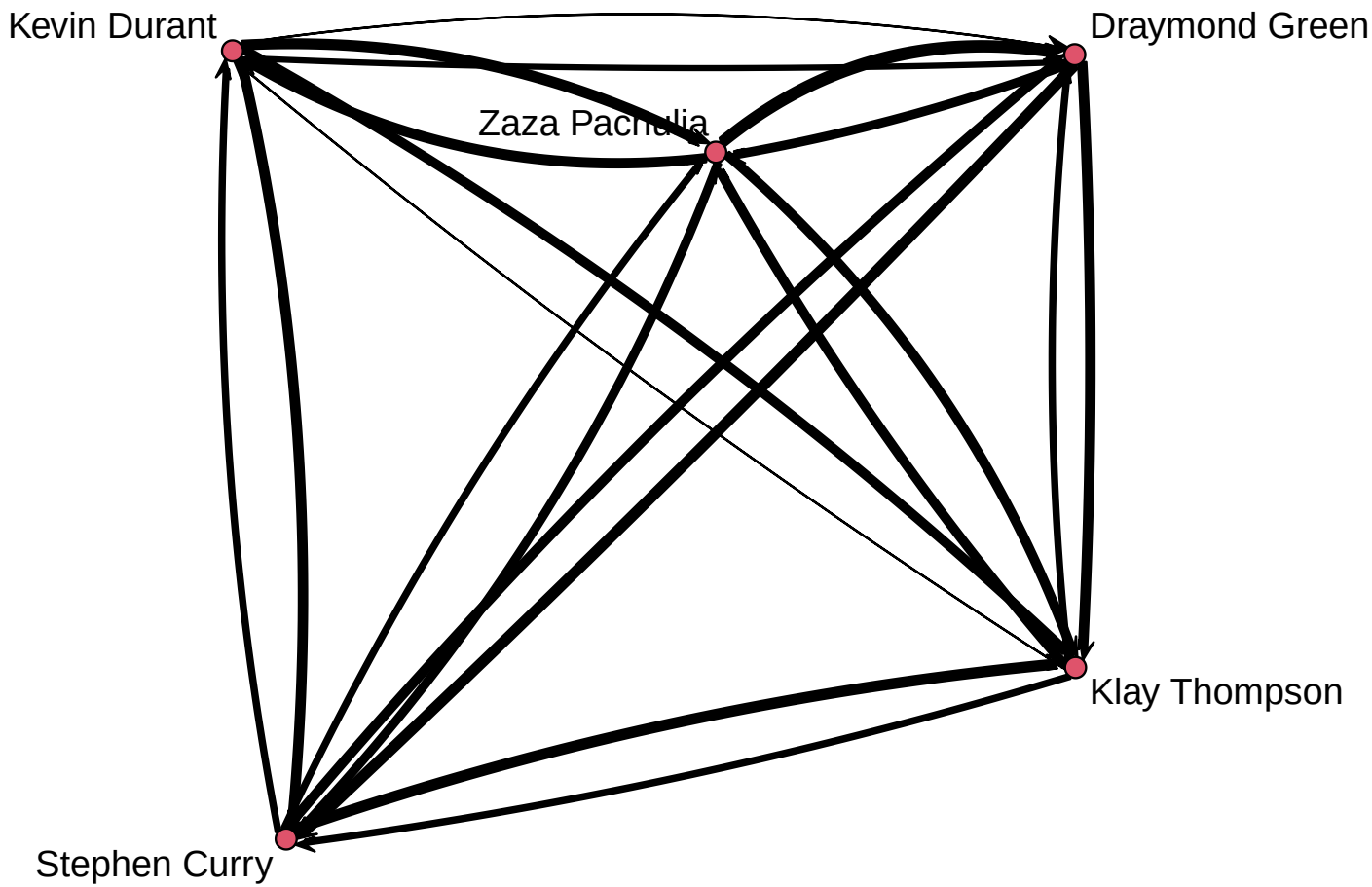
```
plot(n_pass2, displaylabels=T, coord = coords,
     usecurve=T, edge.curve=0.015)
```



NBA: passing network

Number of passes as edge attribute

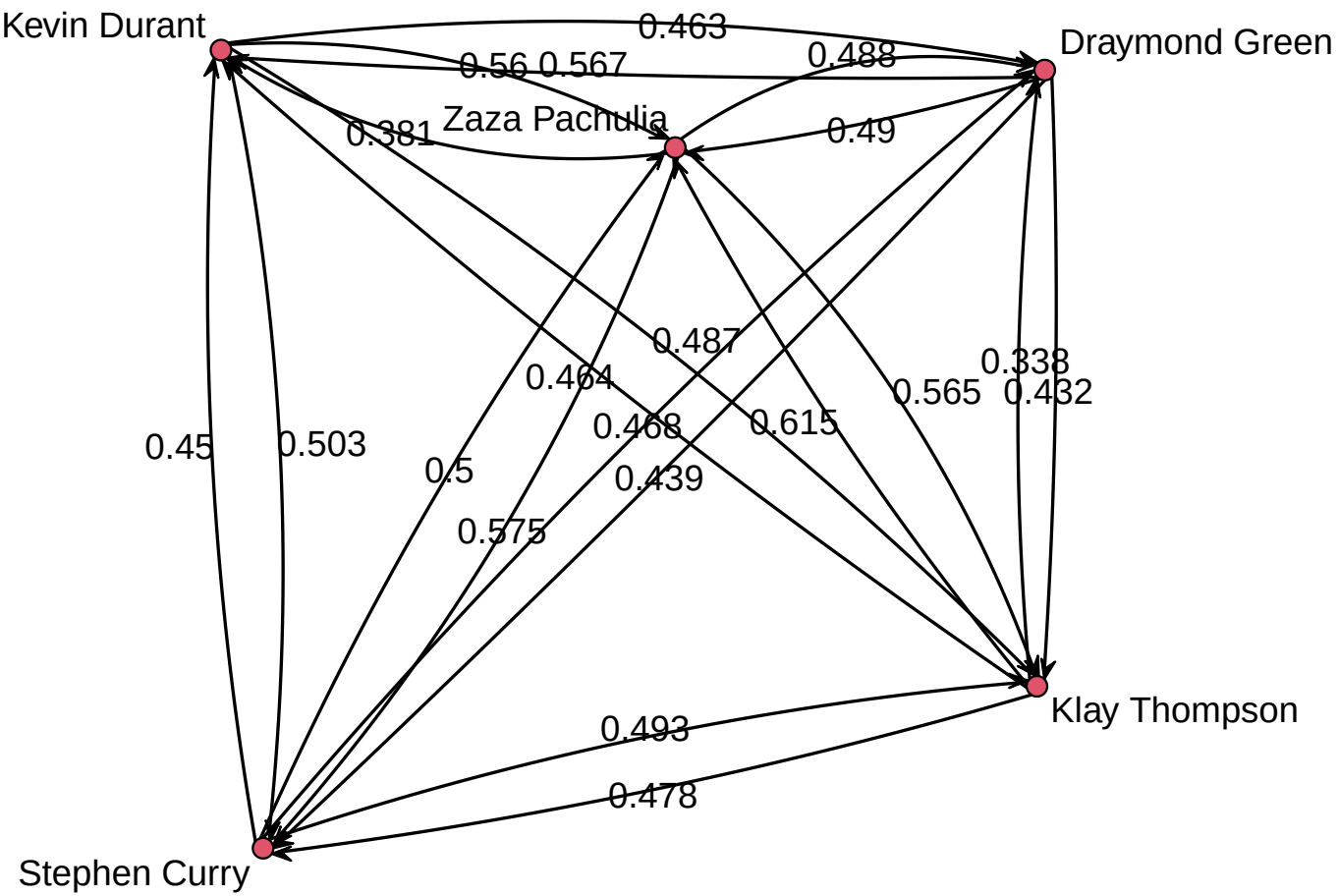
```
z <- 10*(x - min(x))/(max(x)-min(x))
plot(n_pass2, displaylabels=T, coord = coords,
     usecurve=T, edge.curve=0.015,
     edge.lwd = z)
```



NBA: passing network

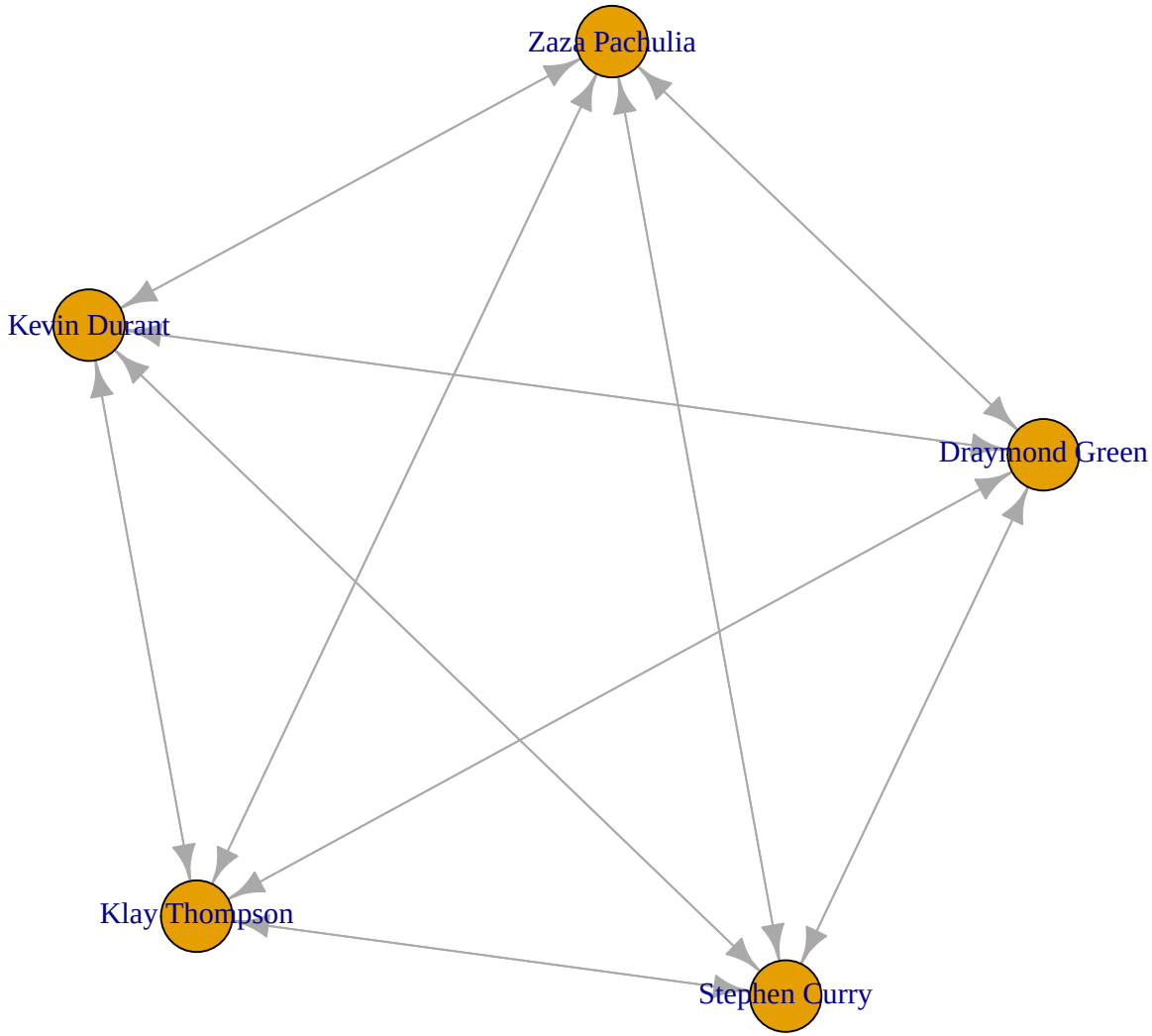
Field goal percentage in each edge

```
plot(n_pass2, displaylabels=T, coord = coords,
     usecurve=T, edge.curve=0.015,
     edge.label = n_pass2 %e% "FGP")
```



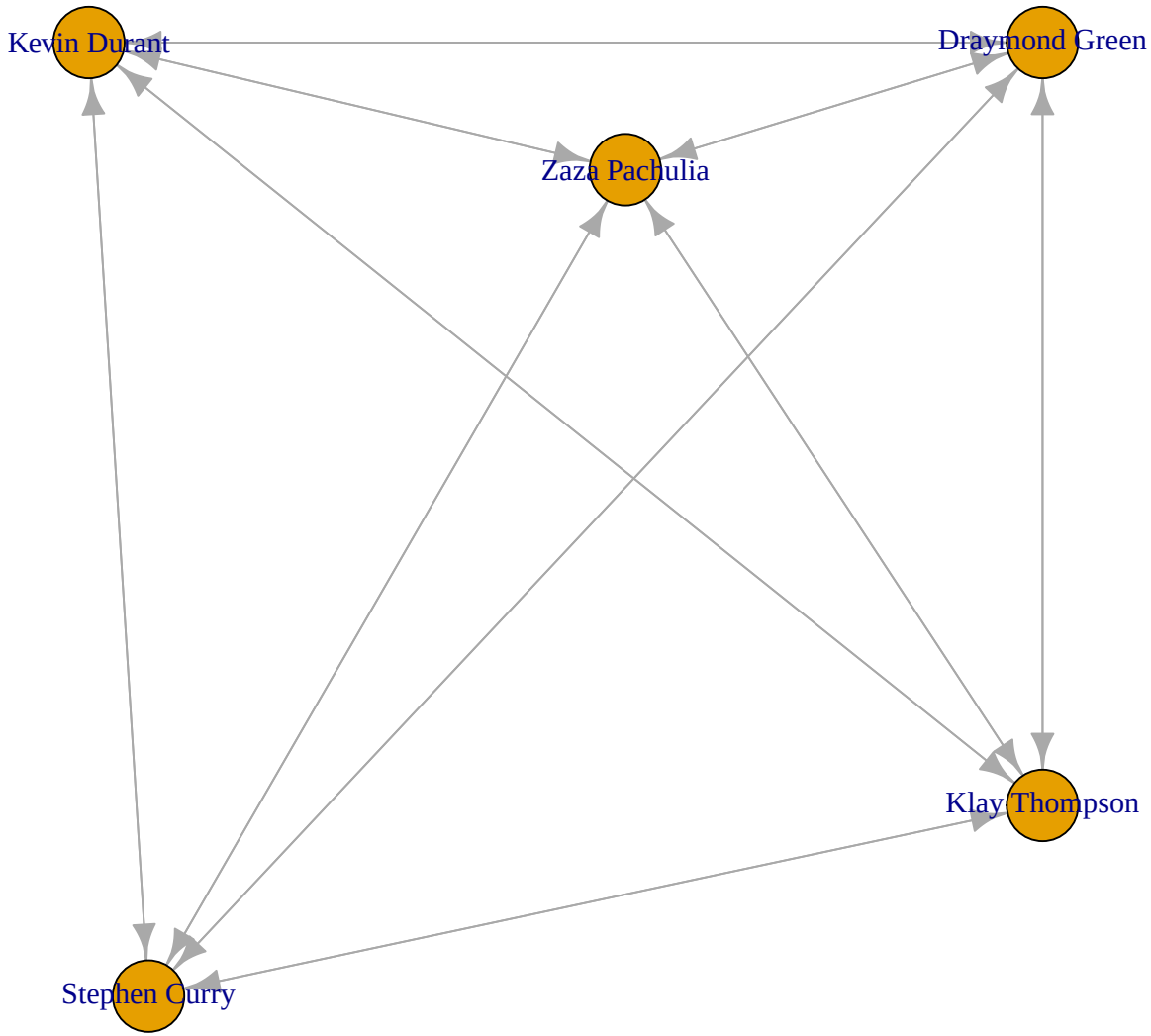
NBA: passing network

```
i_pass2 <- intergraph::asIgraph(n_pass2)
plot(i_pass2, vertex.label = V(i_pass2)$vertex.names)
```



NBA: passing network

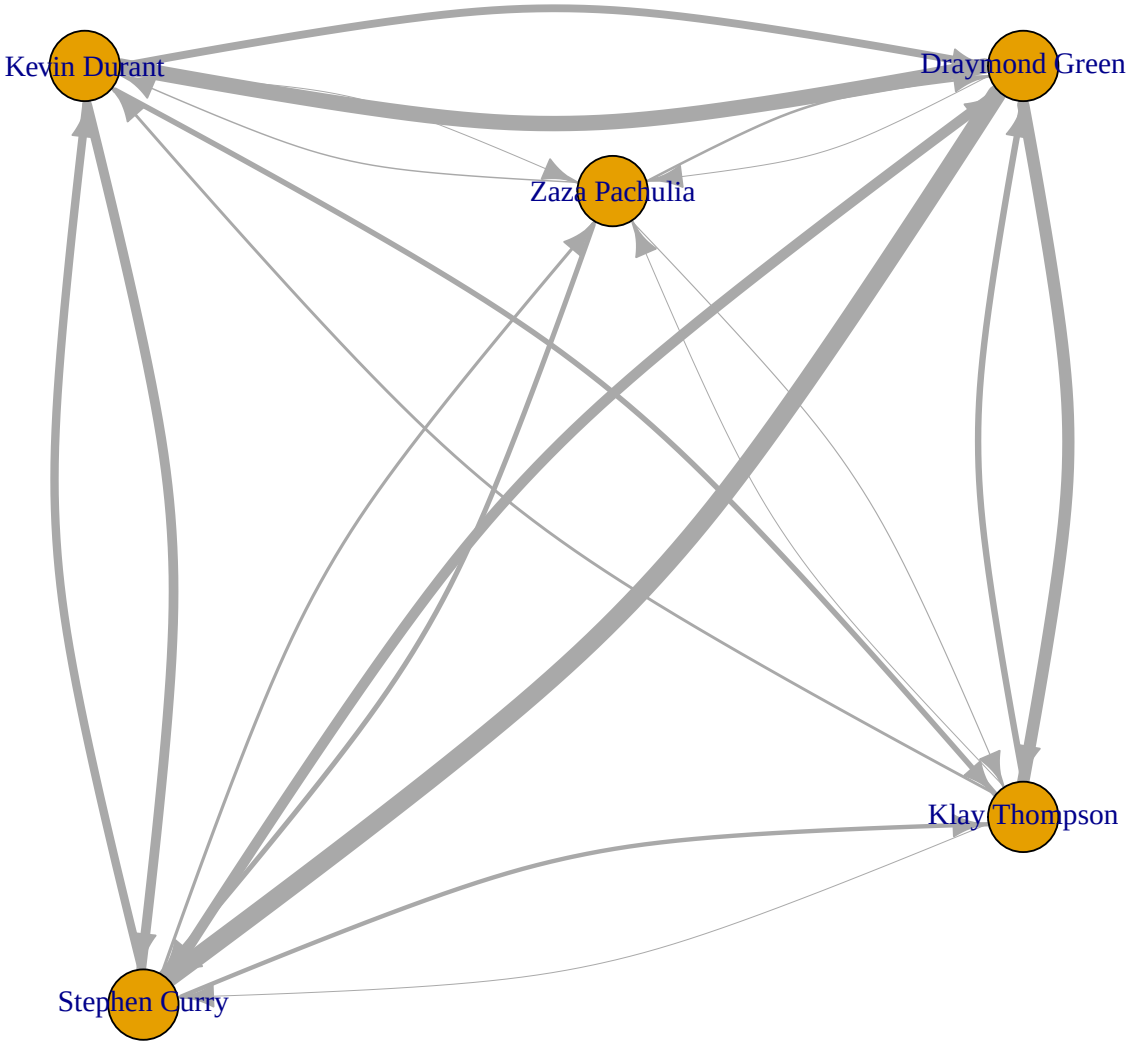
```
plot(i_pass2, vertex.label = V(i_pass2)$vertex.names,
     layout=coords)
```



NBA: passing network

```
x <- E(i_pass2)$passes
z <- 10*(x - min(x))/(max(x)-min(x))

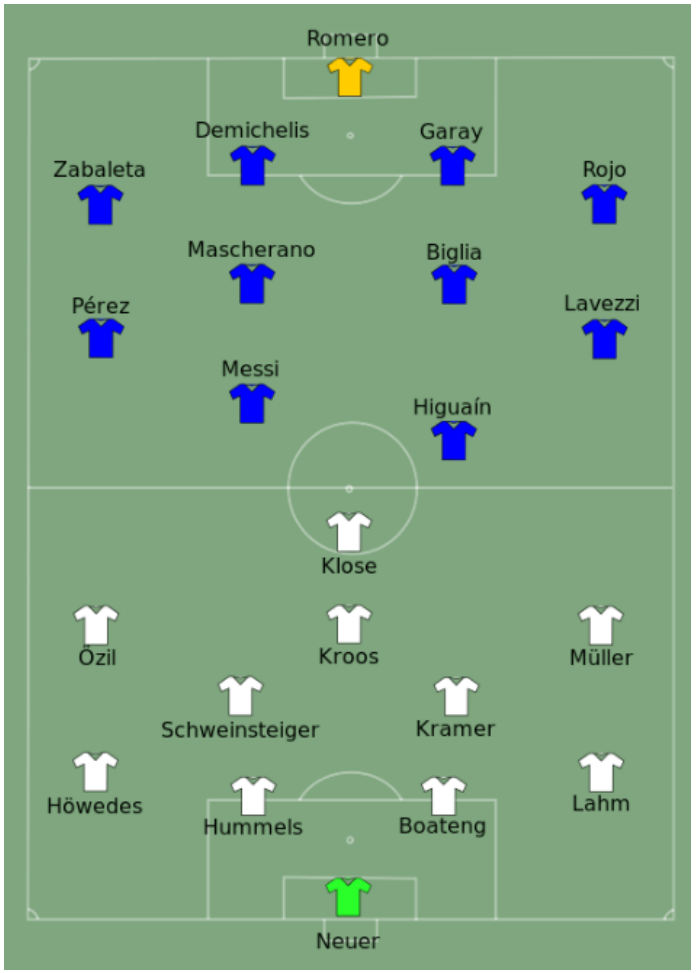
plot(i_pass2, vertex.label = V(i_pass2)$vertex.names,
     layout=coords, edge.width = z, edge.curved=0.2)
```



Passing in football

Passing in football

World cup final Germany 1-0 Argentina



::::

Passing in football

```
data("wc_14_final")
names(wc_14_final)
```

```
## [1] "argentina_team"    "argentina_passing" "germany_team"
## [4] "germany_passing"
```

The passing network is given in the form of adjacency matrix

```
dim(wc_14_final$argentina_passing)
```

```
## [1] 14 14
```

Passing in football

```
head(wc_14_final$argentina_team)
```

```
##       Player_name Shirt_number Position Played_minutes Long_completed
## 1   Sergio ROMERO            1       GK        129'09"              8
## 2  Ezequiel GARAY            2       DF        129'09"              8
## 3  Pablo ZABALETA            4       DF        129'09"              3
## 4    Lucas BIGLIA            6       MF        129'09"              7
## 5      Enzo PEREZ            8       MF         87'59"              1
## 6 Gonzalo HIGUAIN            9       FW         79'23"              0
##   Long_attempted Medium_completed Medium_attempted Short_completed
## 1             30               11                11               1
## 2             10               30                35               5
## 3              8               33                39              11
## 4              9               30                38               6
## 5              1               16                20               9
## 6              0                7                10               7
##   Short_attempted Pass_completed Pass_attempted
## 1               1              20              42
## 2               7              43              52
## 3              17              47              64
## 4              11              43              58
## 5              14              26              35
## 6               8              14              18
```

Passing in football

```
i_arg <- igraph::graph.adjacency(as.matrix(wc_14_final$argentina_passing),
                                 mode="directed",weighted=TRUE,diag=FALSE)
net_arg <- intergraph::asNetwork(i_arg)
```

```
list.edge.attributes(net_arg)
```

```
## [1] "na"     "weight"
```

```
list.vertex.attributes(net_arg)
```

```
## [1] "na"           "vertex.names"
```

Passing in football

Adding vertex attributes

Player positions

```
network::set.vertex.attribute(net_arg, "player.name",
          wc_14_final$argentina_team$Player_name)
get.vertex.attribute(net_arg, "player.name")
```

```
##  [1] "Sergio ROMERO"     "Ezequiel GARAY"    "Pablo ZABALETA"
##  [4] "Lucas BIGLIA"      "Enzo PEREZ"        "Gonzalo HIGUAIN"
##  [7] "Lionel MESSI"      "Javier MASCHERANO" "Martin DEMICHELIS"
## [10] "Marcos ROJO"       "Ezequiel LAVEZZI"  "Fernando GAGO"
## [13] "Rodrigo PALACIO"   "Sergio AGUERO"
```

Passing in football

Player positions

```
network::set.vertex.attribute(net_arg, "player.position",
        as.character(wc_14_final$argentina_team$Position))
get.vertex.attribute(net_arg, "player.position")
```

```
##  [1] "GK" "DF" "DF" "MF" "MF" "FW" "FW" "MF" "DF" "DF" "FW" "MF" "FW" "FW"
```

Passing in football

Number of completed passes and completion %

```
network::set.vertex.attribute(net_arg, "passes.completed",
                wc_14_final$argentina_team$Pass_completed)
```

```
wc_14_final$argentina_team$Completion <- wc_14_final$argentina_team$Pass_completed/
  wc_14_final$argentina_team$Pass_attempted
```

```
network::set.vertex.attribute(net_arg, "completion",
                wc_14_final$argentina_team$Completion)
get.vertex.attribute(net_arg, "completion")
```

```
##  [1] 0.4761905 0.8269231 0.7343750 0.7413793 0.7428571 0.7777778 0.6086957
##  [8] 0.7837838 0.8363636 0.6885246 0.3684211 0.8333333 0.5000000 0.6071429
```

Passing in football

Create a subgraph with starting 11 only

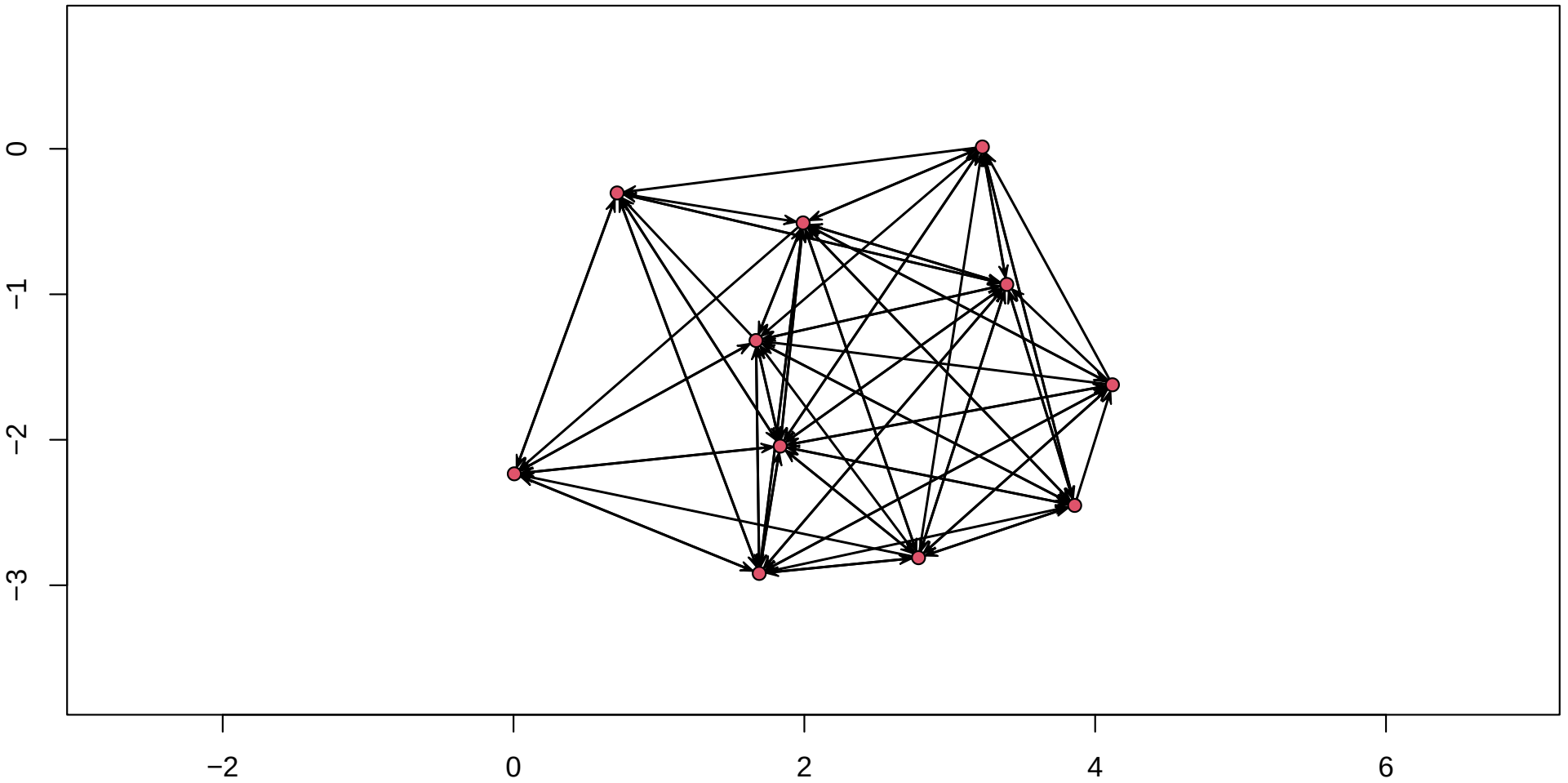
```
net_st <- get.inducedSubgraph(net_arg, v=1:11)
get.vertex.attribute(net_st, "player.name")
```

```
##  [1] "Sergio ROMERO"     "Ezequiel GARAY"    "Pablo ZABALETA"
##  [4] "Lucas BIGLIA"      "Enzo PEREZ"        "Gonzalo HIGUAIN"
##  [7] "Lionel MESSI"      "Javier MASCHERANO" "Martin DEMICHELIS"
## [10] "Marcos ROJO"       "Ezequiel LAVEZZI"
```

Passing in football

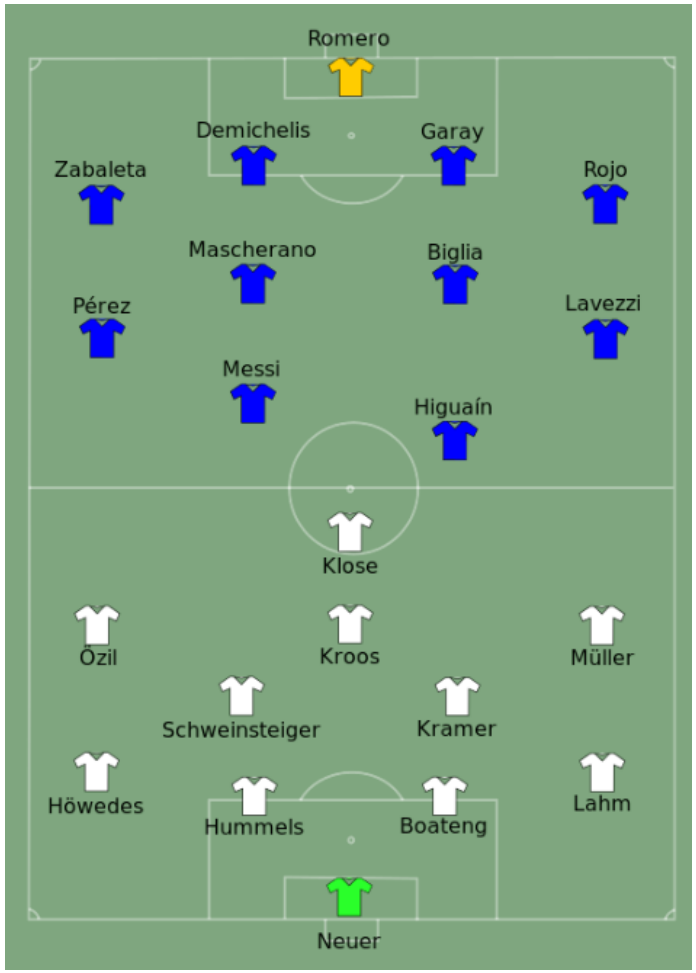
- Random layout
- Save the coordinates

```
set.seed(1)
coords <- plot(net_st, suppress.axes=F)
```



Passing in football

Change the coordinates to get approximate positions of the players on the field

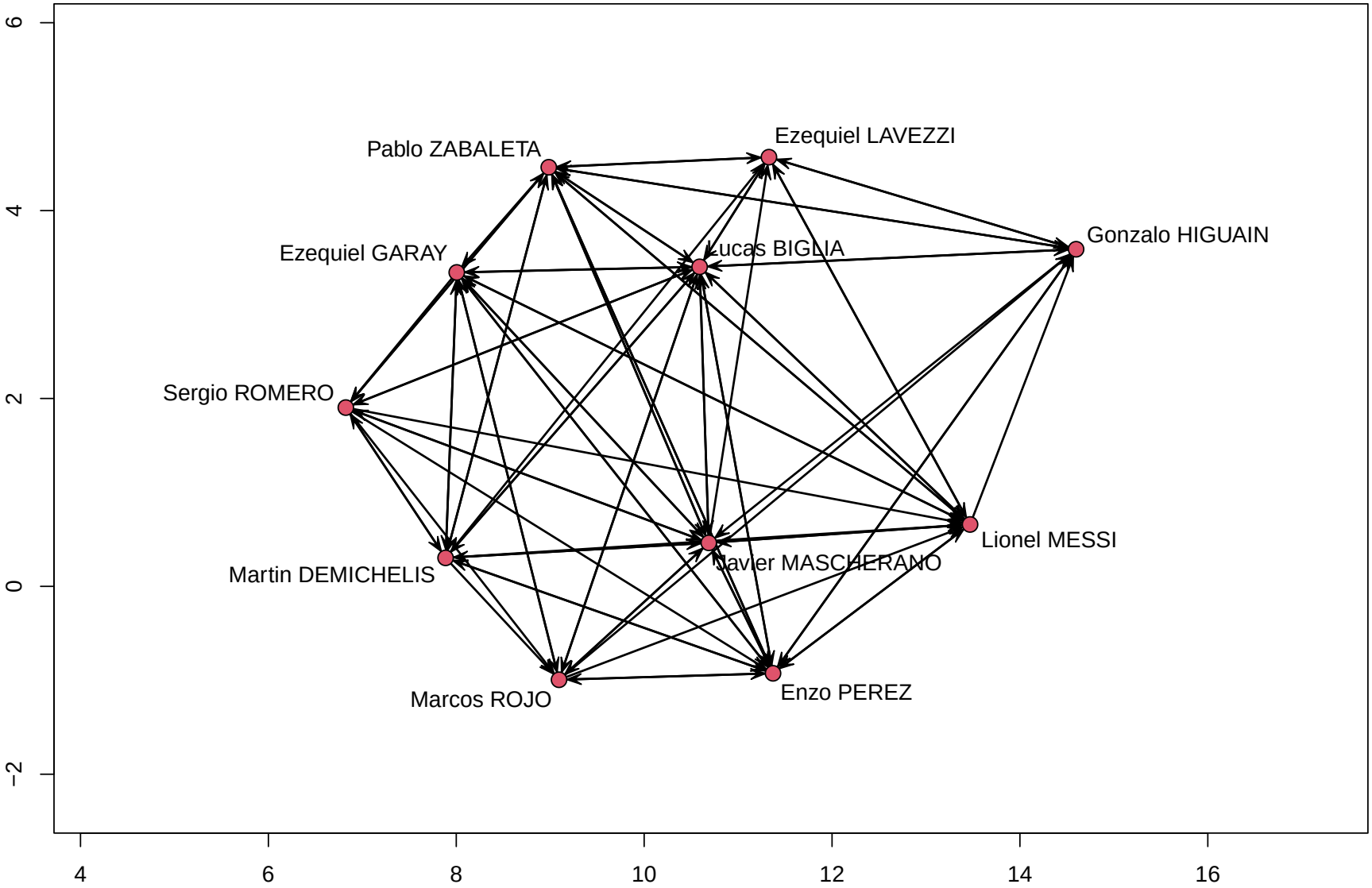


Passing in football

```
## The goalkeeper
coords[1,] <- c(7,2)
## Garay
coords[2,] <- c(8, 3.5)
## Demichelis
coords[9,] <- c(8, 0.5)
## Zabaleta
coords[3,] <- c(9,4.5)
## Rojo
coords[10,] <- c(9, -1)
## Biglia
coords[4,] <- c(10.5, 3.5)
## Mascherano
coords[8,] <- c(10.5, 0.5)
## Perez
coords[5,] <- c(11.5, -1)
## Lavezzi
coords[11,] <- c(11.5,4.5)
#Messi
coords[7,] <- c(13.5, 0.5)
## Higuain
coords[6,] <- c(14.5, 3.5)
```

Passing in football

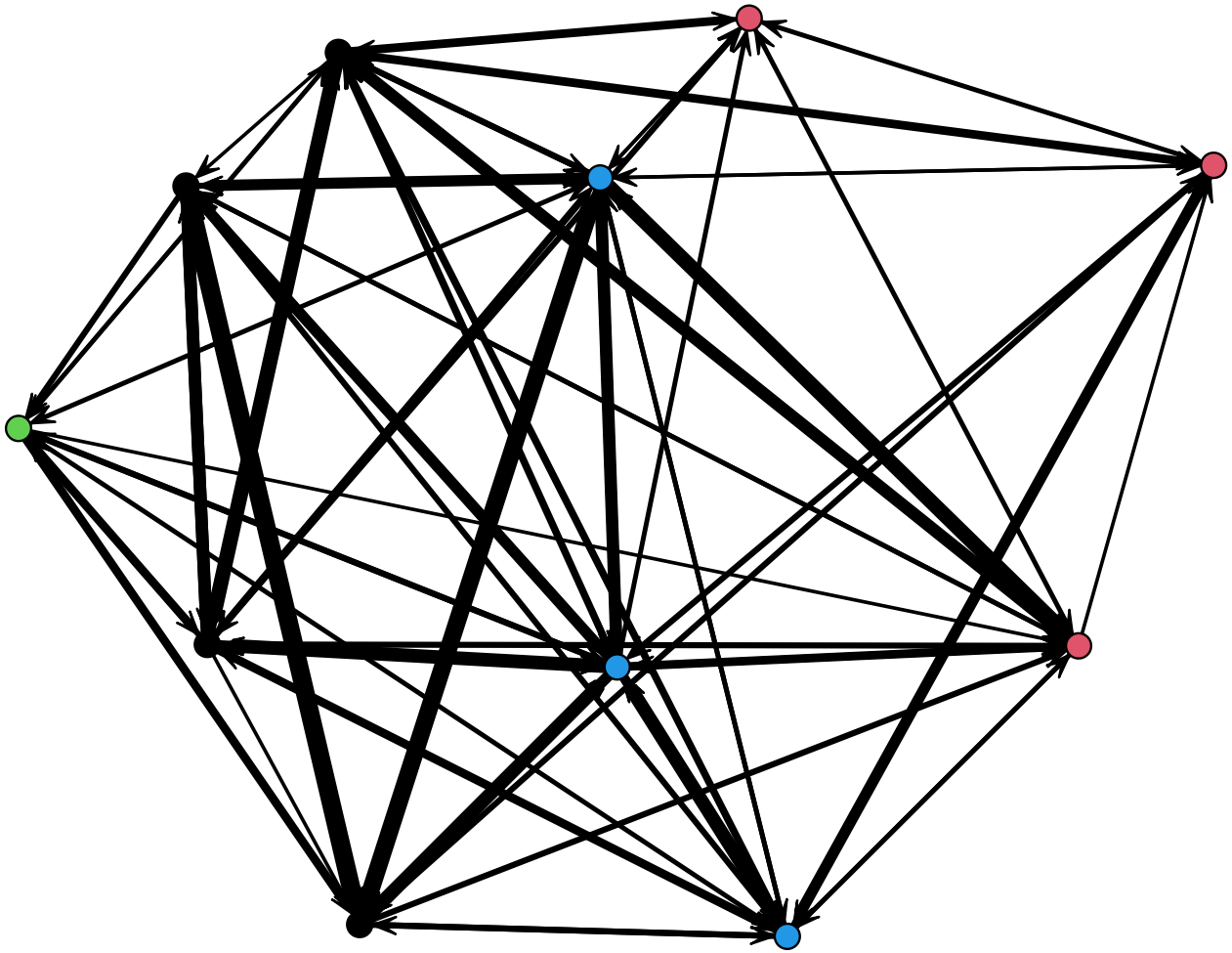
```
plot(net_st, coord=coords, suppress.axes=F,
     label = net_st %v% "player.name")
```



Passing in football

Passing density

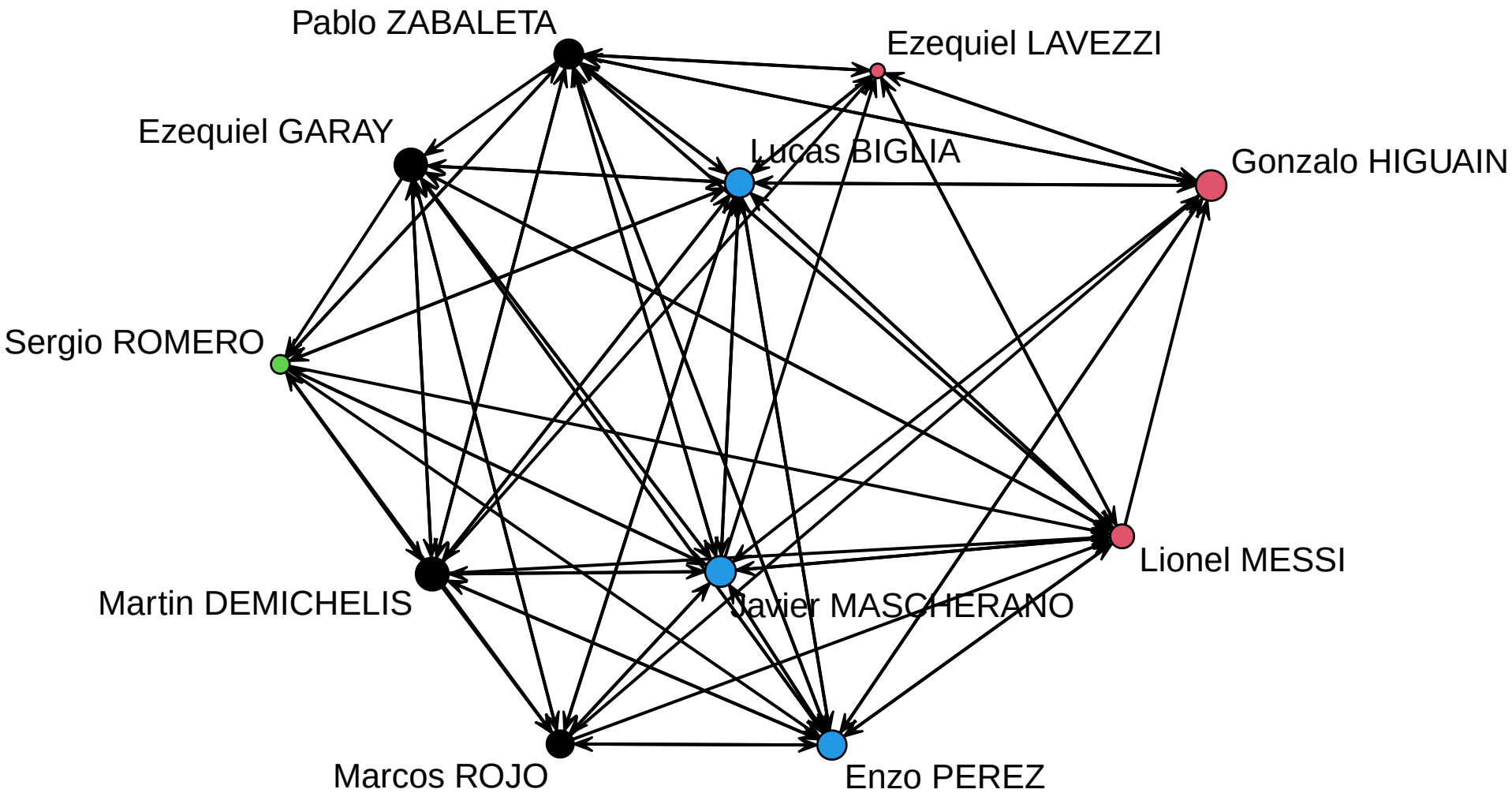
```
plot(net_st, vertex.col="player.position", edge.lwd="weight",
     coord=coords)
```



Passing in football

Completion rate

```
plot(net_st, vertex.col="player.position",
     coord=coords, vertex.cex = 2*(net_st %v% "completion"),
     label = net_st %v% "player.name")
```



Passing in football

Last: create a chord diagram

```
st_mat <- as.matrix(net_st, matrix.type = "adjacency", attrname = "weight")
colnames(st_mat) <- net_st %v% "player.name"
rownames(st_mat) <- colnames(st_mat)
```

Passing in football

```
chordDiagram(st_mat, directional = T)
```

